



UNIVERSITY OF TARTU



Enterprise System Integration (MTAT.03.229)

LECTURE 4: THE MICROSERVICES ARCHITECTURE

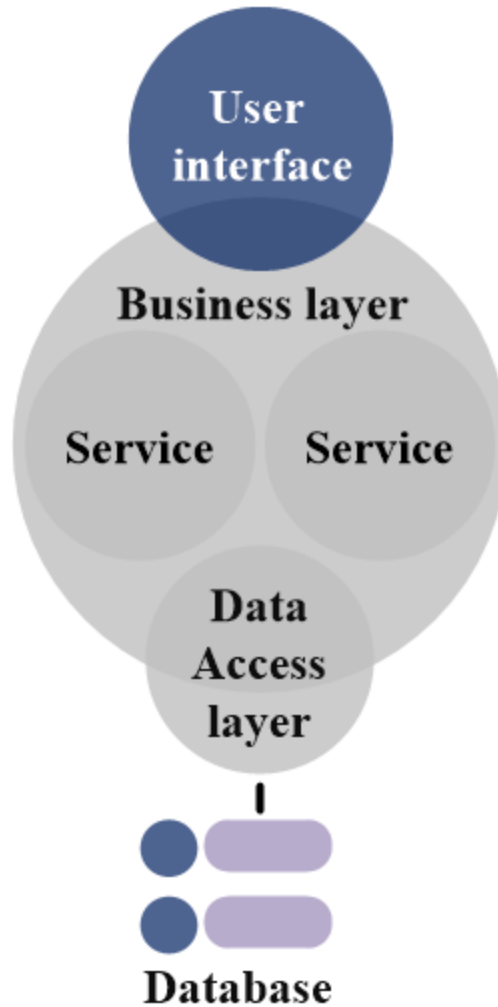
**MOHAMAD GHARIB
UNIVERSITY OF TARTU**

Monolithic vs SOA vs Mservices arch.

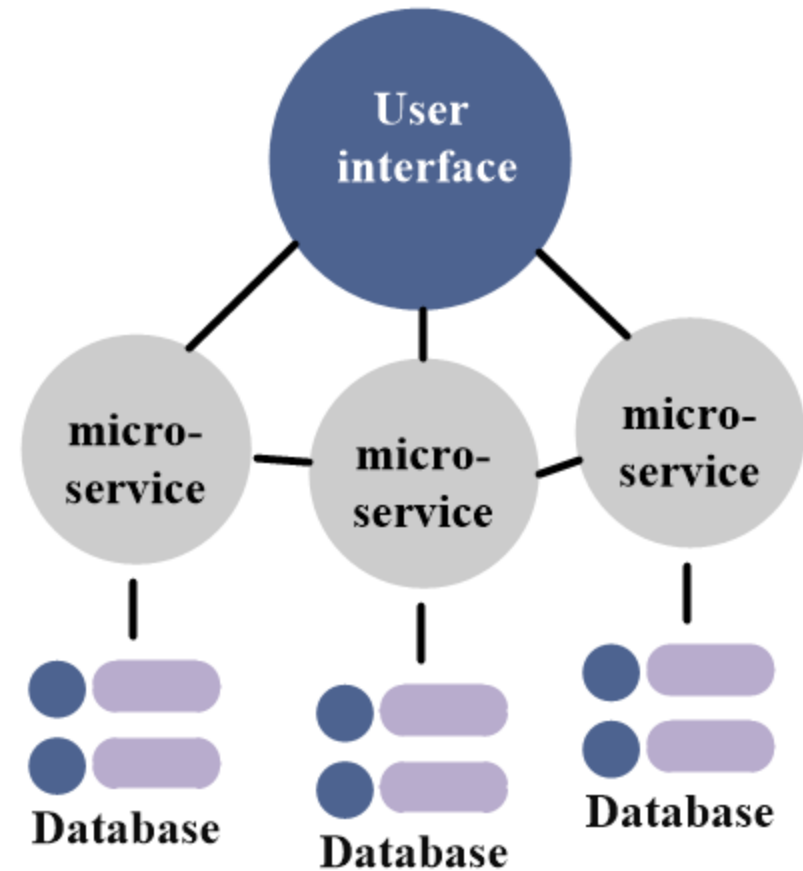
Monolithic architecture



SOA architecture

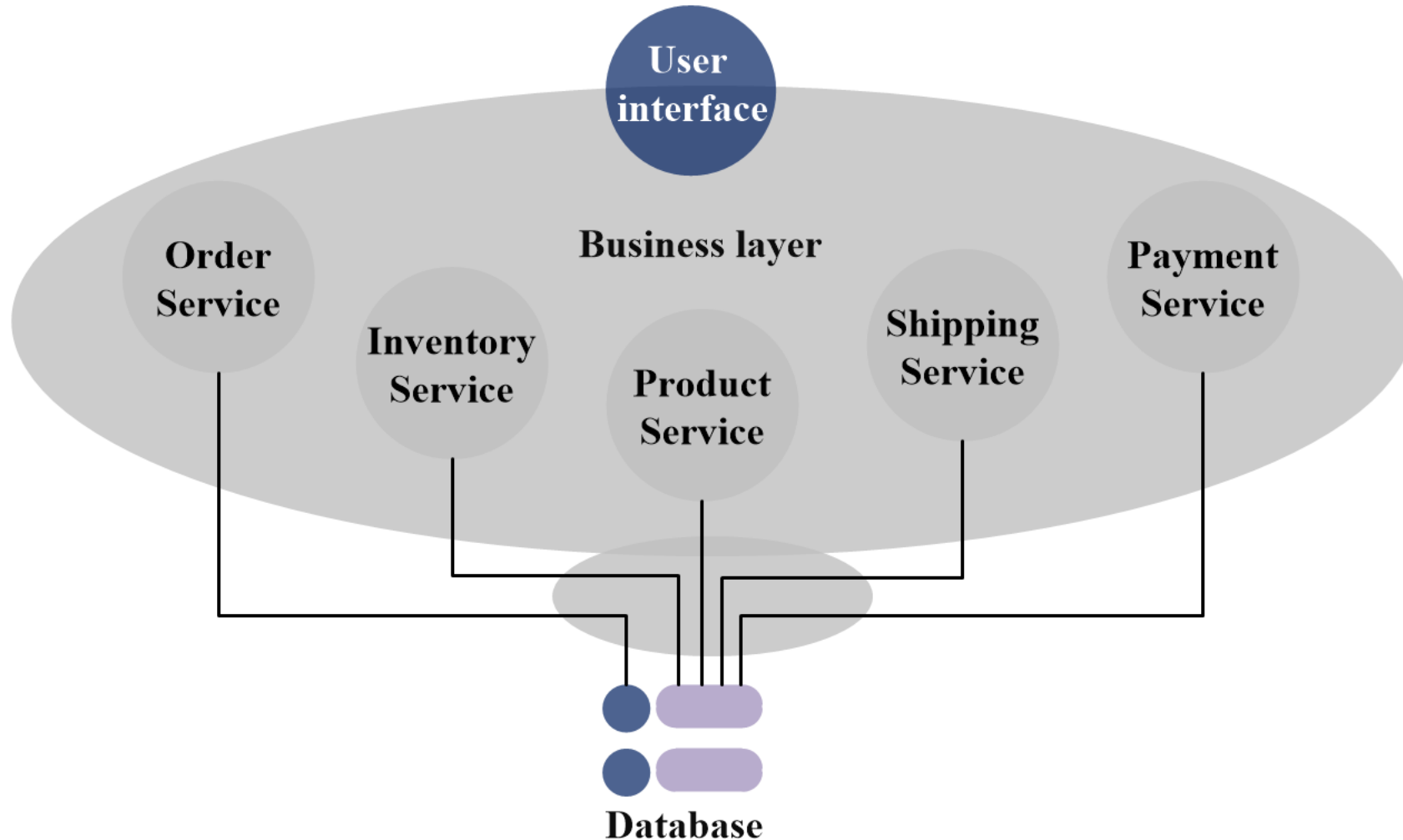


Micro-service architecture

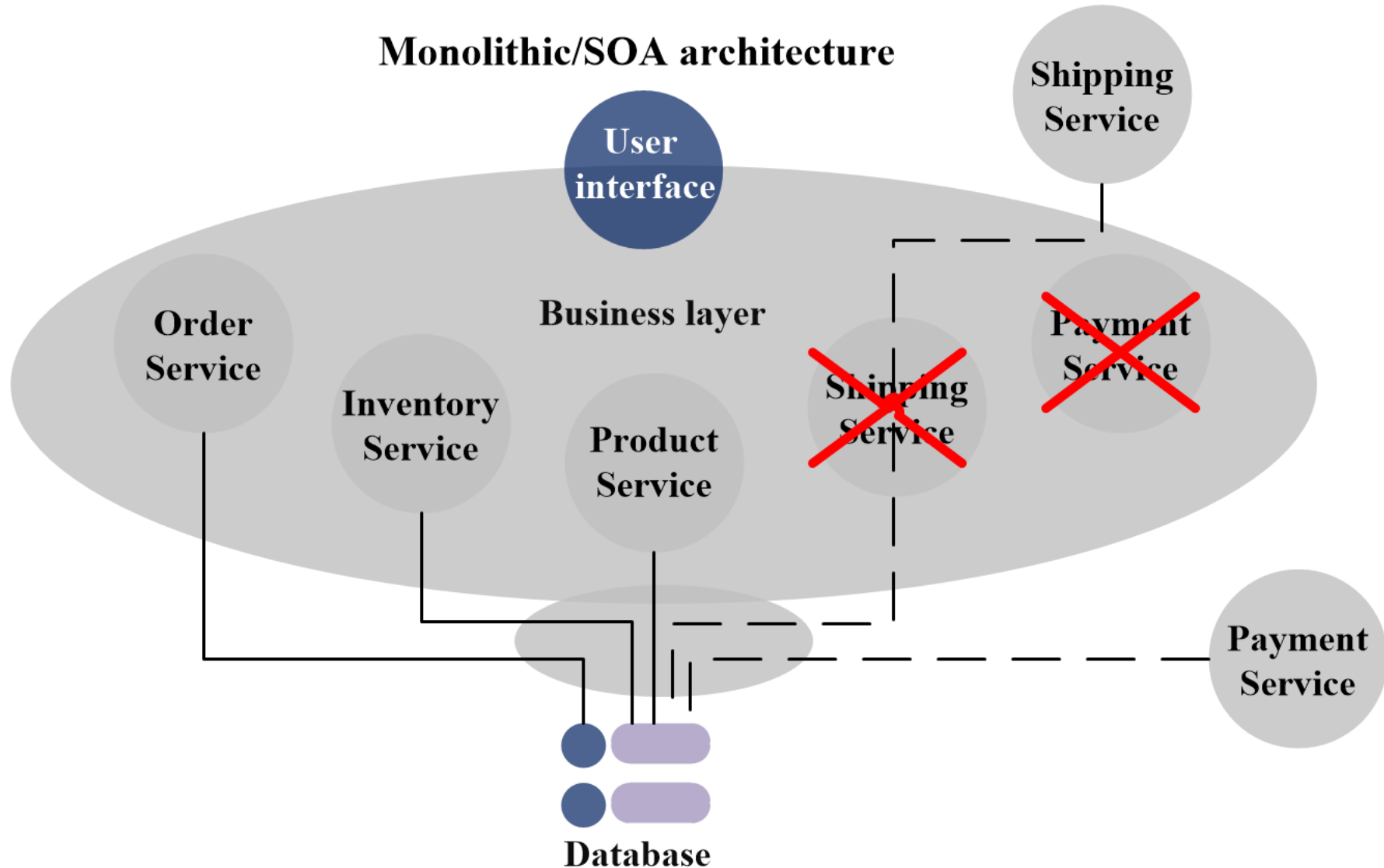


Monolithic vs SOA vs Mservices arch.

Monolithic/SOA architecture

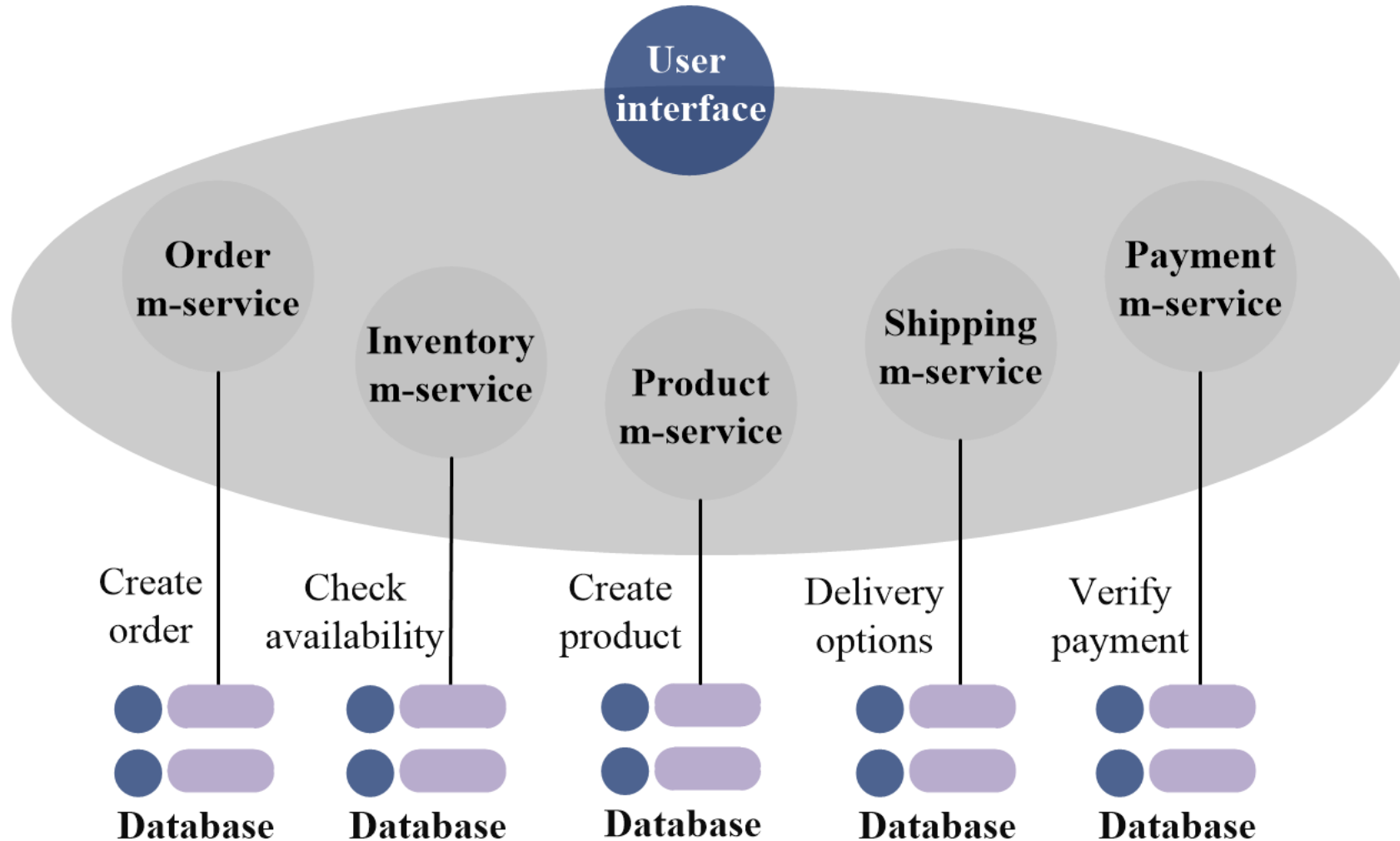


Monolithic vs SOA vs Mservices arch.



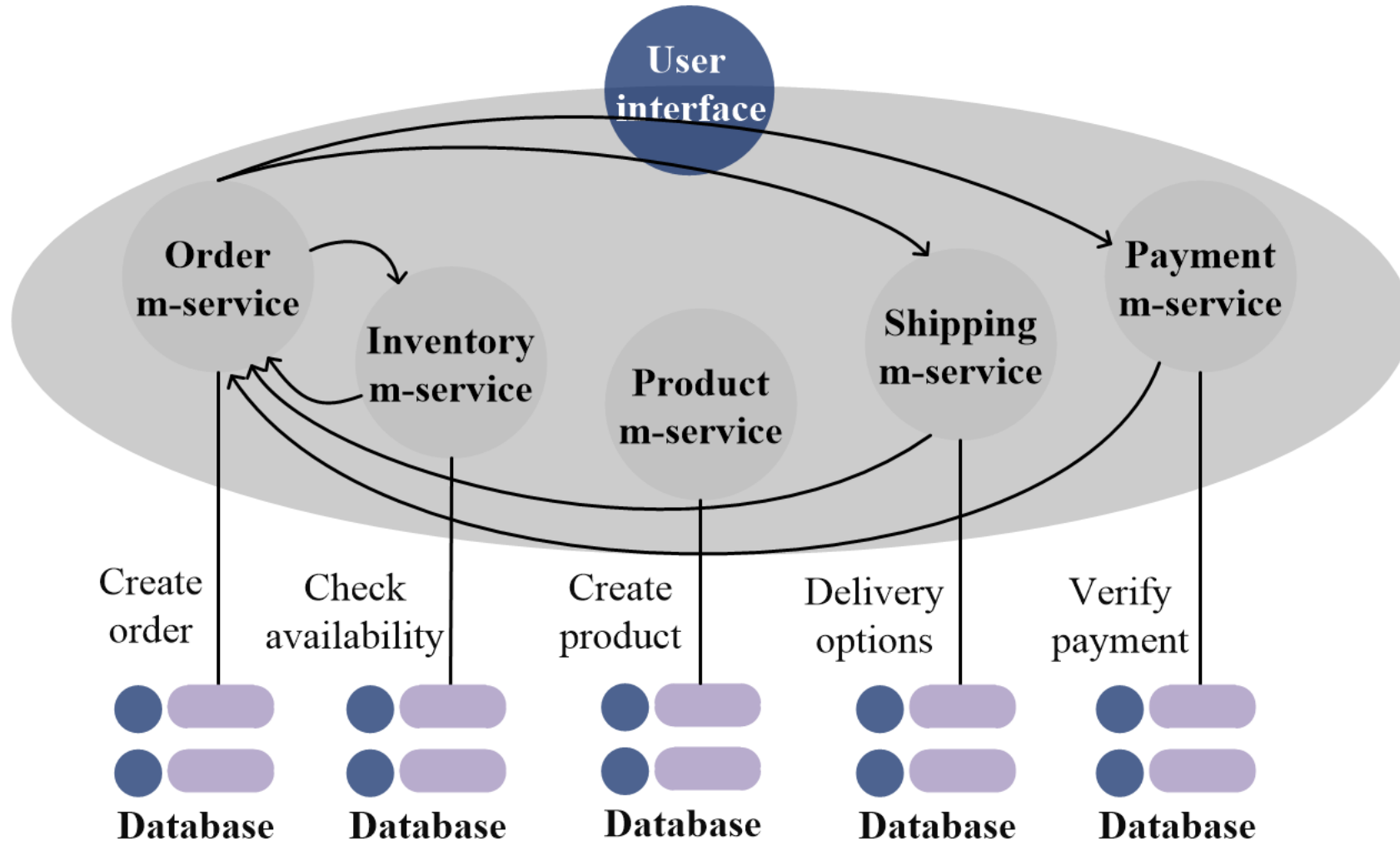
Monolithic vs SOA vs Mservices arch.

Micro-service architecture



Monolithic vs SOA vs Mservices arch.

Micro-service architecture



Characteristics of Microservices

Microservices

Specialized: each microservice is designed for solving a specific problem/performing a specific set of functionalities.

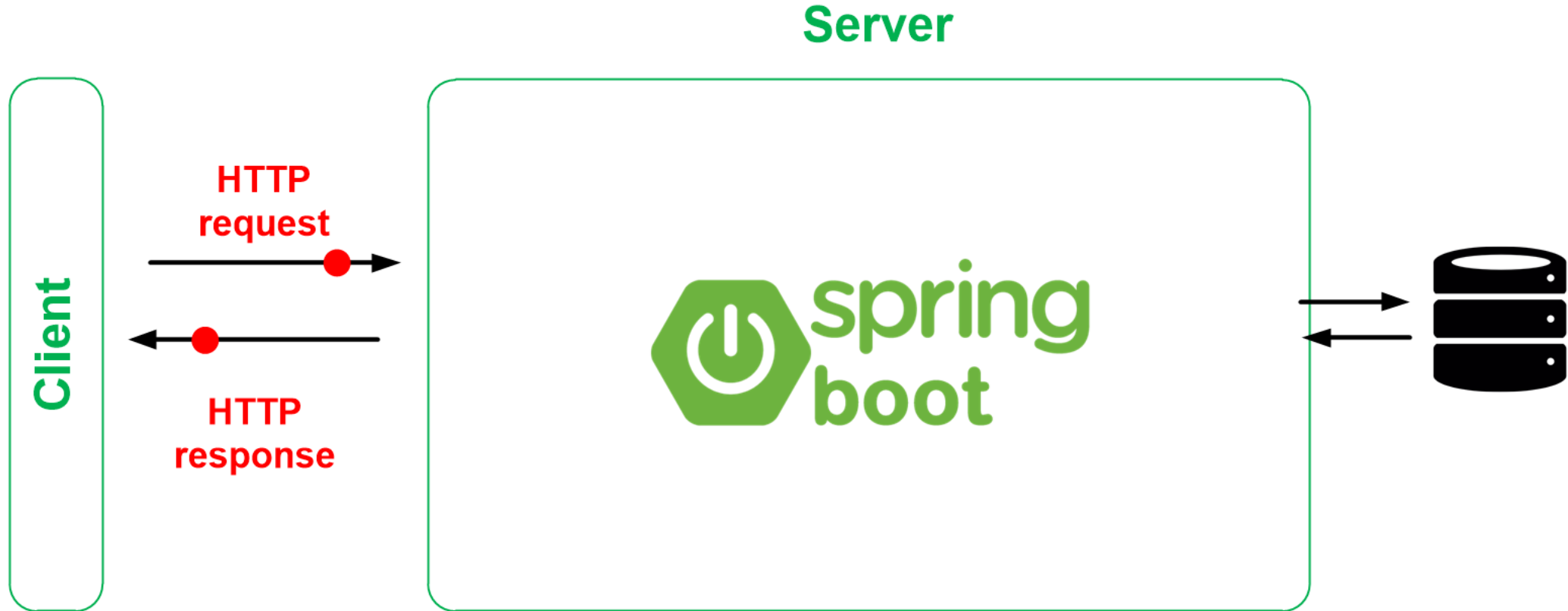
Autonomous : a microservice can be developed, and deployed without affecting the functioning of other services. They do not need to share code nor implementation with one another services, and any communication between individual microservices occur via well-defined APIs.

Microservices architecture

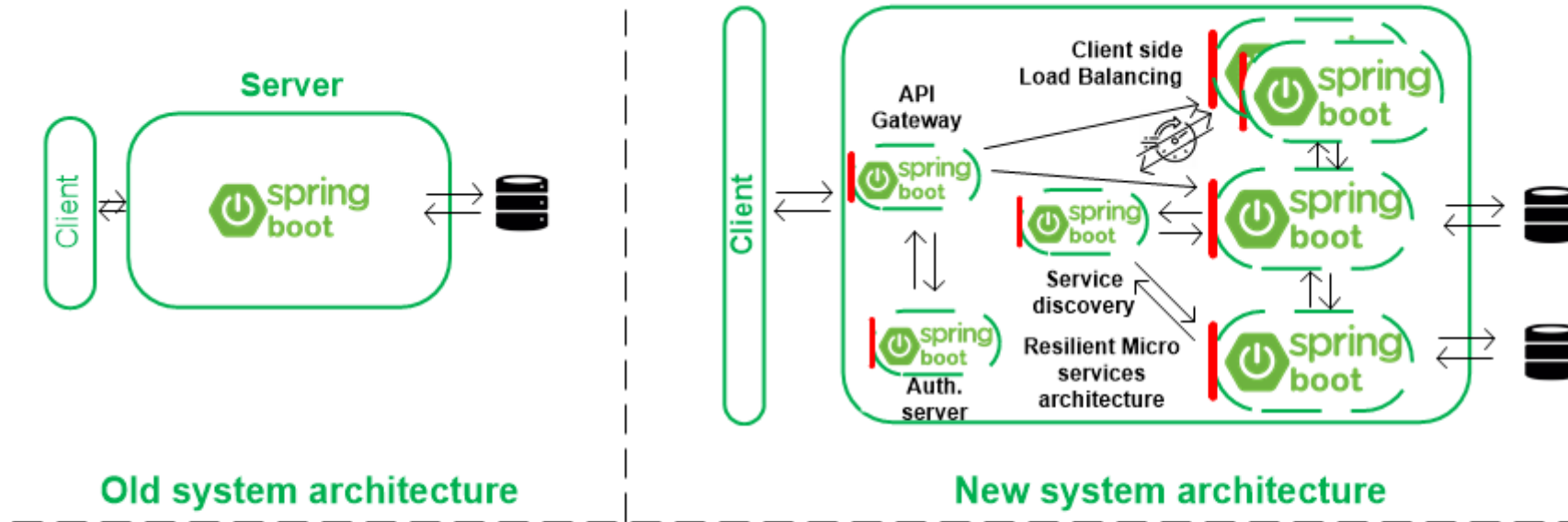
Technological freedom: a microservice architectures don't follow a "one size fits all" approach, any microservice can be build using the best technology that fits its functionalities.

Resilience: service independence increases an application's **resilience**. Such architecture, can handle total service failure by degrading functionality.

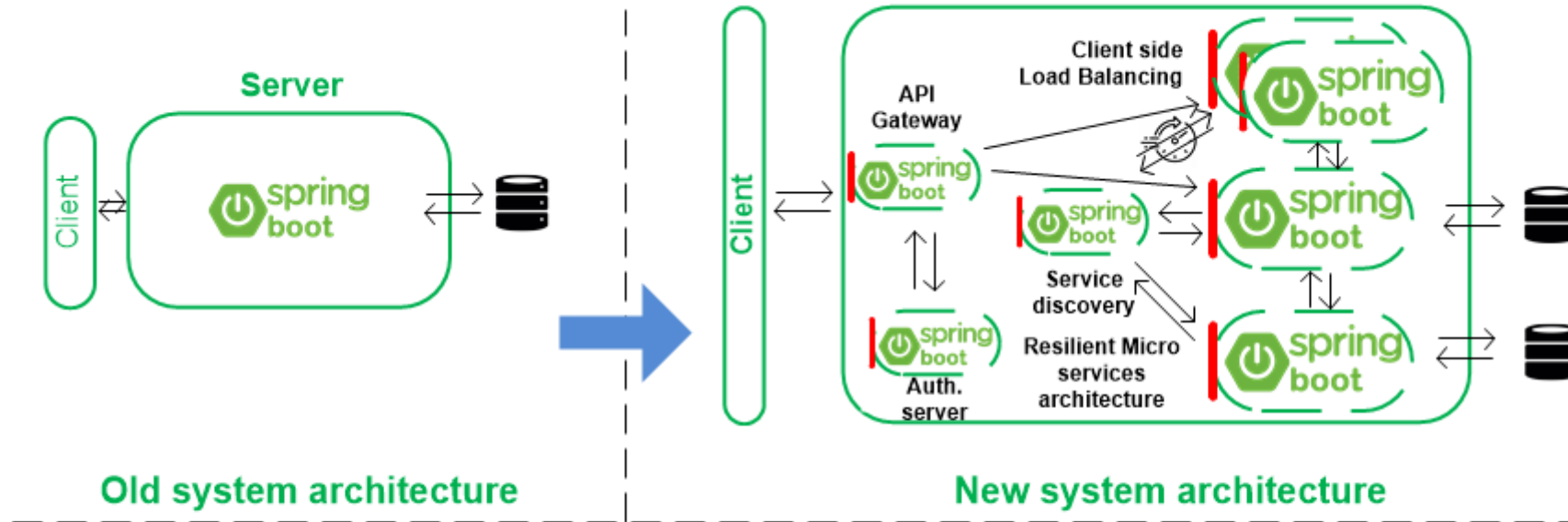
What we have already learned?



What we need to learn?

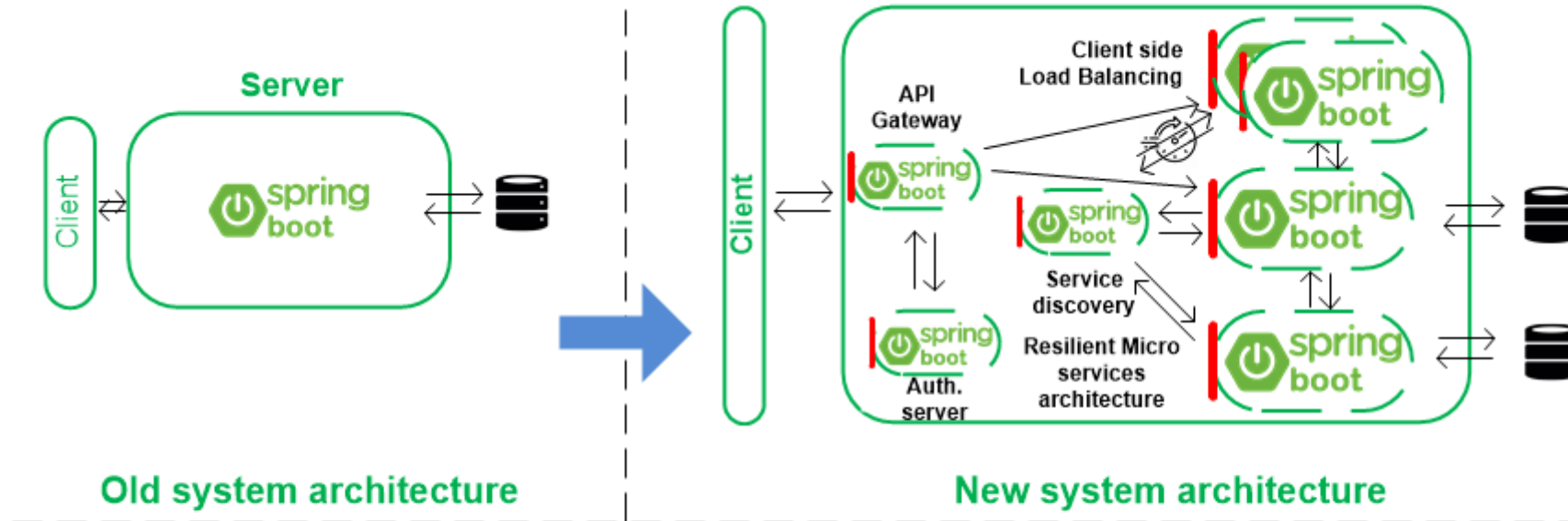


What we need to learn?



How we can transfer the system
from old to the new architecture?

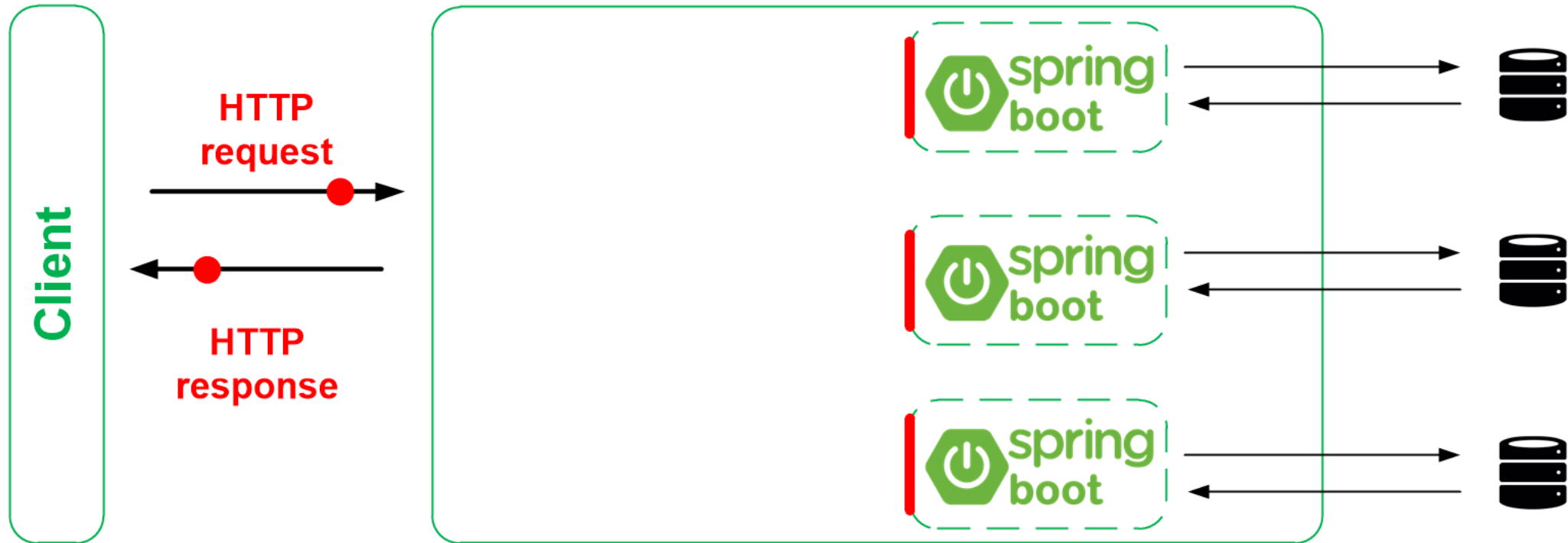
What we need to learn?



How we can transfer the system
from old to the new architecture?

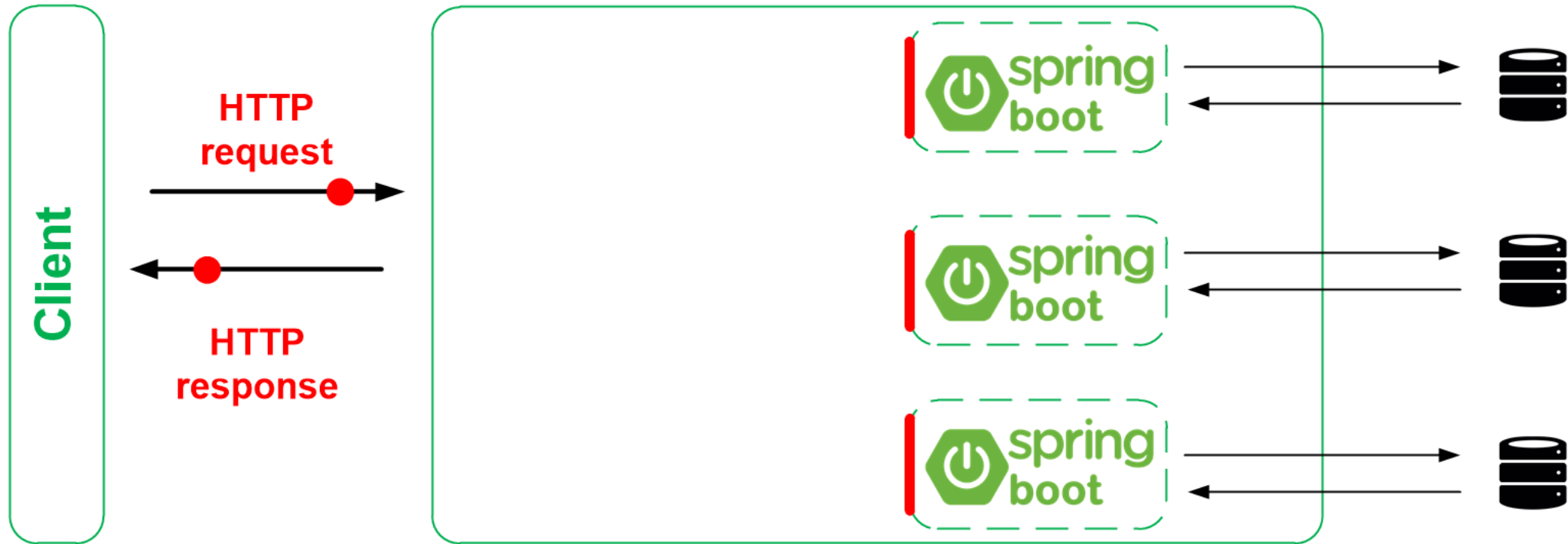
- Design patterns
- Implementation technologies

What we need to learn?



Microservices creation?

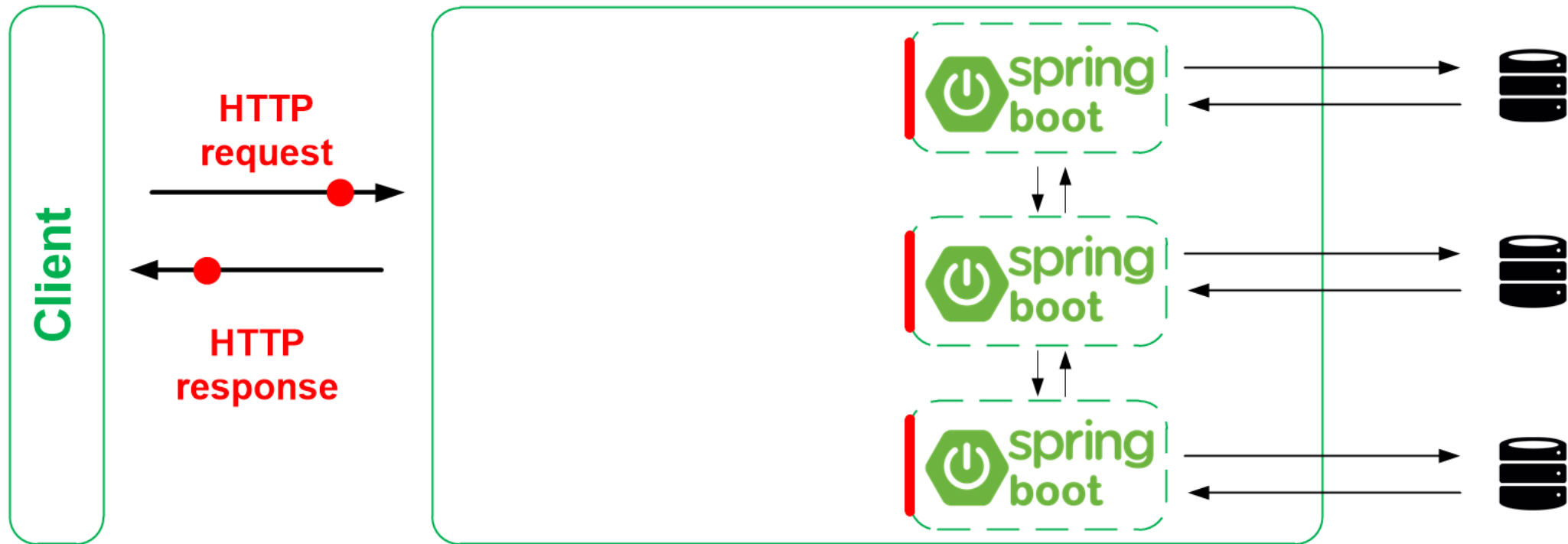
What we need to learn?



Microservices creation?

Not really, a microservice is basically a **Spring Boot Project**.

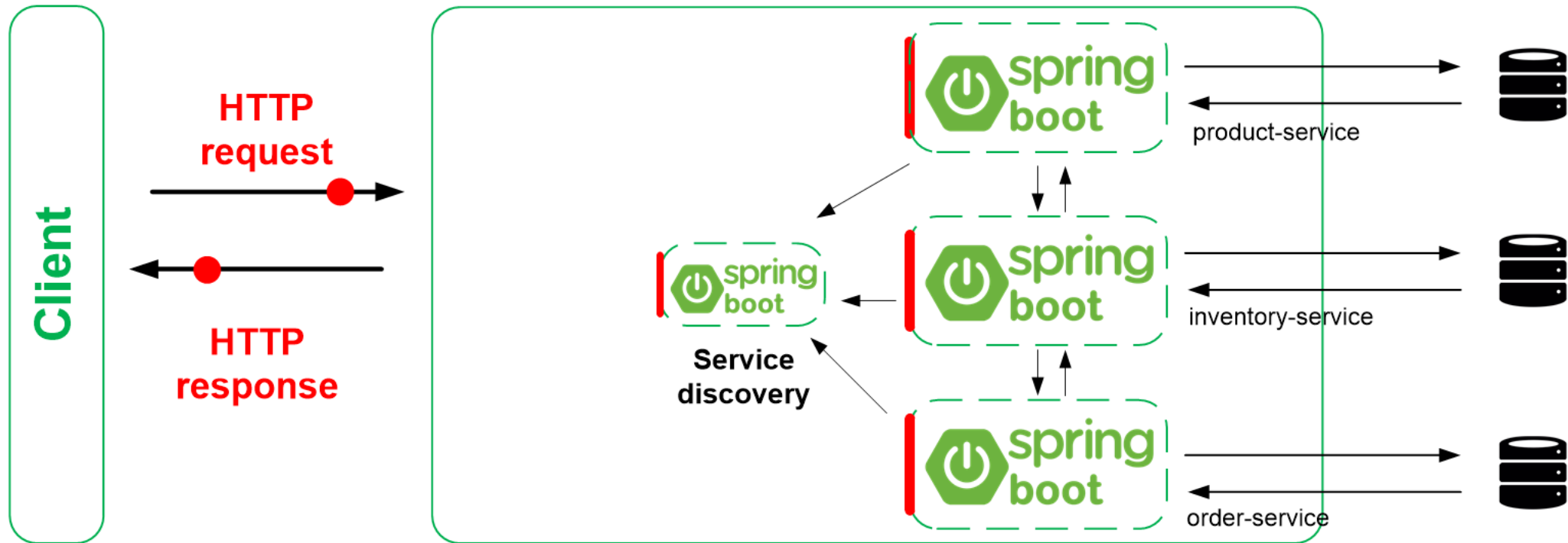
What we need to learn?



Microservices communications:

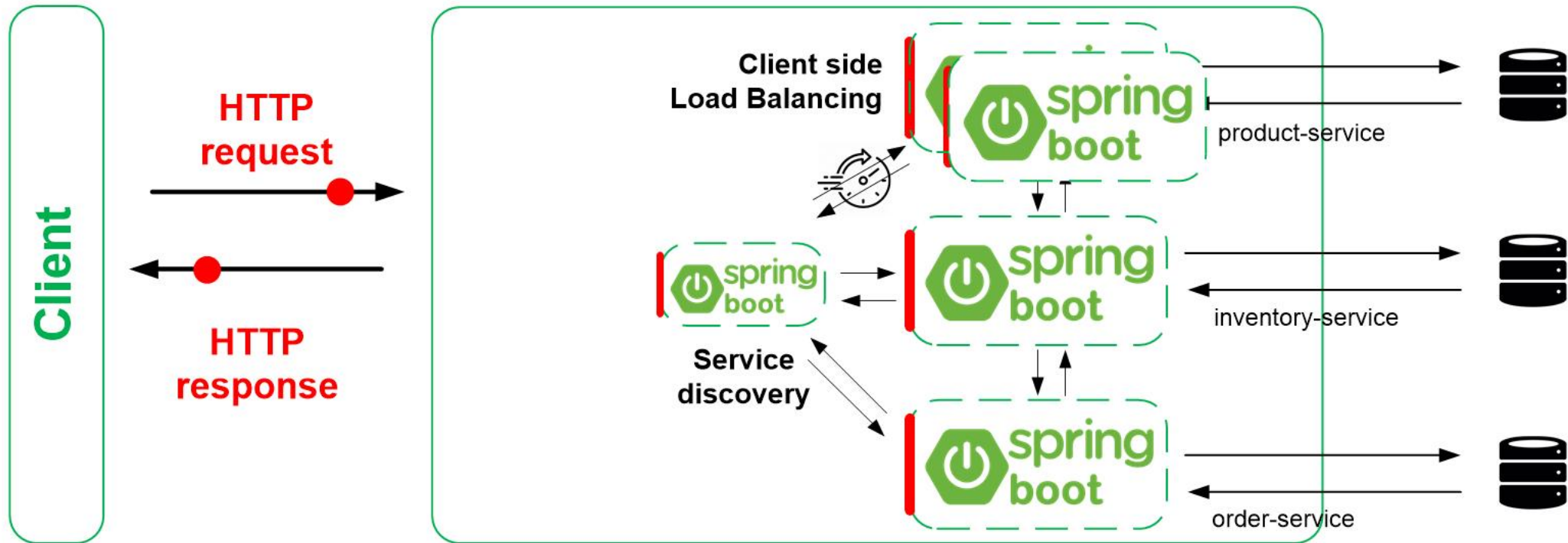
Spring WebFlux - Reactive RESTFUL APIs - Reactive Programming - WebClient

What we need to learn?



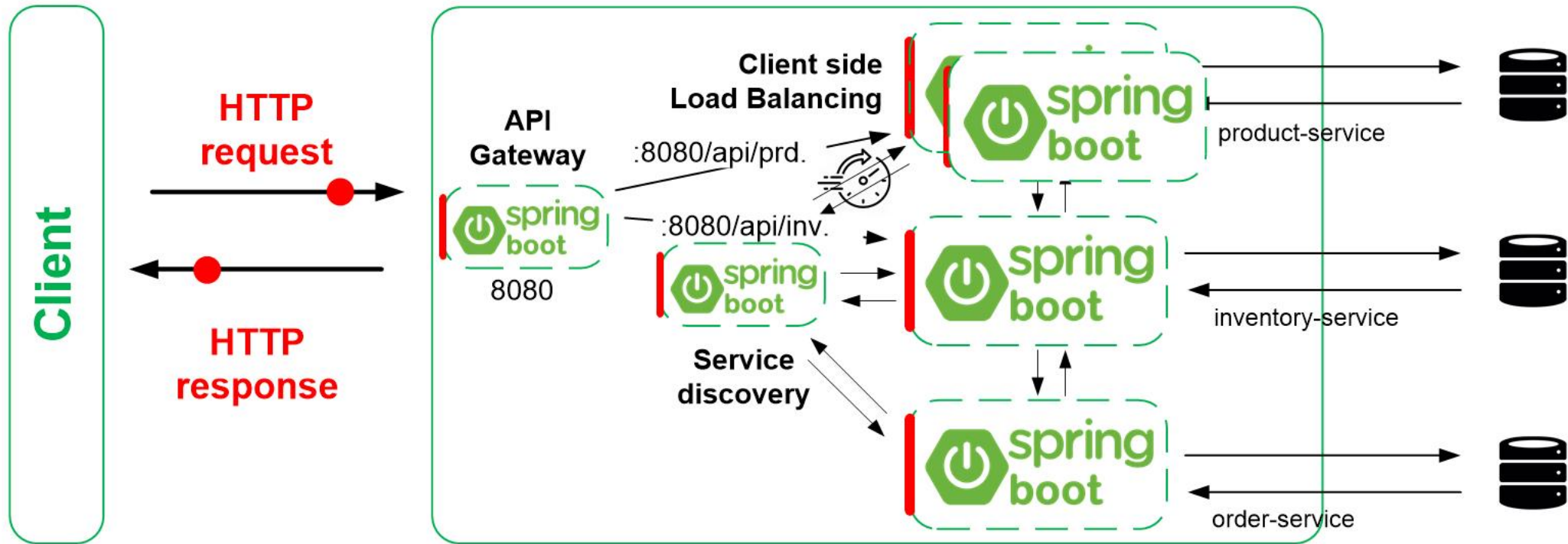
Services Discovery - Eureka: is one of the key features of a **microservice-based architecture**. Trying to hand-configure each client can be difficult, time-consuming, and error-prone.

What we need to learn?



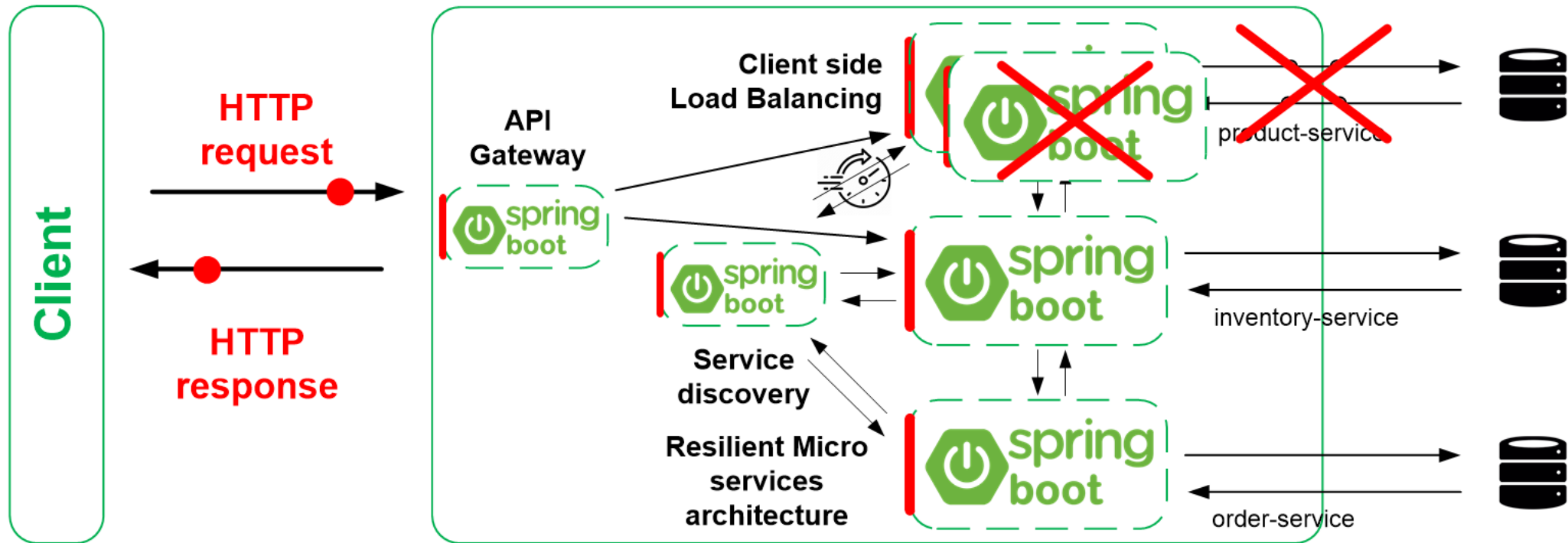
Load balancing is the process of distributing a set of tasks over a set of resources, with the aim of making their overall processing more efficient.

What we need to learn?



API Gateway acts as a "front door" for applications to access data, business logic, or functionality from your backend services.

What we need to learn?



Resilient microservice(s) patterns

What if the performance of microservice(s) or the overall system is degrading?

Microservices communications

Reactive Programming - WebFlux

Reactive programming is a programming paradigm that promotes an **asynchronous, non-blocking, event-driven approach** to data processing.

Reactive programming involves modeling **data** and **events** as observable **data streams** and implementing **data processing routines** to **react** to the **changes** in those **streams**.

Reactive Programming - WebFlux

Reactive programming is a programming paradigm that promotes an **asynchronous, non-blocking, event-driven approach** to data processing.

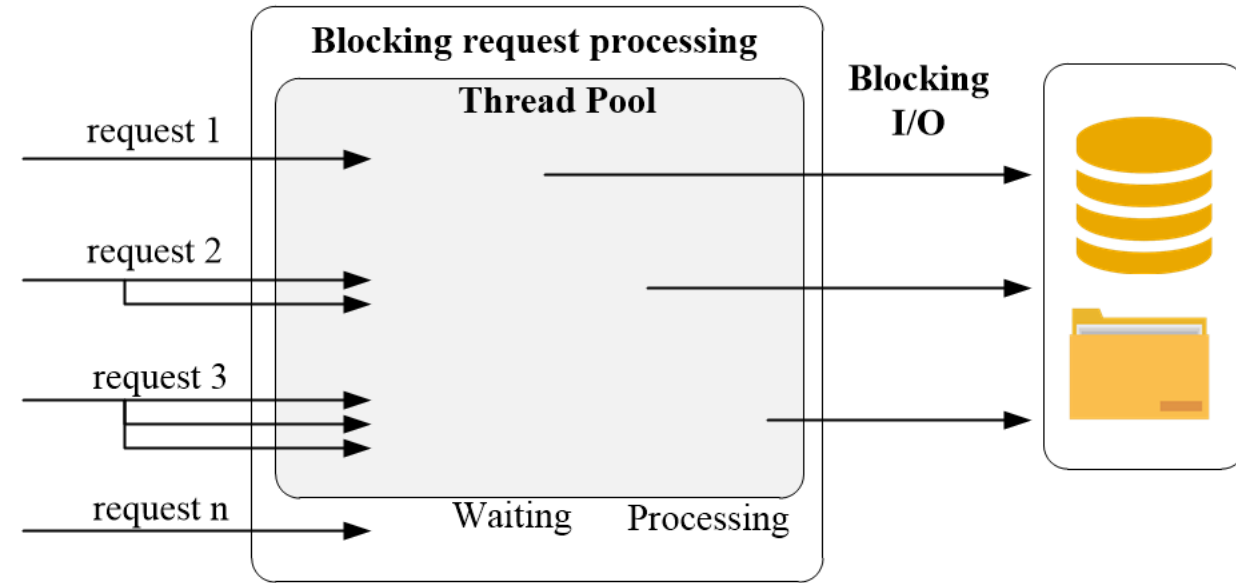
Reactive programming involves modeling **data** and **events** as observable **data streams** and implementing **data processing routines** to **react** to the **changes** in those **streams**.

What is asynchronous, non-blocking request processing?

Blocking vs non-blocking Req. Processing

Blocking Request Processing

In traditional MVC applications, a new thread is created when a request comes to the server.

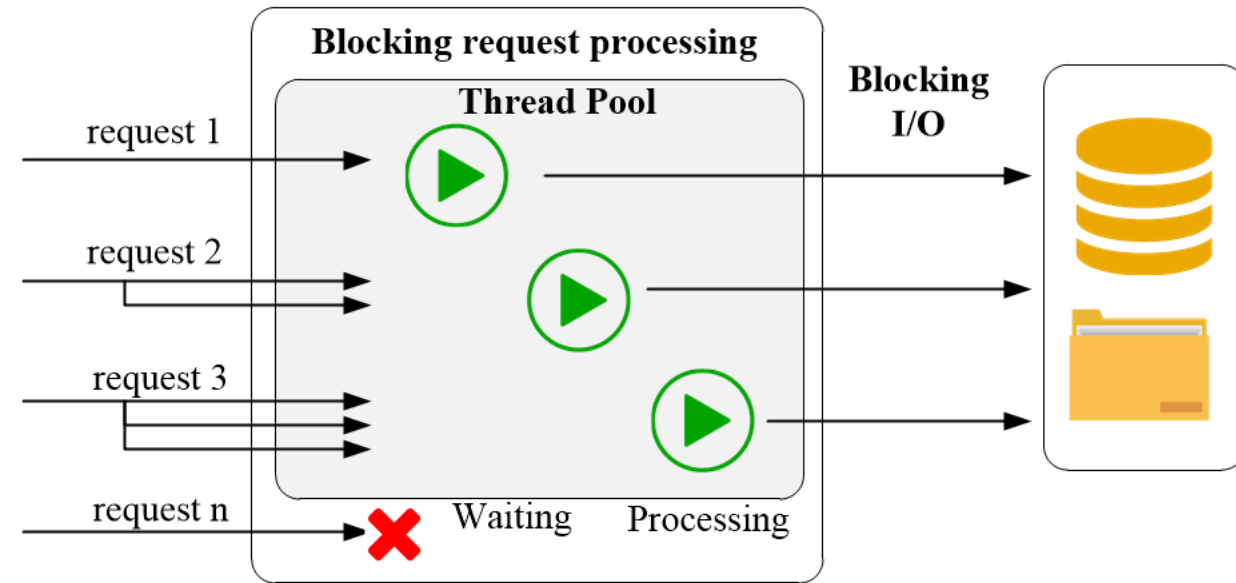


Blocking vs non-blocking Req. Processing

Blocking Request Processing

In traditional MVC applications, a new thread is created when a request comes to the server.

It delegates the request to worker threads for I/O operations such as database access etc.



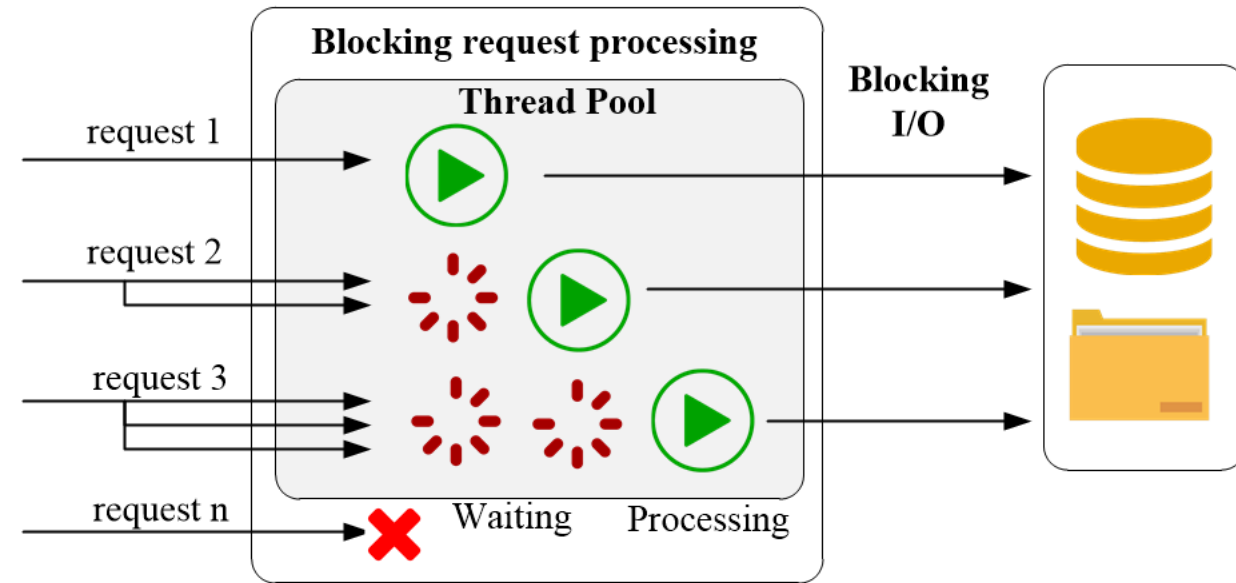
Blocking vs non-blocking Req. Processing

Blocking Request Processing

In traditional MVC applications, a new thread is created when a request comes to the server.

It delegates the request to worker threads for I/O operations such as database access etc.

During the time worker **threads are busy**, the thread (request thread) remains in waiting status, and thus it is blocked. It is also called synchronous request processing



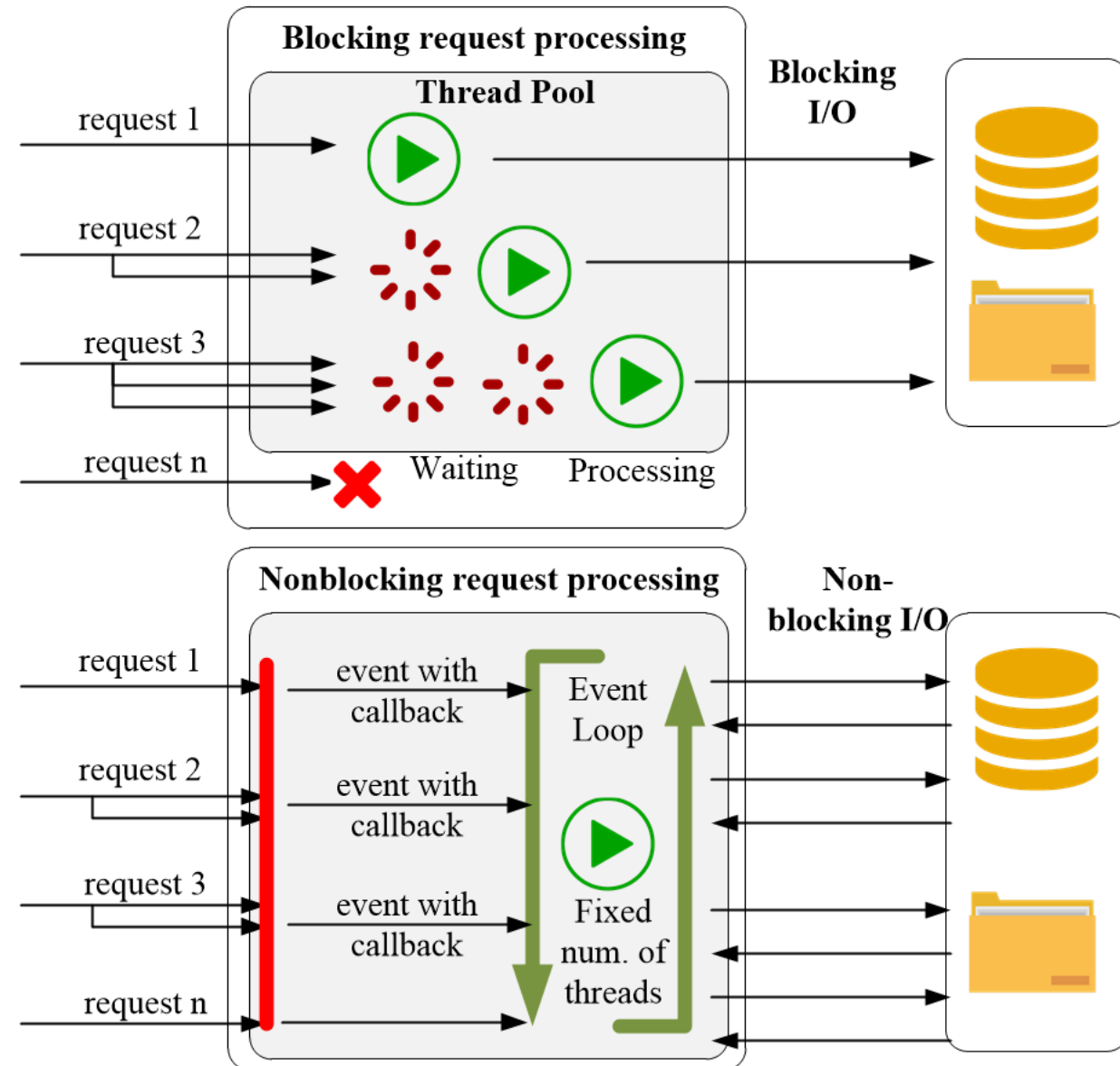
Blocking vs non-blocking Req. Processing

Non-blocking Request Processing

There is only one request thread receiving the request. All incoming requests come with an **event handler** and **callback information**.

Request thread delegates the incoming requests to a thread pool, which delegates the request to its handler function and immediately starts processing other incoming requests from the request thread.

When the handler function is complete, one thread from the pool collects the response and passes it to the callback function.



Reactive Programming

The term, “reactive,” refers to programming models that are built around reacting to changes. It is built around publisher-subscriber pattern (observer pattern). In the reactive style of programming, we make a request for resources and start performing other things. When the data is available, we get the notification along with data information on the call-back function. The callback function handles the response as per application/user needs.

Reactive web programming is great for applications that have streaming data, and clients that consume it and stream it to their users. It is not great for developing traditional CRUD applications.

If you’re developing the next Facebook or Twitter with lots of data, a reactive API might be just what you’re looking for.

Reactive Streams API

The new Reactive Streams API was created by engineers from Netflix, Pivotal, Lightbend, RedHat, Twitter, and Oracle, among others and are now part of Java 9. It defines four interfaces:

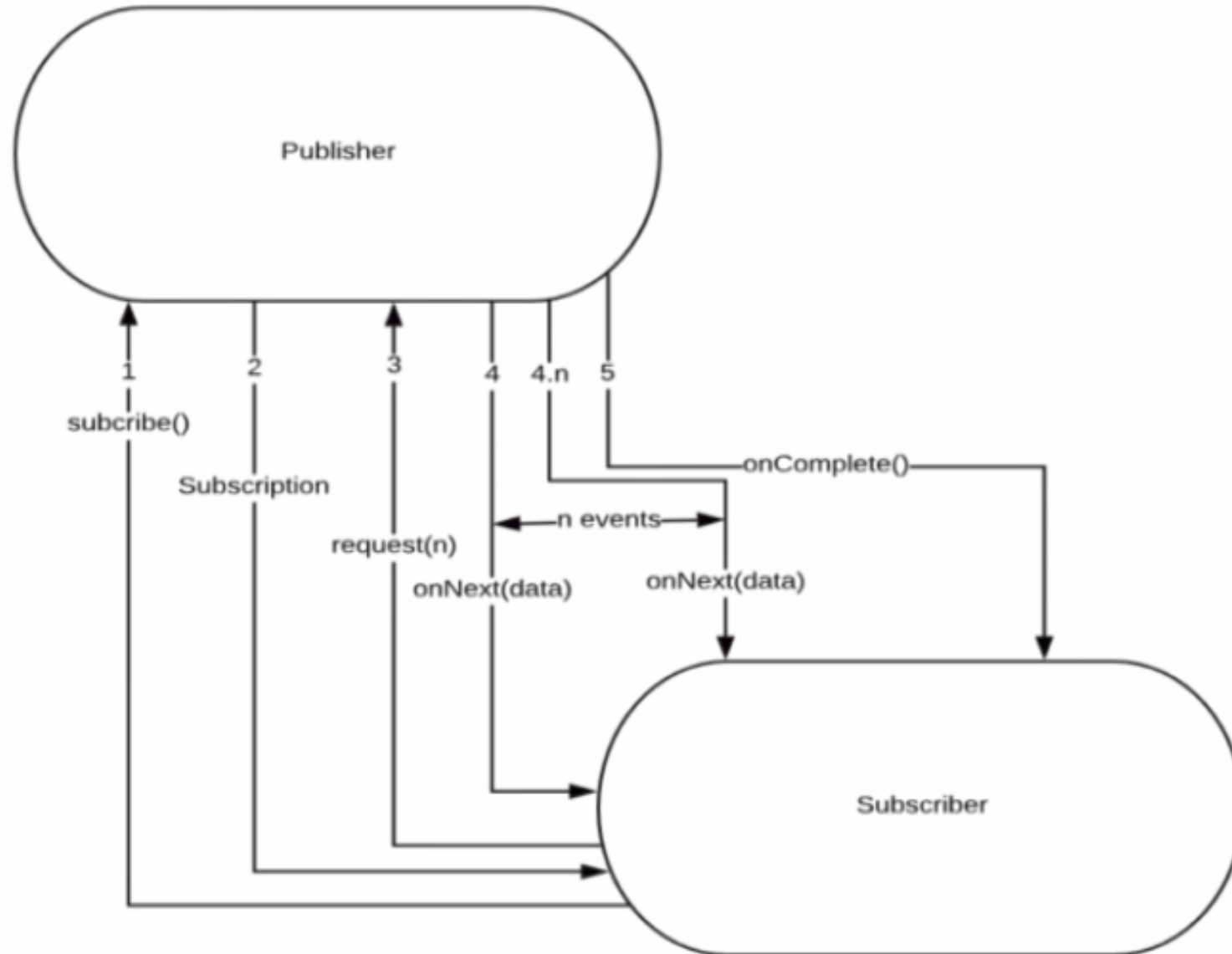
Publisher: Emits a sequence of events to subscribers according to the demand received from its subscribers. A publisher can serve multiple subscribers.

Subscriber: Receives and processes events emitted by a Publisher. Please note that no notifications will be received until `Subscription#request(long)` is called to signal the demand. It has four methods to handle various kinds of responses received.

Subscription: Defines a one-to-one relationship between a Publisher and a Subscriber. It can only be used once by a single Subscriber. It is used to both signal desire for data and cancels demand (and allow resource cleanup).

Processor: Represents a processing stage consisting of both a Subscriber and a Publisher and obeys both contracts.

Reactive Streams API



What is Spring WebFlux?

Spring WebFlux is a parallel version of Spring MVC and supports fully non-blocking reactive streams. It supports the back pressure concept and uses Netty as the inbuilt server to run reactive applications. If you are familiar with the Spring MVC programming style, you can easily work on webflux also.

Spring webflux uses project reactor as the reactive library. Reactor is a Reactive Streams library; therefore, all of its operators support non-blocking back pressure. It is developed in close collaboration with Spring.

Spring WebFlux heavily uses two publishers :

- **Mono**: Returns 0 or 1 element.
- **Flux**: Returns 0...N elements.

```
Mono<String> mono = Mono.just("Alex");  
Mono<String> mono = Mono.empty();
```

```
Flux<String> flux = Flux.just("A", "B", "C"); Flux<String> flux = Flux.fromArray(new  
String[]{"A", "B", "C"}); Flux<String> flux = Flux.fromIterable(Arrays.asList("A", "B",  
"C")); //To subscribe call method flux.subscribe();
```

Practical session 4

Microservices communication – PS4

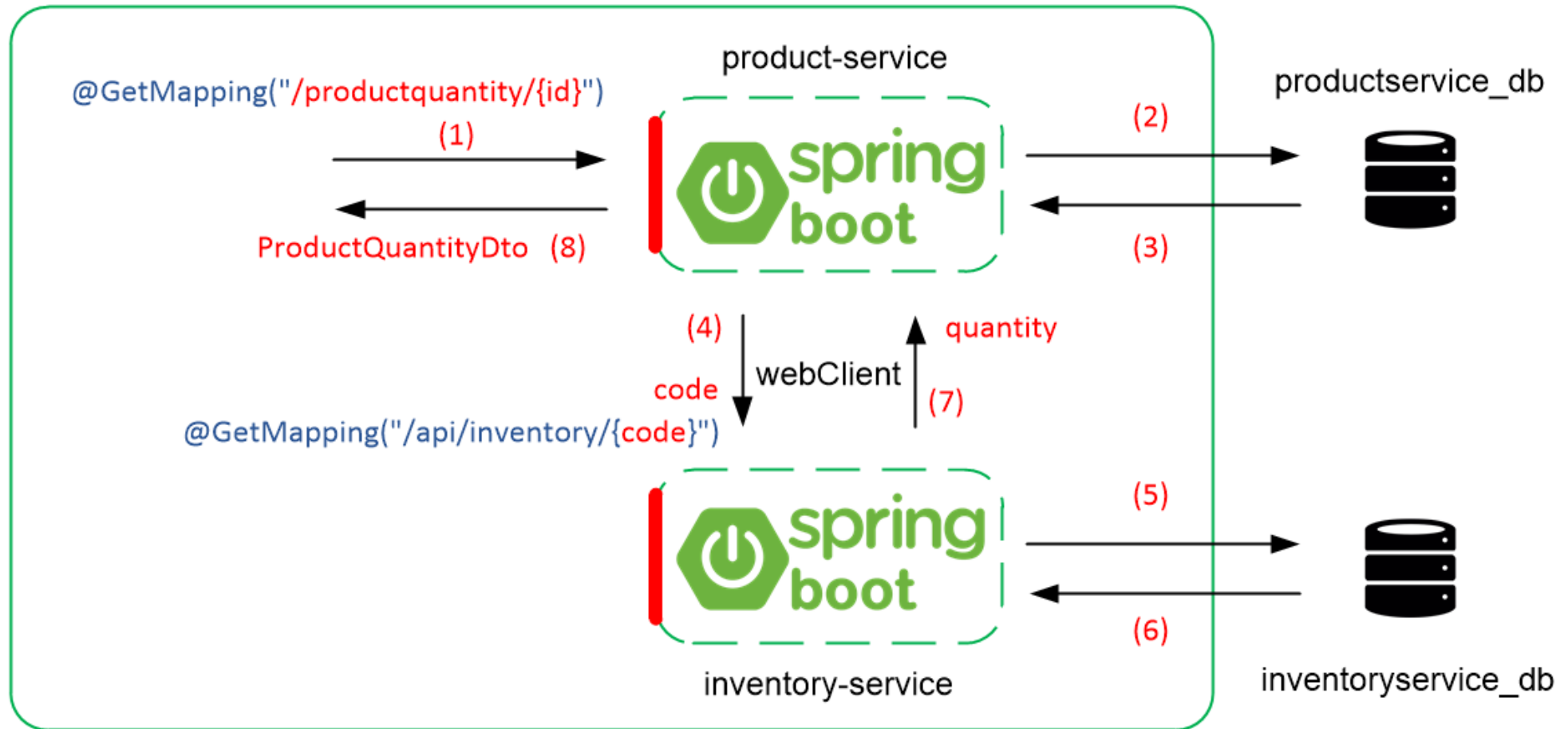
In this example, we have two microservices:

- **product-service:** fetch, add, update and delete products.
- **inventory-service:** only contain the quantity of a specific product that can be identify based on **product code**.

Each microservice has its own **database**.

The **product-service** will fetch the **quantity** of a product from the **inventory-service** using **WebClient** that is an interface representing the main entry point for performing web requests.

Microservices communication – PS4



inventory-service

```
@RestController
public class InventoryController {
    ..
    @GetMapping("/api/inventory/{code}")
    public Integer quantityInStock(@PathVariable String code) {
        return inventoryService.quantityInStock(code);
    }
}
```

```
@Entity
..
public class InventoryItem {
    @Id
    private String id;
    private String code;
    private Integer quantity;
}
```

```
@Service
public class InventoryService {
    ...
    public Integer quantityInStock(String code){
        InventoryItem inventoryItem = inventoryRepository.findByCode(code);
        return inventoryItem.getQuantity();
    }
};
```

product-service - model

```
public class Product {  
    @Id  
    private String id;  
    private String code;  
    private String name;  
    private String description;  
    private BigDecimal price;  
    ...  
}
```

```
public class InventoryItemDto {  
    @Id  
    private String id;  
    private String code;  
    private Integer quantity;  
}
```

product-service - model

```
public class Product {
    @Id
    private String id;
    private String code;
    private String name;
    private String description;
    private BigDecimal price;
    ...
}
```

```
public class InventoryItemDto {
    @Id
    private String id;
    private String code;
    private Integer quantity;
}
```

```
public class ProductQuantityDto {
    @Id
    private String id;
    private String code;
    private String name;
    private String description;
    private BigDecimal price;
    private Integer quantity;
    ...
}
```

product-service - controller

```
@RestController
@RequestMapping("/api")
public class ProductController {
    ...
    @GetMapping("/productquantity/{id}")
    public Optional<ProductQuantityDto> getProductWithQuantity(@PathVariable String id) {
        return productService.getProductWithQuantity(id);
    }
    ...
}
```



product-service

```
@Service
@Slf4j
public class ProductService {
    @Autowired
    private WebClient.Builder webClientBuilder;

    ...

}
```

```
@SpringBootApplication
public class ProductServiceApplication {

    @Bean
    public WebClient.Builder getWebClientBuilder() {
        return WebClient.builder();
    }

    public static void main(String[] args) {
        SpringApplication.run(ProductServiceApplication.class, args);
    }

}
```

product-service - service

@Service

...

```
public Optional<ProductQuantityDto> getProductWithQuantity(String id) {
    Optional<Product> product = productRepository.findById(id);
    return product.map(this::mapToProductQuantityDto);
}
private ProductQuantityDto mapToProductQuantityDto(Product product) {
    Integer quantity = webClientBuilder.build()
        .get()
        .uri("http://localhost:8083/api/inventory/{code}", product.getCode())
        .retrieve()
        .bodyToMono(Integer.class)
        .block();
    return ProductQuantityDto.builder()
        .id(product.getId())
        .name(product.getName())
        .description(product.getDescription())
        .price(product.getPrice())
        .code(product.getCode())
        .quantity(quantity)
        .build();
}
```

inventory-service

```
@Entity
```

```
..
```

```
public class InventoryItem {
```

```
@Configuration
```

```
public class DataInitializer {
```

```
    @Bean
```

```
    public CommandLineRunner loadProductsData(ProductRepository productRepository)
```

```
{
```

```
    return args -> {
```

```
        Product product1 = new Product();
```

```
        product1.setId("01");
```

```
        product1.setCode("p-109-09");
```

```
        product1.setName("light vehicle");
```

```
        product1.setDescription("Can also be used for light work");
```

```
        product1.setPrice(BigDecimal.valueOf(1233));
```

```
        productRepository.save(product1);
```

```
        ...
```

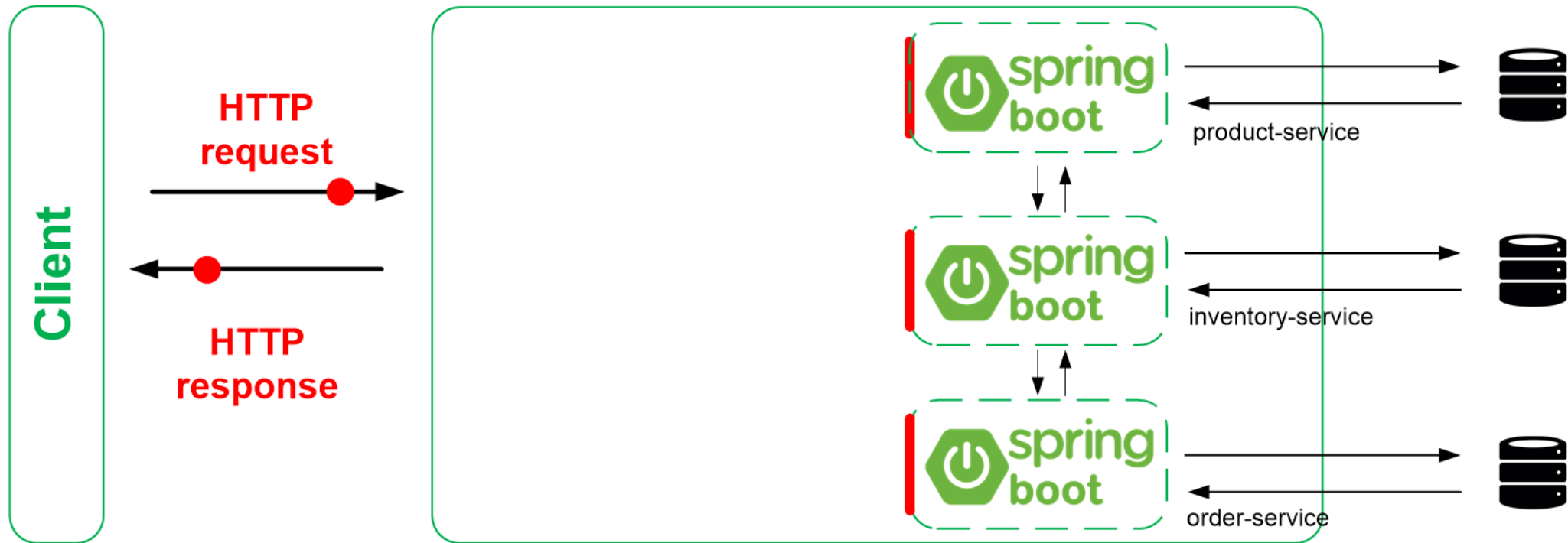
```
    };
```

```
}
```

```
}
```

Services Discovery

Services Discovery





Services Discovery

Service Discovery is one of the key features of a **microservice-based architecture**. Trying to hand-configure each client can be difficult, time-consuming, and error-prone.

Why do we need a Service Discovery?

Microservices are dynamic in nature, which means a **single *Microservice* may *have*** multiple instances running at the same time. These instances will either have a **different IP address or a different port or both**.

Services Discovery

Service Discovery is one of the key features of a **microservice-based architecture**. Trying to hand-configure each client can be difficult, time-consuming, and error-prone.

Why do we need a Service Discovery?

Microservices are dynamic in nature, which means a **single *Microservice* may *have*** multiple instances running at the same time. These instances will either have a **different IP address or a different port or both**.

This creates a problem since we cannot always know for sure the **address of a *Microservice* instance**, how **many instances** are there, or how **to select** one of them. Keep in mind that the number of these instances can change at any time.

Services Discovery

Service Discovery is one of the key features of a **microservice-based architecture**. Trying to hand-configure each client can be difficult, time-consuming, and error-prone. To solve this problem, Netflix has developed Eureka, which is a Service Discovery Server and Client.

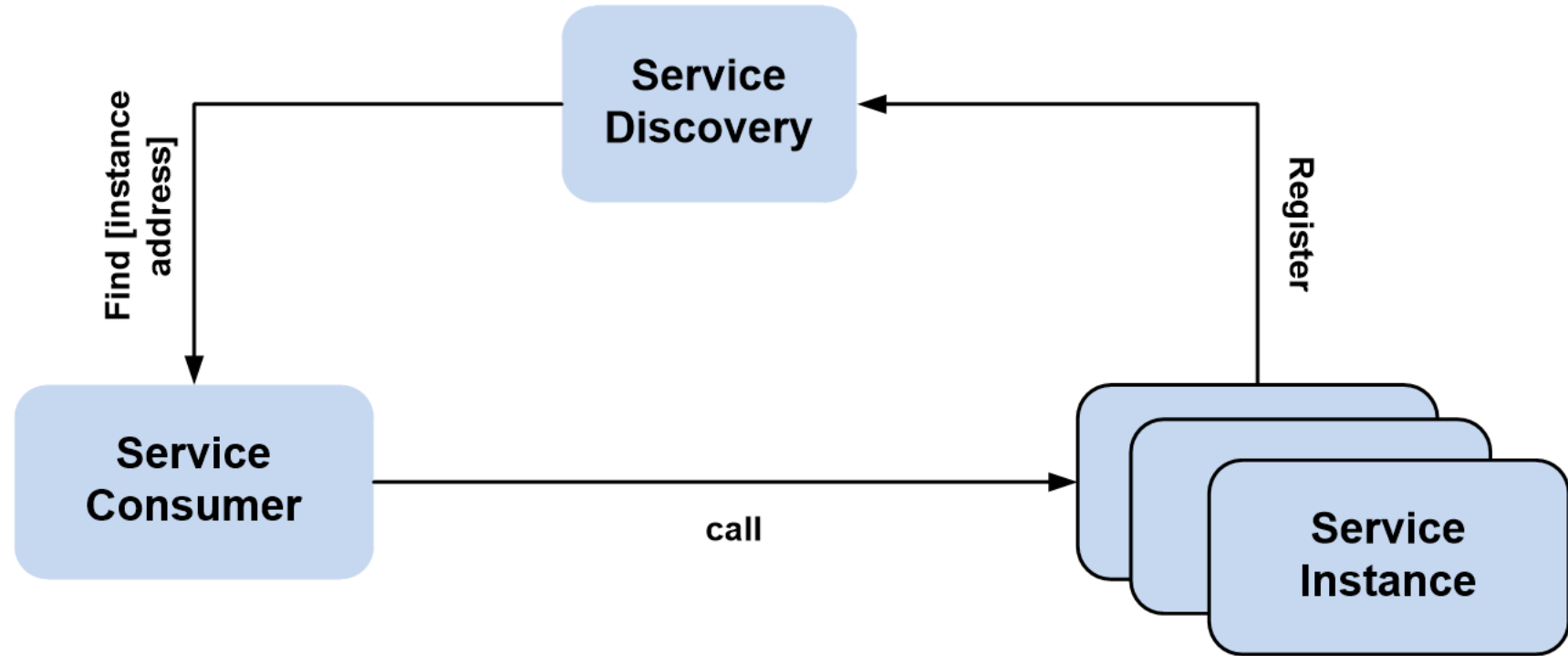
Why do we need a Service Discovery?

Microservices are dynamic in nature, which means a **single *Microservice* may *have*** multiple instances running at the same time. These instances will either have a **different IP address or a different port or both**.

This creates a problem since we cannot always know for sure the **address of a *Microservice* instance**, how **many instances** are there, or how **to select** one of them. Keep in mind that the number of these instances can change at any time.

Service Discovery solve this problem by providing a mechanism that keeps track of all the services and their instances. All the Microservices register themselves with the **Service Discovery** and keep updating their network information regularly.

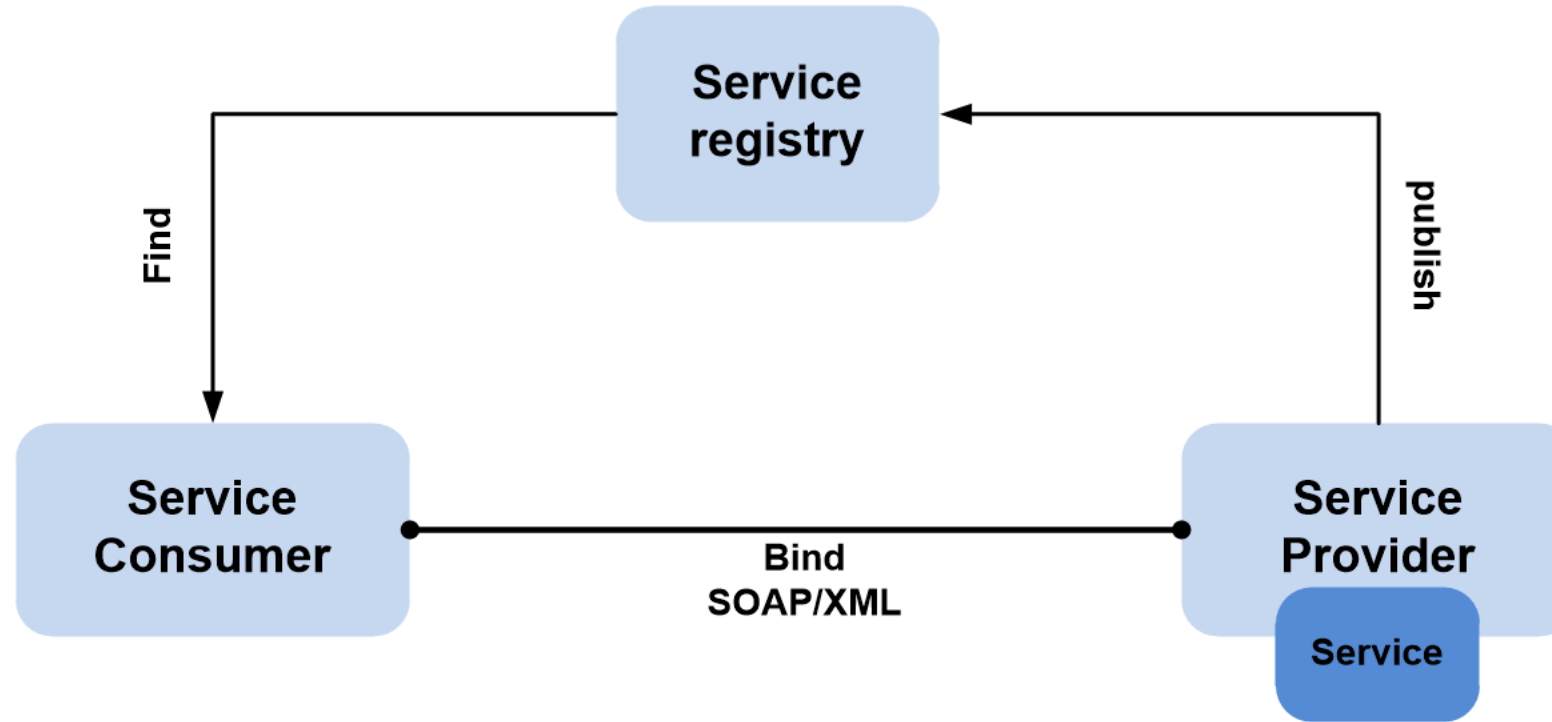
Services Discovery – looks familiar?



Client-Side Service Discovery - the problem can be resolved by creating a **Discovery Server** where our **microservices can register** themselves.

The discovery server will keep a track of all the instances, their hosts, ports, and health status. If a service needs to communicate with another service, it needs to get the instance info from this **discovery server**.

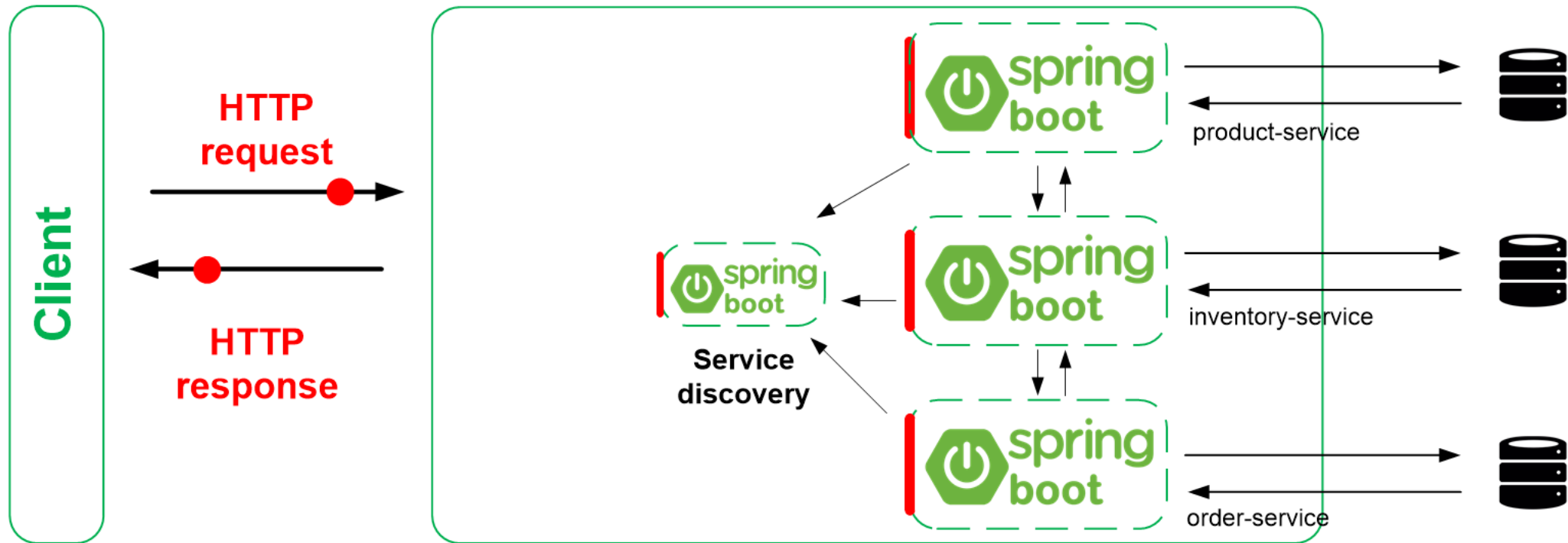
Do you remember SOAP?



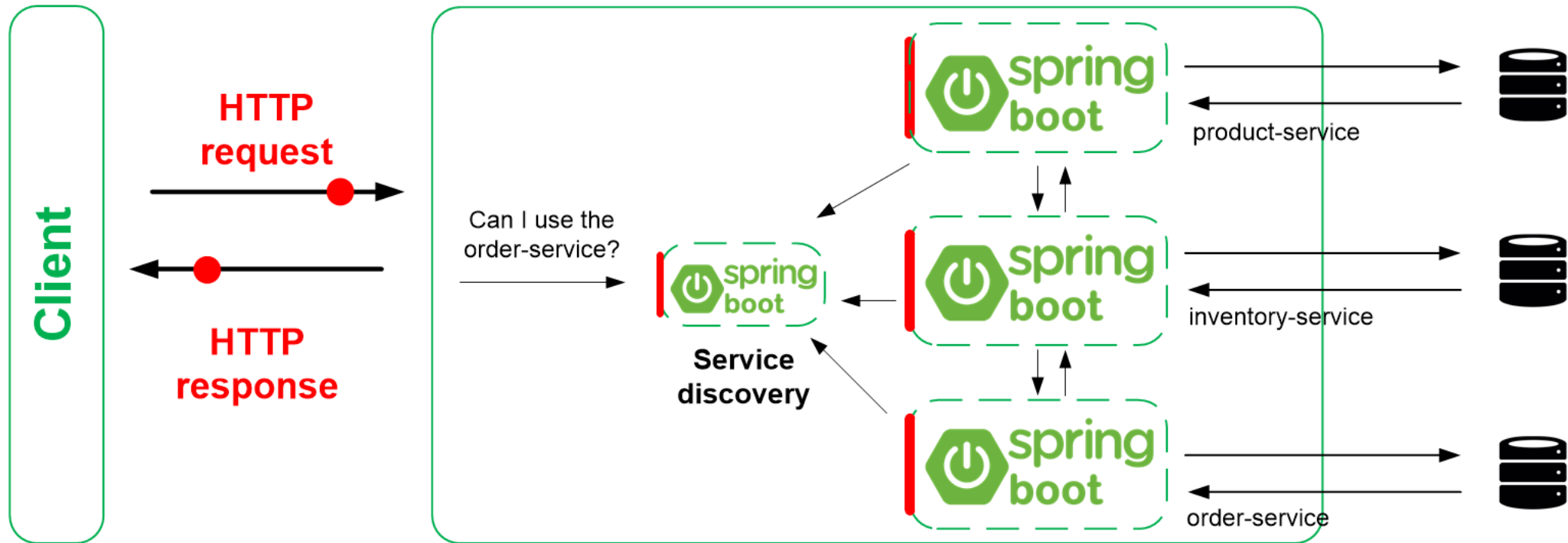
Client-Side Service Discovery - the problem can be resolved by creating a **Discovery Server** where our **microservices can register** themselves.

The discovery server will keep a track of all the instances, their hosts, ports, and health status. If a service needs to communicate with another service, it needs to get the instance info from this **discovery server**.

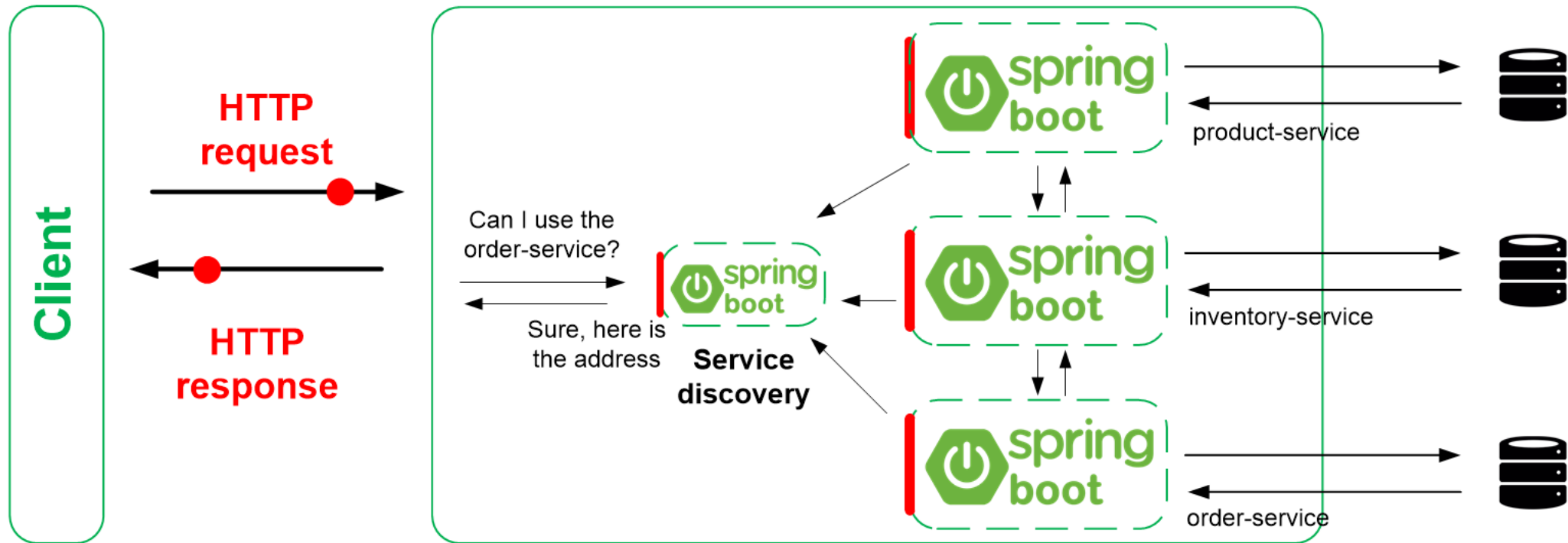
Services discovery - Eureka



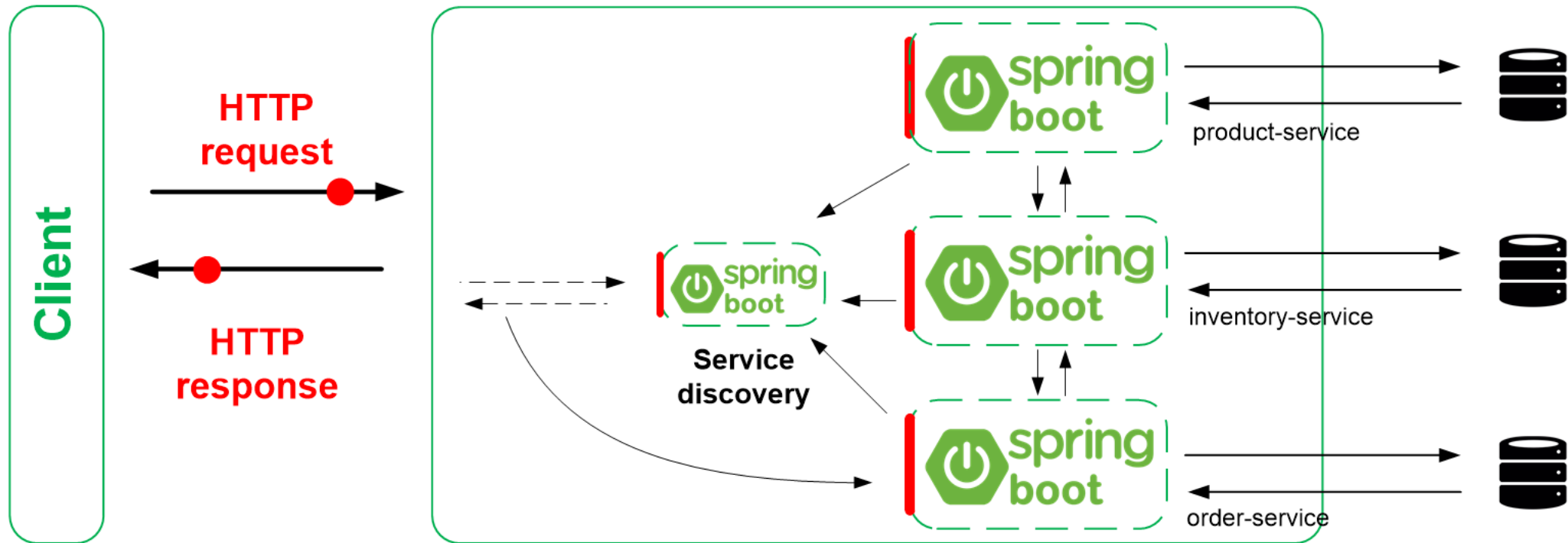
Services discovery - Eureka



Services discovery - Eureka

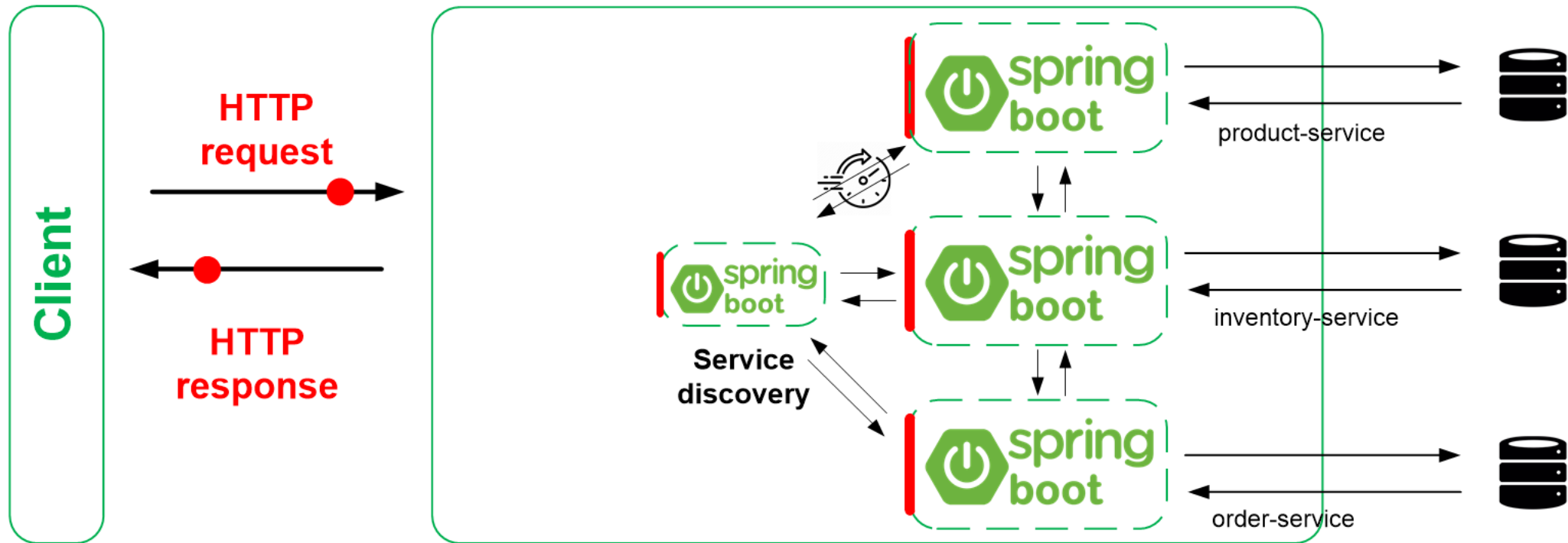


Services discovery - Eureka

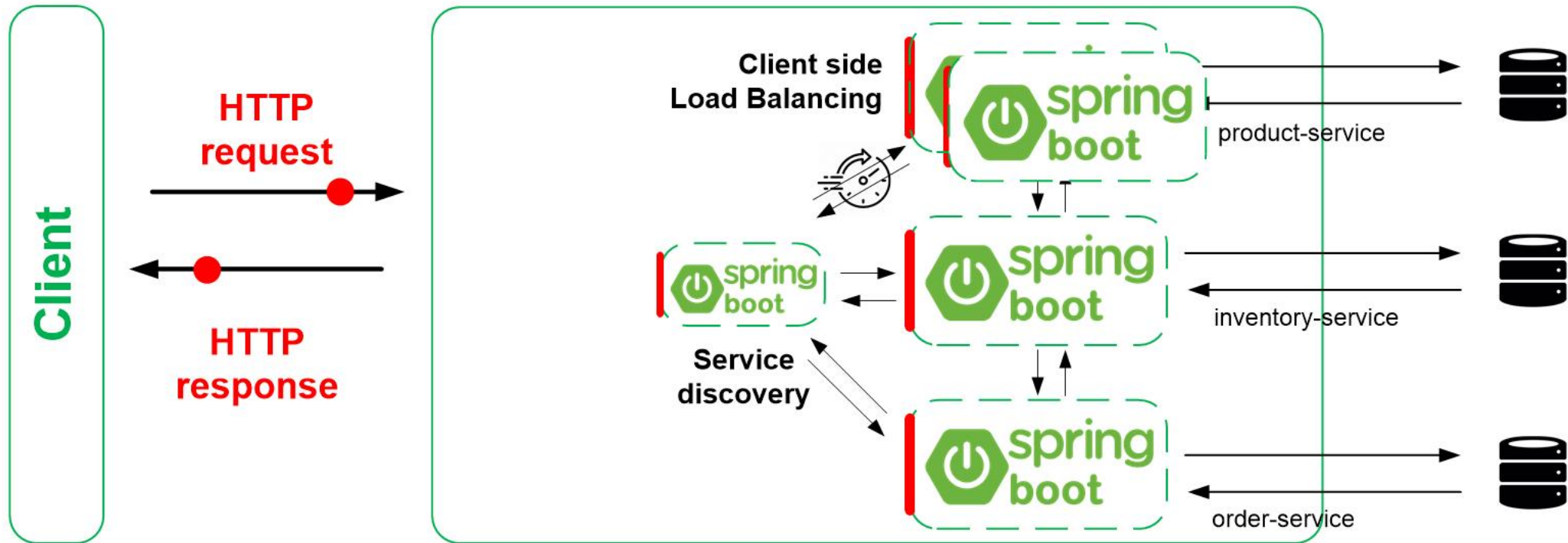


Microservices load balancing

Microservices load balancing



Microservices load balancing



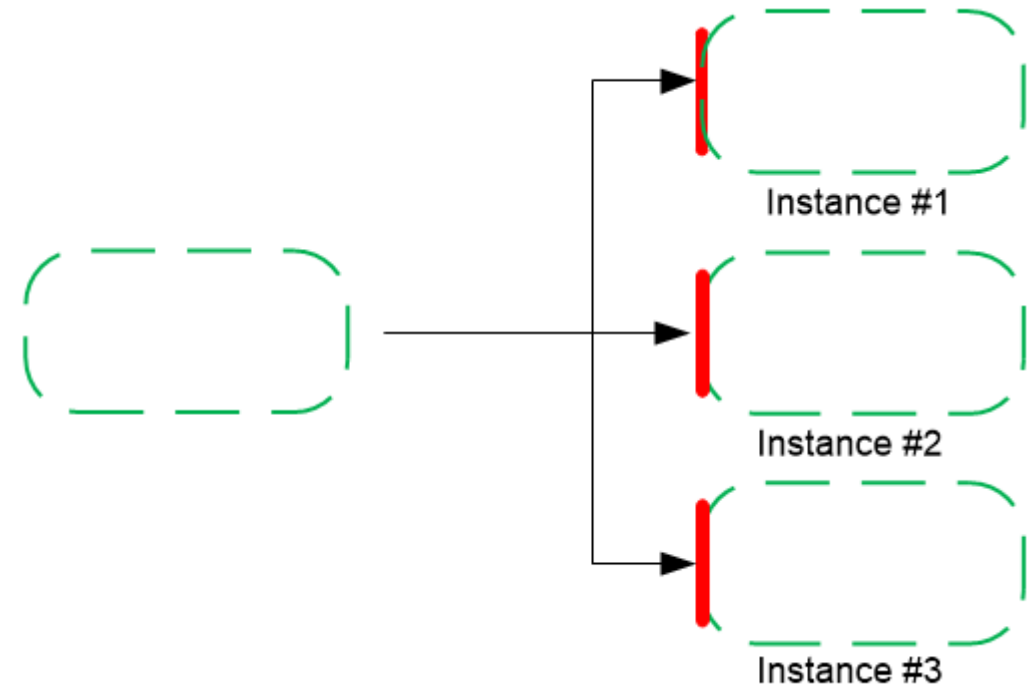
Microservices load balancing

Load balancing is the process of distributing a set of tasks over a set of resources, with the aim of making their overall processing more efficient.

Load balancing can optimize the response time and avoid unevenly overloading some compute nodes while other compute nodes are left idle.

In microservices arch., there are **two type**:

- **Server side** load balancing, and
- **Client side** load balancing.



Microservices load balancing

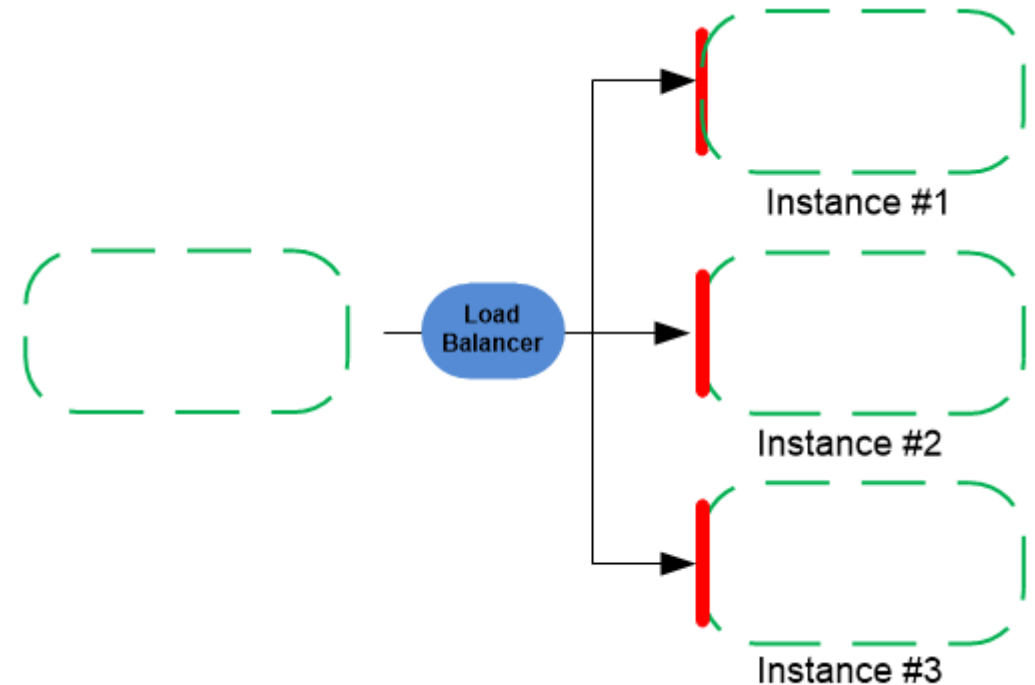
Load balancing is the process of distributing a set of tasks over a set of resources, with the aim of making their overall processing more efficient.

Load balancing can optimize the response time and avoid unevenly overloading some compute nodes while other compute nodes are left idle.

In microservices arch., there are **two type**:

- **Server side** load balancing, and
- **Client side** load balancing.

In server side load balancing, the traffic is distributed by a **load distributor** that distributes (equally or according to certain rules) the work to the **instances**.



Microservices load balancing

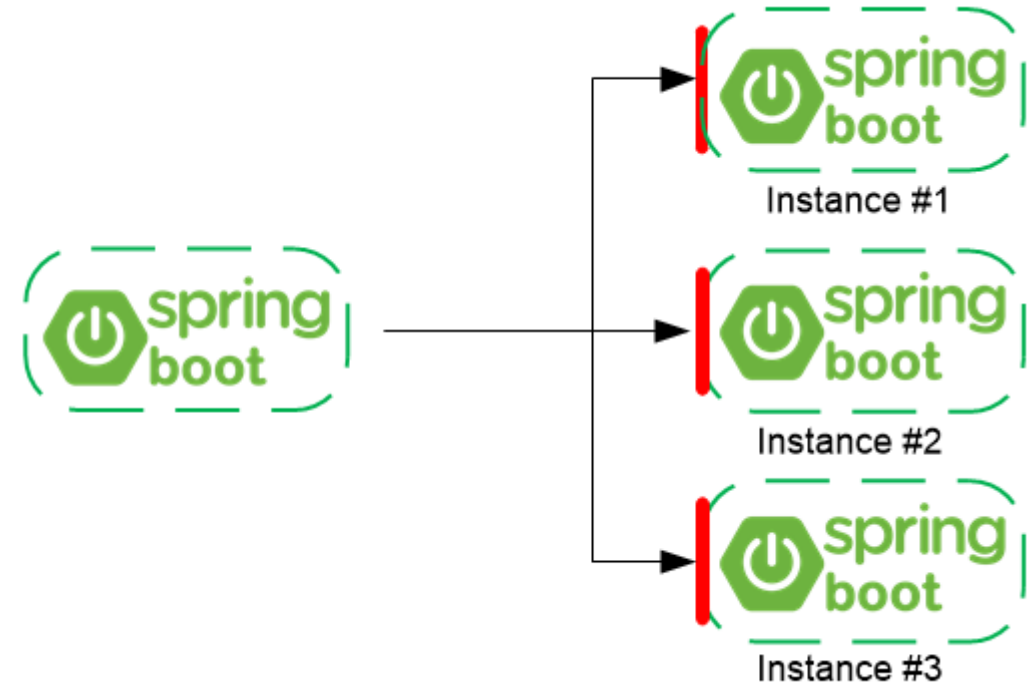
Load balancing is the process of distributing a set of tasks over a set of resources, with the aim of making their overall processing more efficient.

Load balancing can optimize the response time and avoid unevenly overloading some compute nodes while other compute nodes are left idle.

In microservices arch., there are **two type**:

- **Server side** load balancing, and
- **Client side** load balancing.

In clients side load balancing, client handles the load balancing. In this case, the client should know the addresses of all instances of a service, which can be done through a service registry.



Practical session 5

Services discovery – Eureka

The project is composed of **three different Spring Boot projects** that represent two different microservices (the same as last week), and another service to work as **a discovery server**.

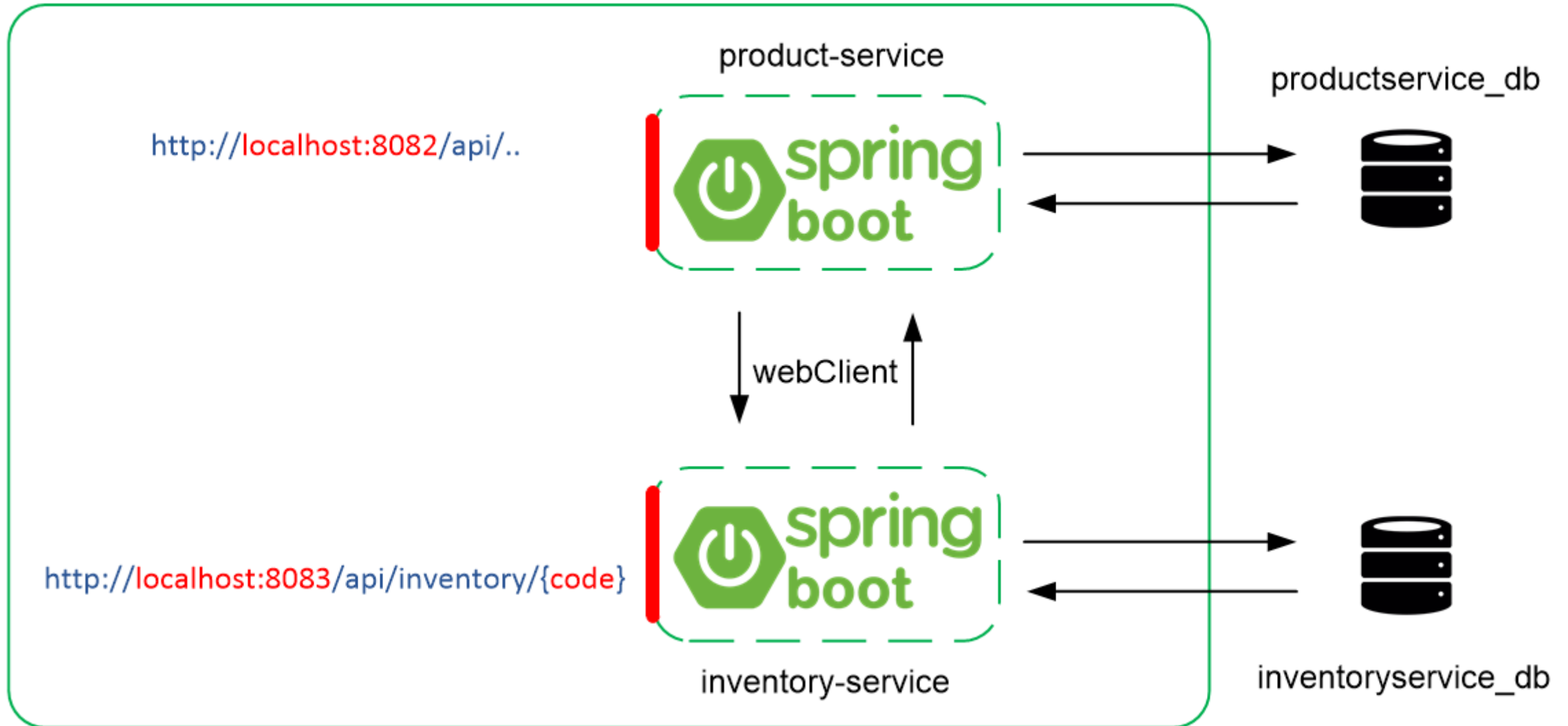
We will create the **discovery server** and **register** the other **two services** with it.

We will also enable **a client-side load balancing** for one service.

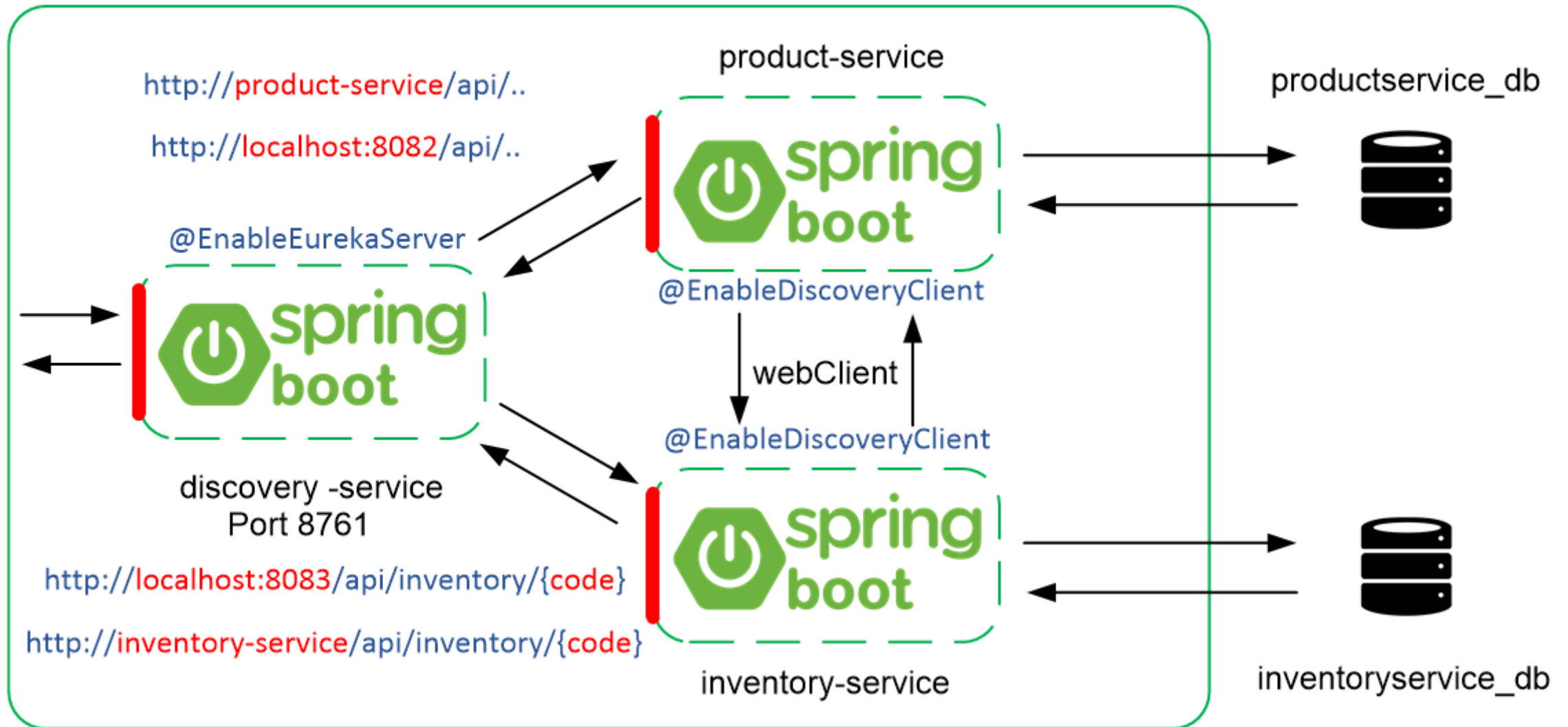
We just need to change its port to **0** in its **application.properties**.

When the port is changed to **0**, Spring Boot will assign an available port for each new instance of the service. **getWebClientBuilder** in the **ProductServiceApplication**

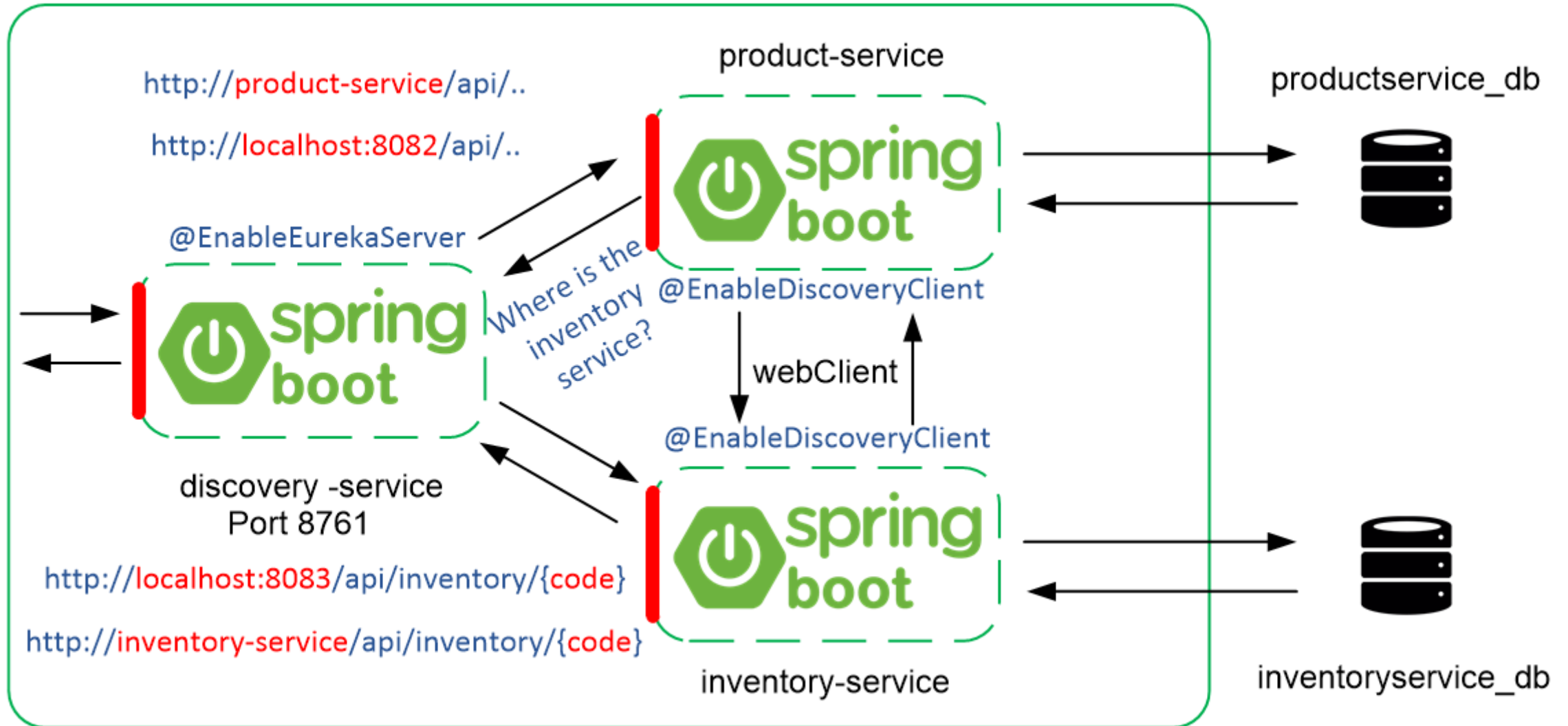
Services discovery



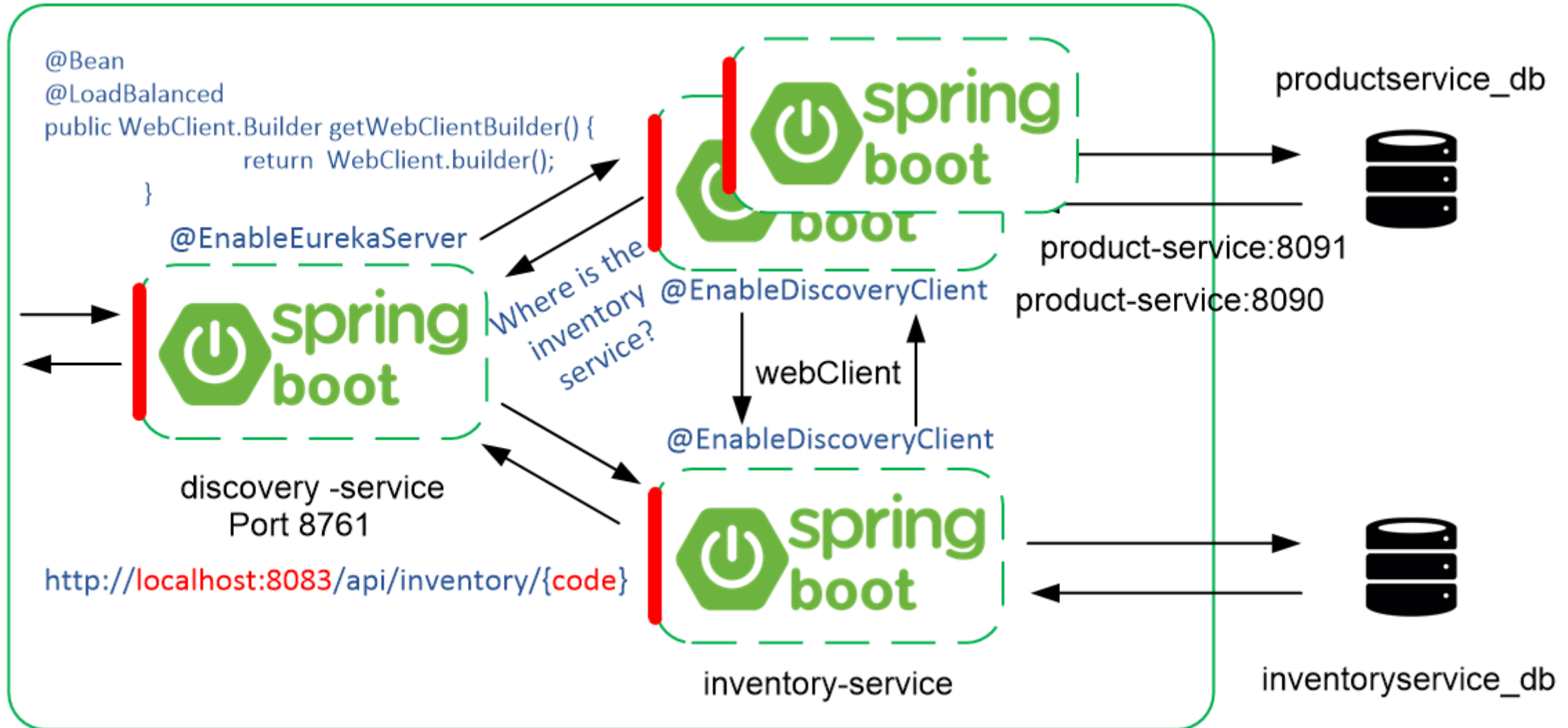
Services discovery



Services discovery



Microservices loadbalancing



API Gateway



API Gateway

What is an API Gateway?

API Gateway

What is an API Gateway?

API Gateway acts as a "front door" for applications to access data, business logic, or functionality from your backend services.

API Gateway can serve as a protection layer, administering security, scalability, and high availability of services.

API Gateway can serve as the single-entry point for defined back-end APIs and microservices.

Advantages of API Gateway?

API Gateway

What is an API Gateway?

API Gateway acts as a "front door" for applications to access data, business logic, or functionality from your backend services.

API Gateway can serve as a protection layer, administering security, scalability, and high availability of services.

API Gateway can serve as the single-entry point for defined back-end APIs and microservices.

Advantages of API Gateway

- An API gateway is a way to **decouple** the client interface from your backend implementation.
- **Simplifies** client **interaction** as it need to access only **a single point**. The client will not know the **internal architecture of the system**.
- They can handled **Authentication**.
- They can improve the **security** of the application (Microservices Arch.) as access of external calls to all the microservices can be **controlled**.

API Gateway

Disadvantages of API Gateway?

Advantages of API Gateway

- An API gateway is a way to **decouple** the client interface from your backend implementation.
- **Simplifies** client **interaction** as it need to access only **a single point**. The client will not know the **internal architecture of the system**.
- They can handled **Authentication**.
- They can improve the **security** of the application (Microservices Arch.) as access of external calls to all the microservices can be **controlled**.

API Gateway

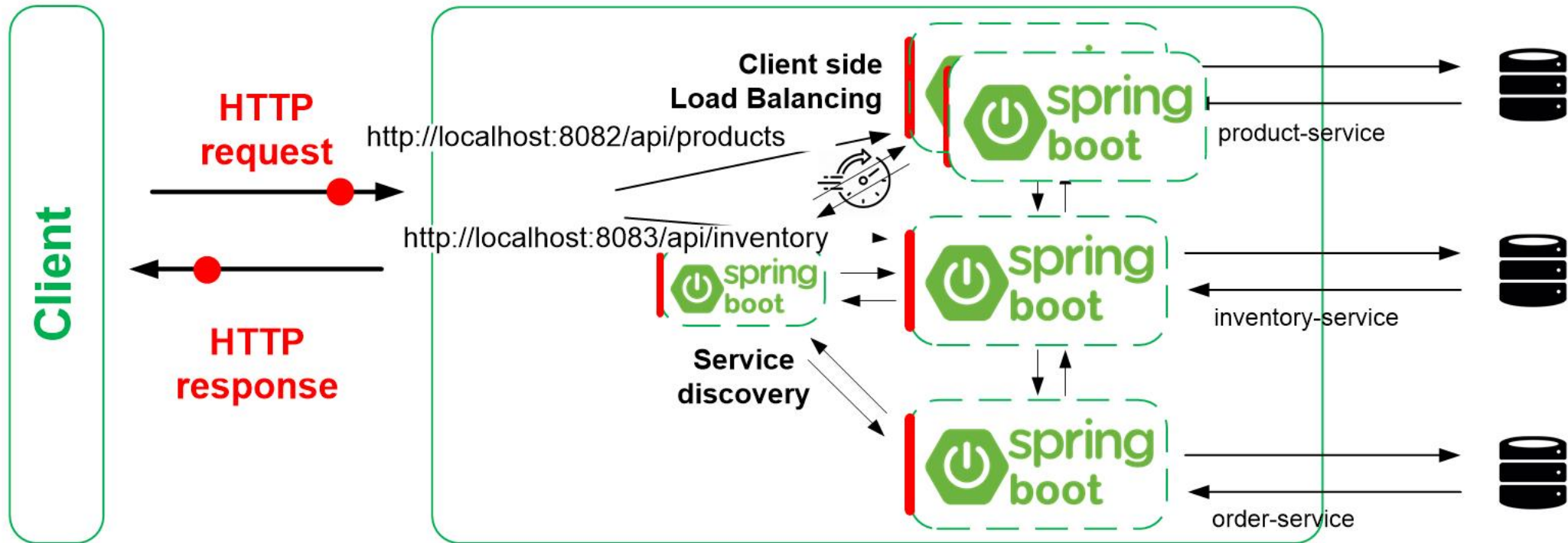
Disadvantages of API Gateway

- They can cause **performance degrade (e.g., delay)**.
- A discovery service is **a must** for the functionality of an **API gateway**, i., e., they **depend** on other services.
- If they **fail**, everything may **fail** – **a single point of failure**.

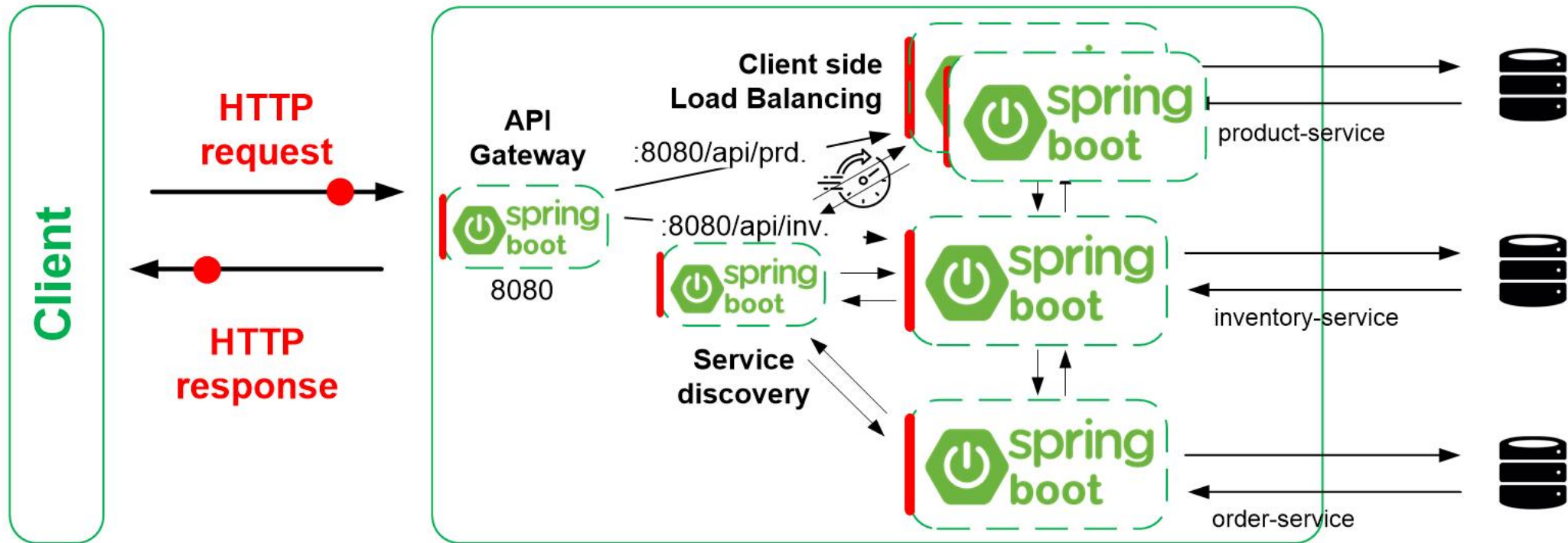
Advantages of API Gateway

- An API gateway is a way to **decouple** the client interface from your backend implementation.
- **Simplifies** client **interaction** as it need to access only **a single point**. The client will not know the **internal architecture of the system**.
- They can handled **Authentication**.
- They can improve the **security** of the application (Microservices Arch.) as access of external calls to all the microservices can be **controlled**.

API Gateway



API Gateway

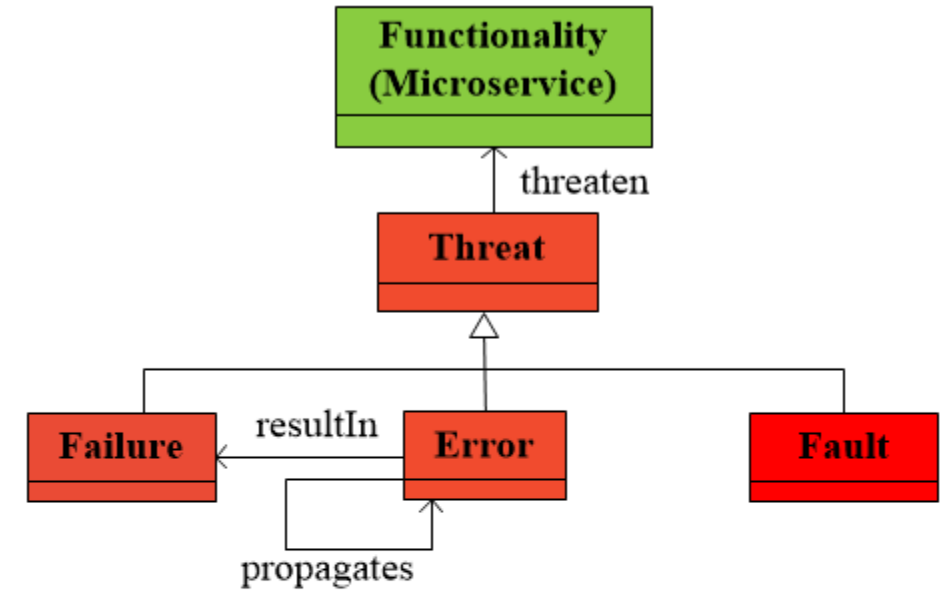


Resilient microservices patterns

Key Dependability/Resilience Concepts

A *threat* represents a potential incident that can *threaten* that may negatively influence the expected functionality of a microservice and/or the overall system.

A *threat* can be specialized into three different concepts a *fault*, an *error*, and a *failure*.

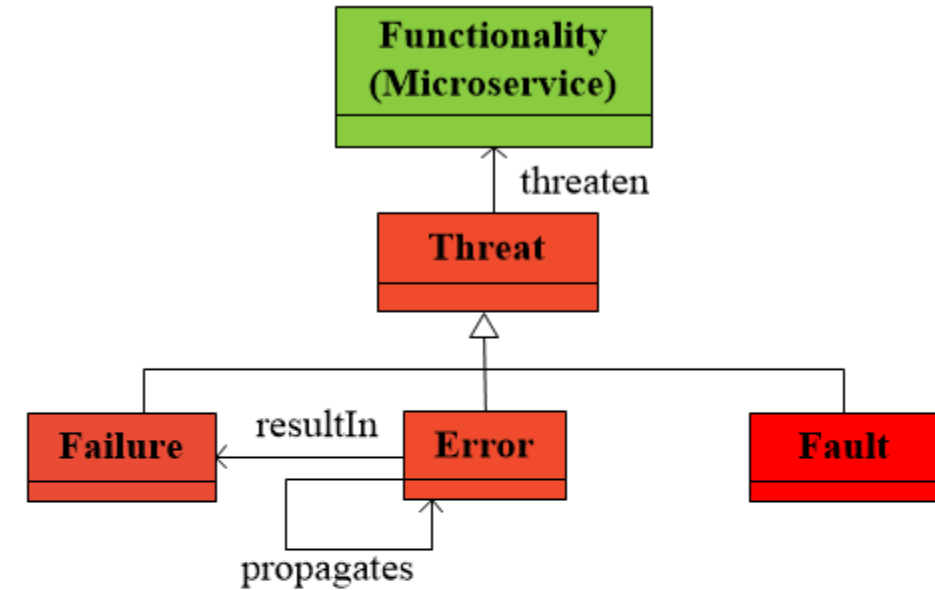


Key Dependability/Resilience Concepts

A *threat* represents a potential incident that can *threaten* that may negatively influence the expected functionality of a microservice and/or the overall system.

A *threat* can be specialized into three different concepts a *fault*, an *error*, and a *failure*.

- A *fault* is a “defect” in the system that may produce an *error*.
- An *error* is an undesirable state that may *propagate* into other *errors*, and it can *result in* a *failure*.
- A *failure* is a transition from a correct (expected) behavior to an incorrect one at the level of a microservice and/or the overall system.

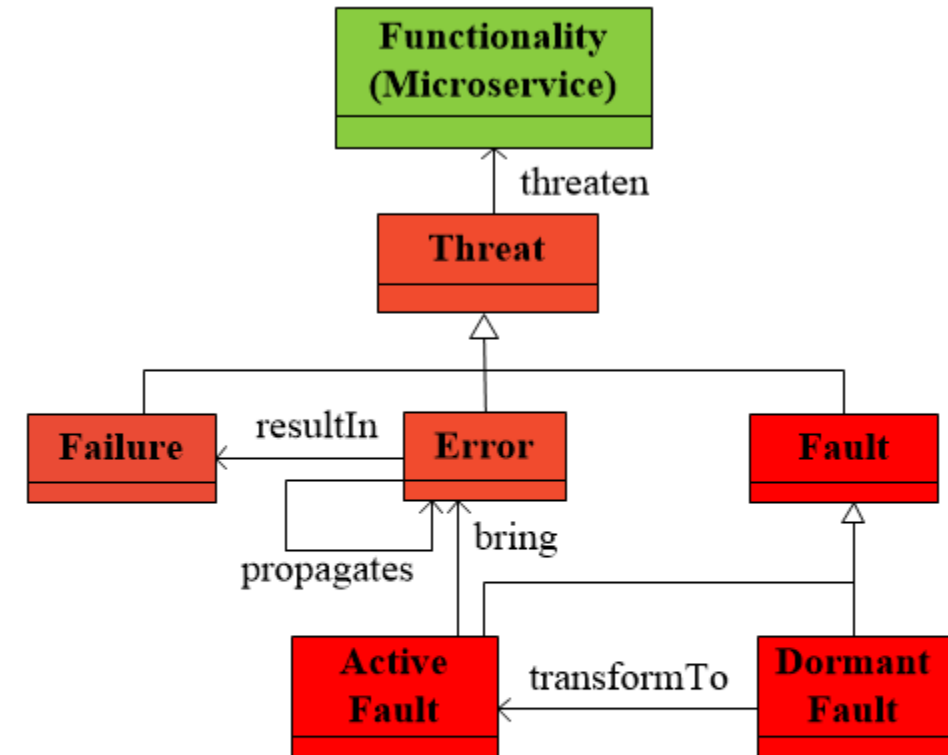


Key Dependability/Resilience Concepts

A *threat* represents a potential incident that can *threaten* that may negatively influence the expected functionality of a microservice and/or the overall system.

A *threat* can be specialized into three different concepts a *fault*, an *error*, and a *failure*.

- A *fault* is a “defect” in the system that may produce an *error*. A *fault* can be either
 - A *active fault* that brings an *error*, or
 - A *dormant fault* that can be triggered by some events and *transforms* into an *active fault*, which brings an *error*.



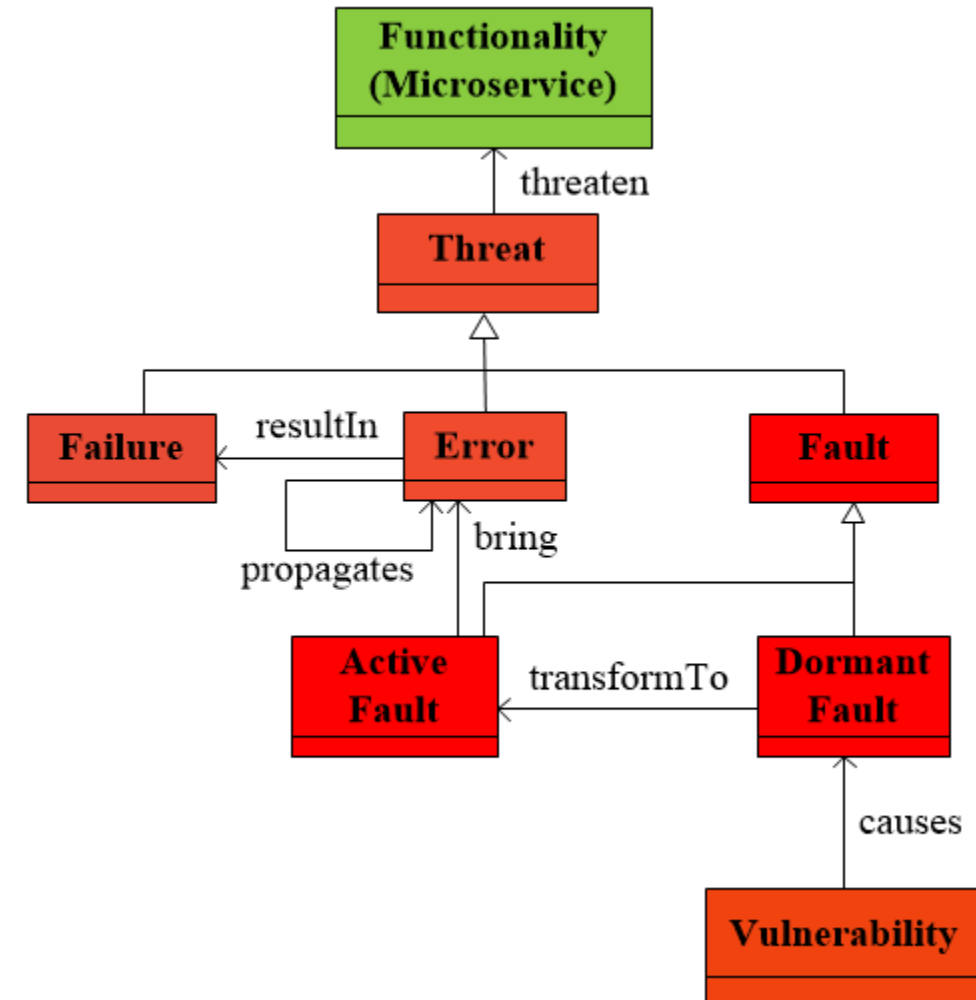
Key Dependability/Resilience Concepts

A *threat* represents a potential incident that can *threaten* that may negatively influence the expected functionality of a microservice and/or the overall system.

A *threat* can be specialized into three different concepts a *fault*, an *error*, and a *failure*.

- A *fault* is a “defect” in the system that may produce an *error*. A *fault* can be either
 - A *active fault* that brings an *error*, or
 - A *dormant fault* that can be triggered by some events and *transforms* into an *active fault*, which brings an *error*.

A *vulnerability* is a weakness in the system that can lead to a *dormant fault*.



Key Dependability/Resilience Concepts

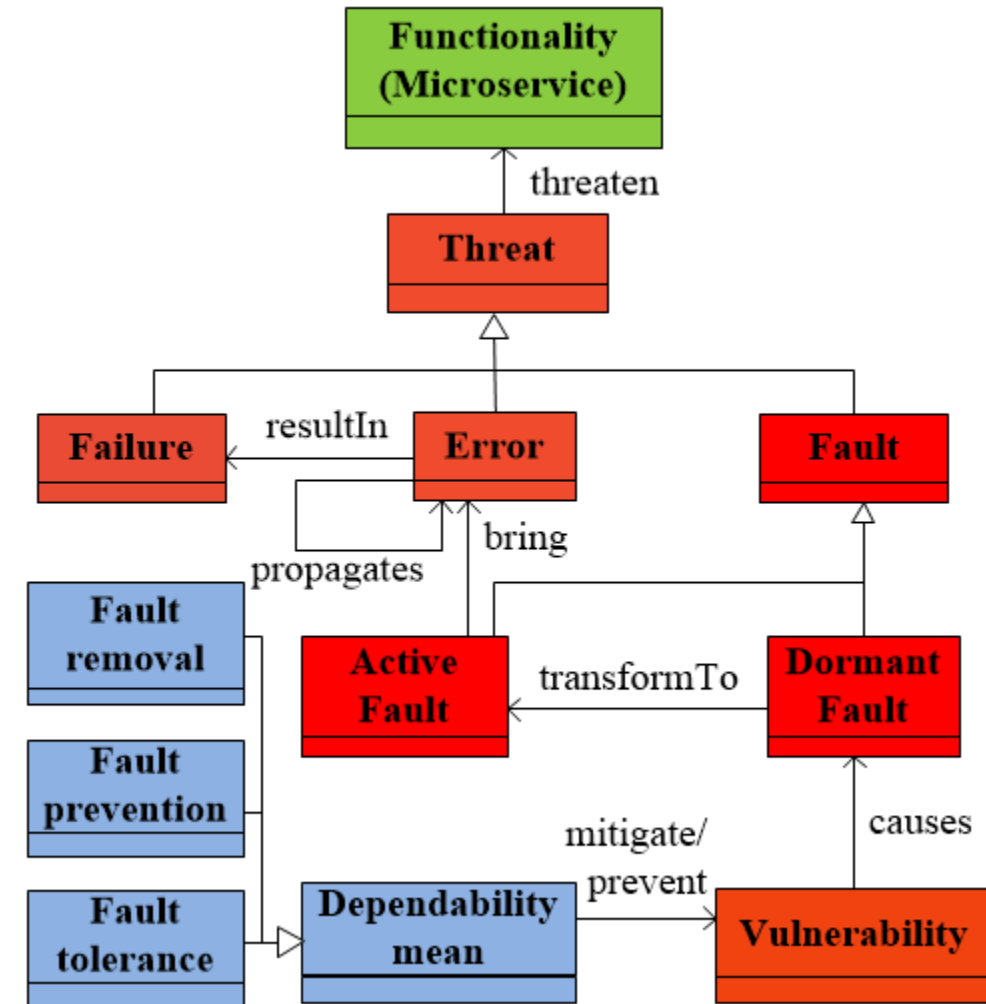
A **dependability mean** can be defined as a countermeasure/technique to be implemented in order to *mitigate/prevent* the occurrence/existence of a *vulnerability*.

Dependability means can be broadly grouped into:

Fault removal aims at reducing the number and severity of faults, where the optimal goal of fault removal is detecting the presence of faults during the system design, and then to remove them;

Fault prevention aims at preventing the occurrence or introduction of faults;

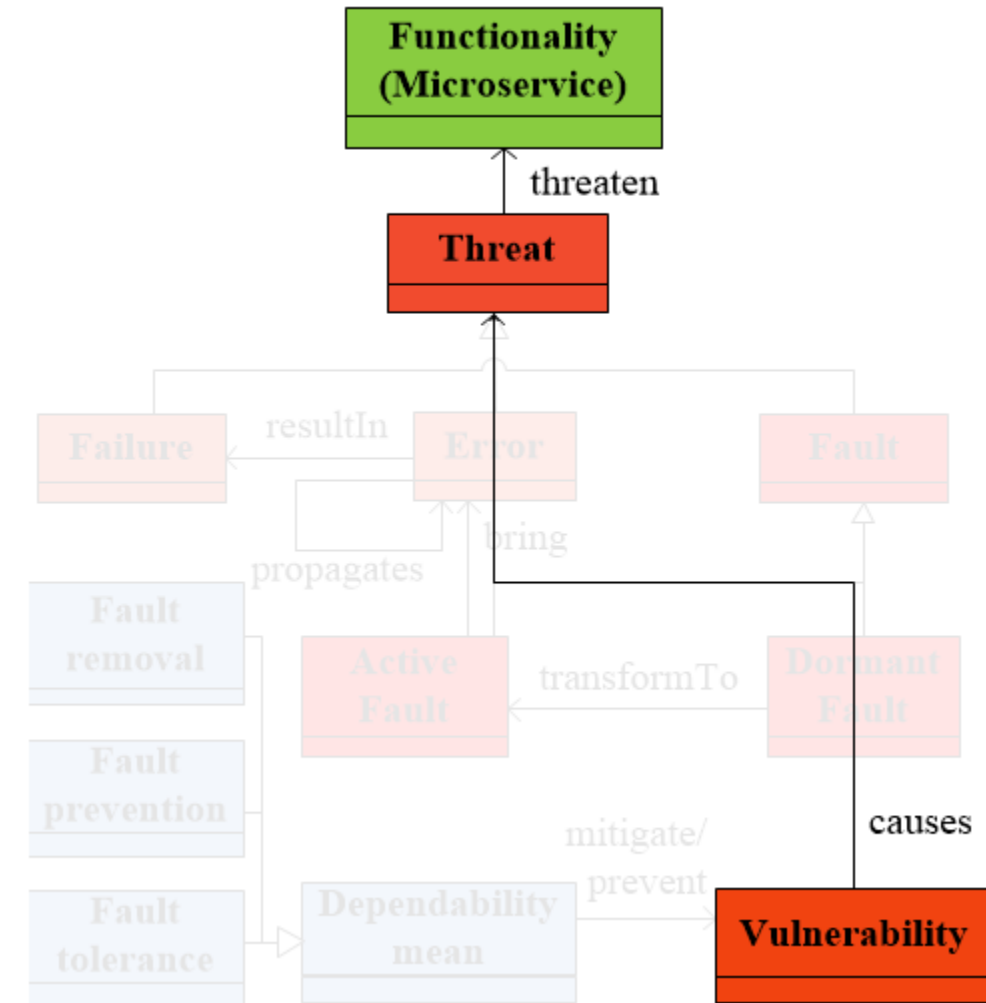
Fault tolerance aims at avoiding failures in the presence of faults.



Key Dependability/Resilience Concepts

A **vulnerability** is a weakness in the system that can lead to a *dormant fault*.

A **vulnerability** may **cause** a **threat** that may negatively influence the expected **functionality of a microservice** and/or the overall system.



Key Dependability/Resilience Concepts

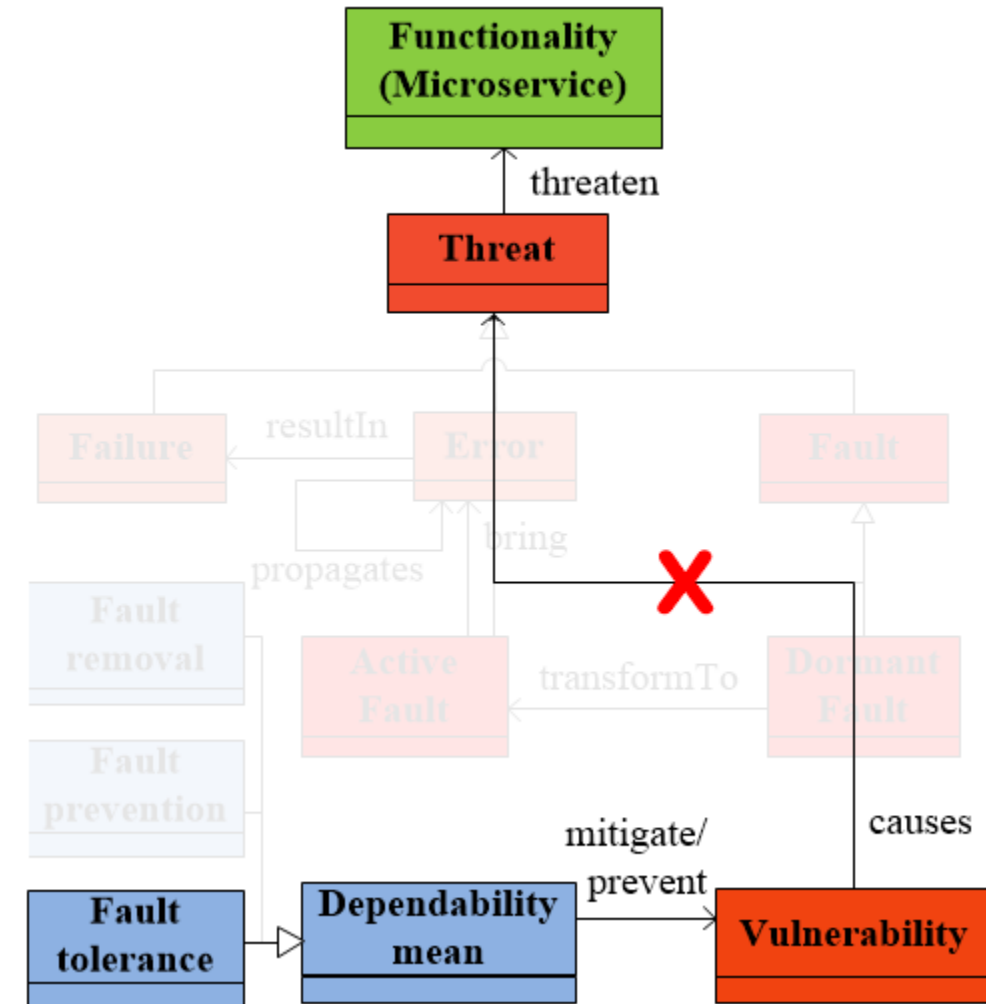
A **vulnerability** is a weakness in the system that can lead to a *dormant fault*.

A **vulnerability** may **cause** a **threat** that may negatively influence the expected **functionality of a microservice** and/or the overall system.

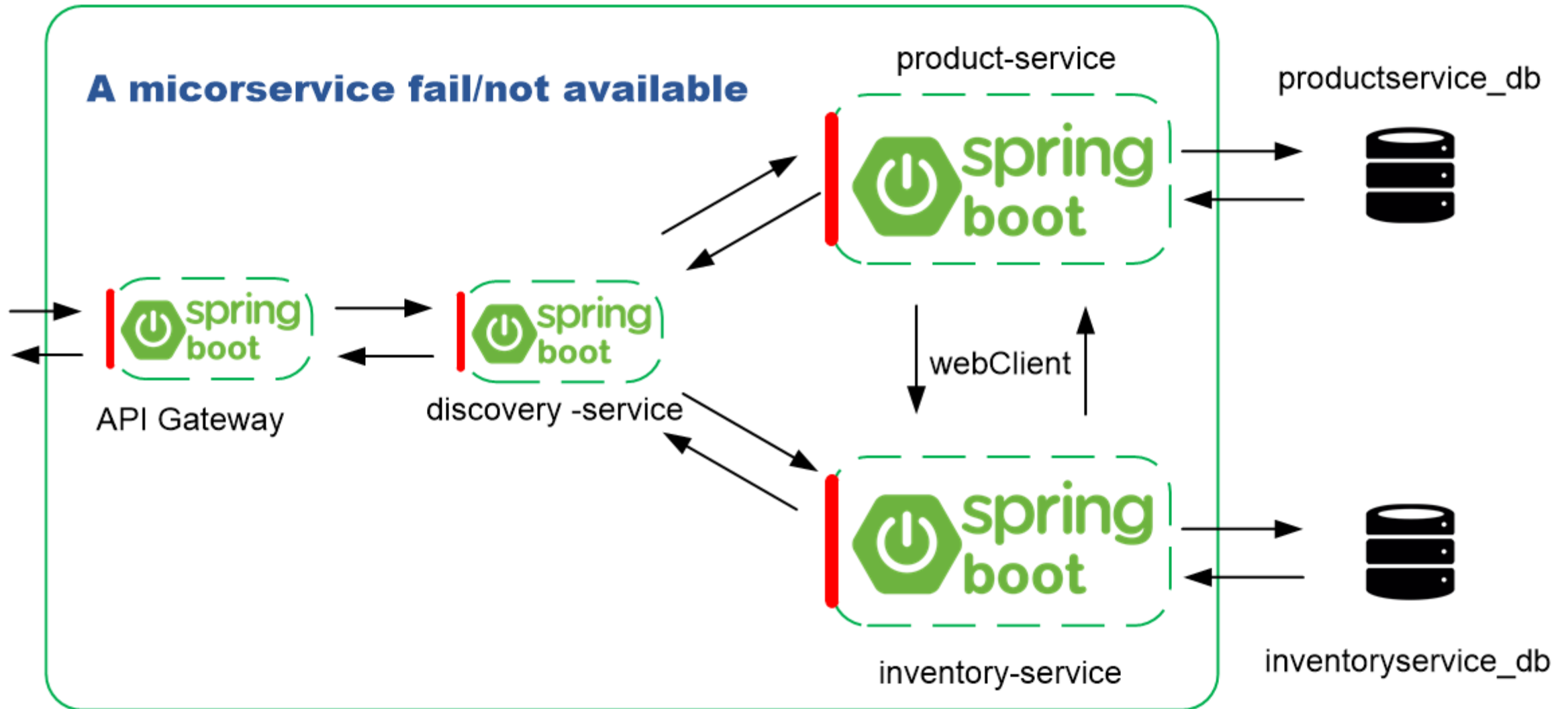
A **dependability mean** to be implemented to mitigate/prevent the **occurrence/existence** of a **vulnerability**.

Which **dependability mean** we need?

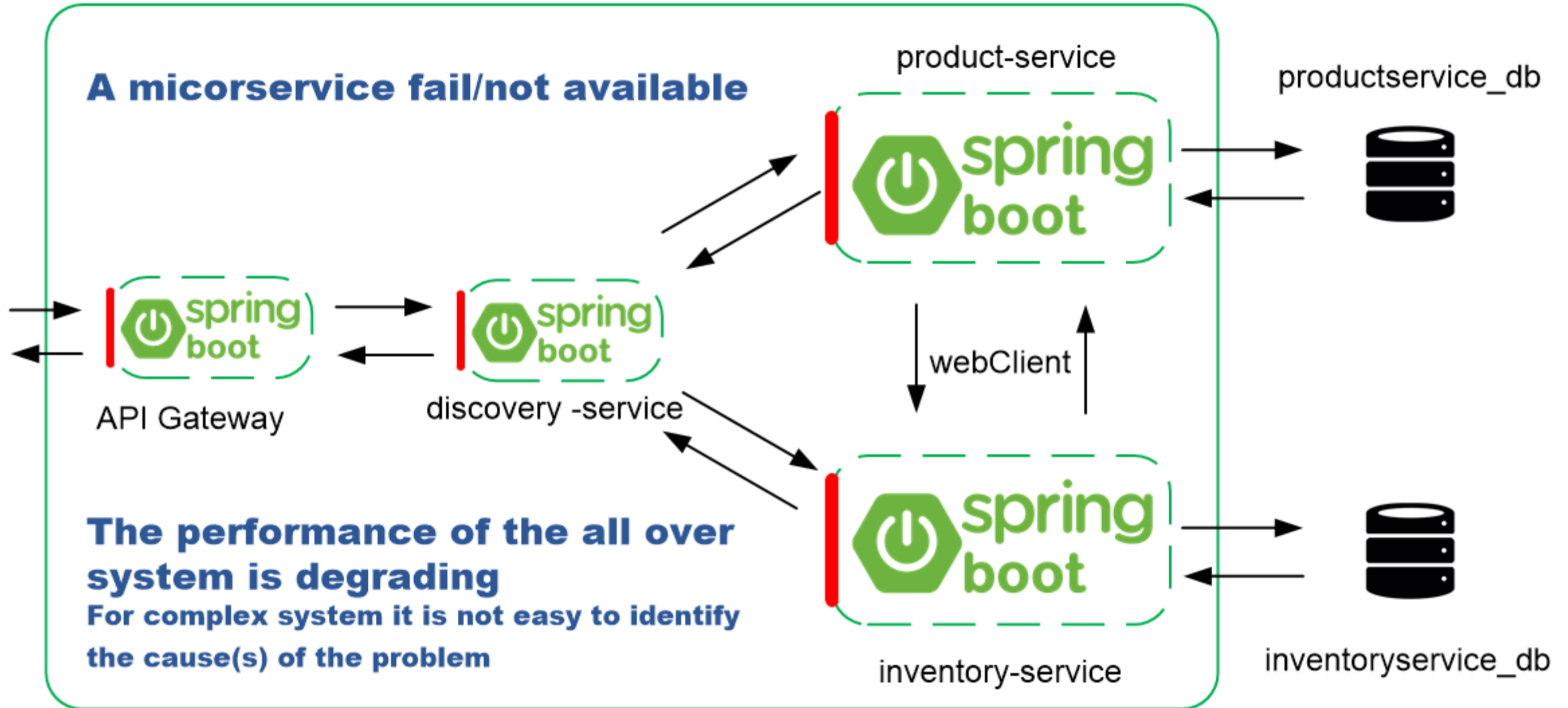
Fault tolerance aims at avoiding failures in the presence of **faults**.



What may go wrong (vulnerabilities)?



What may go wrong (vulnerabilities)?

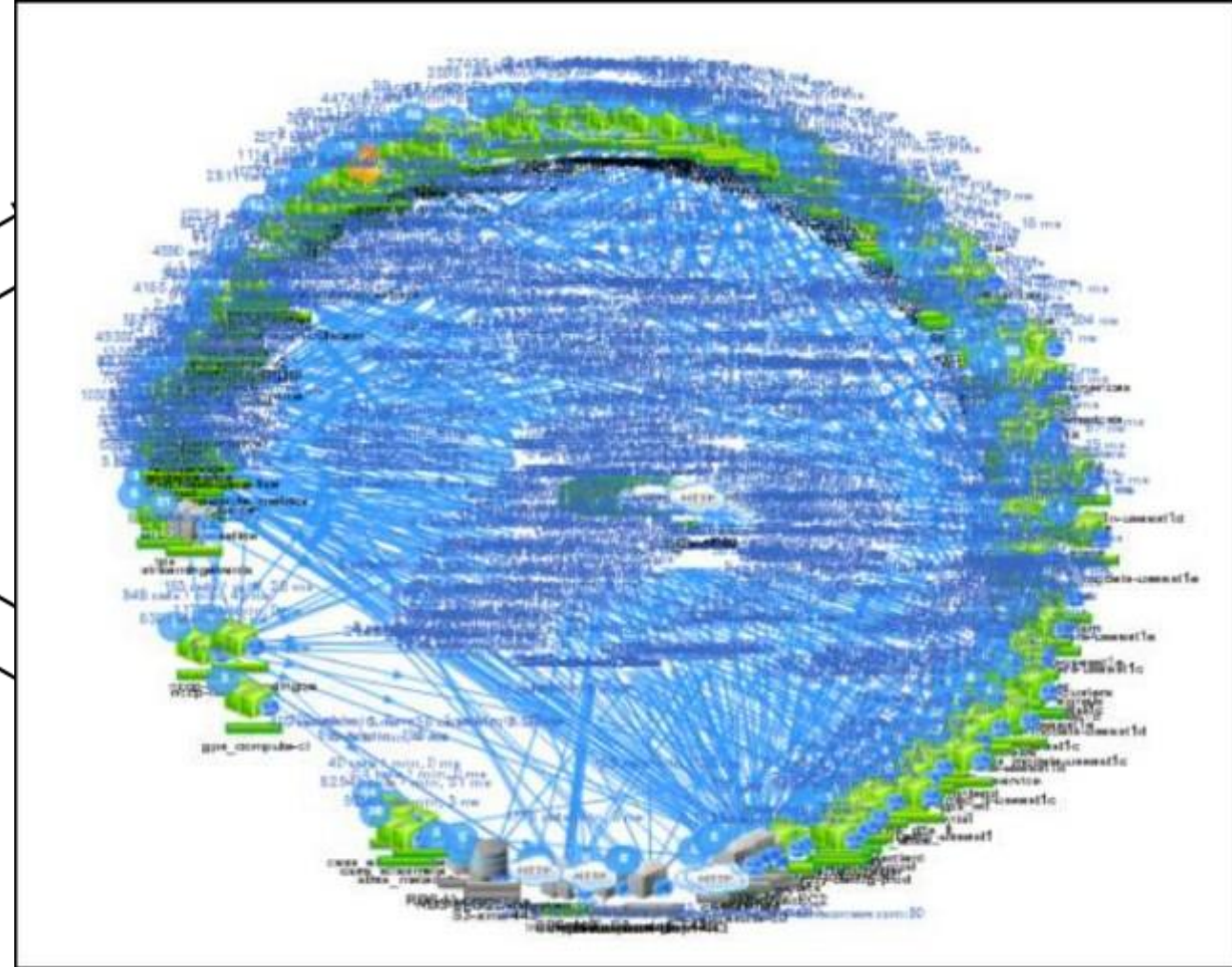


What may go wrong (vulnerabilities)?

A microservice fail/not available

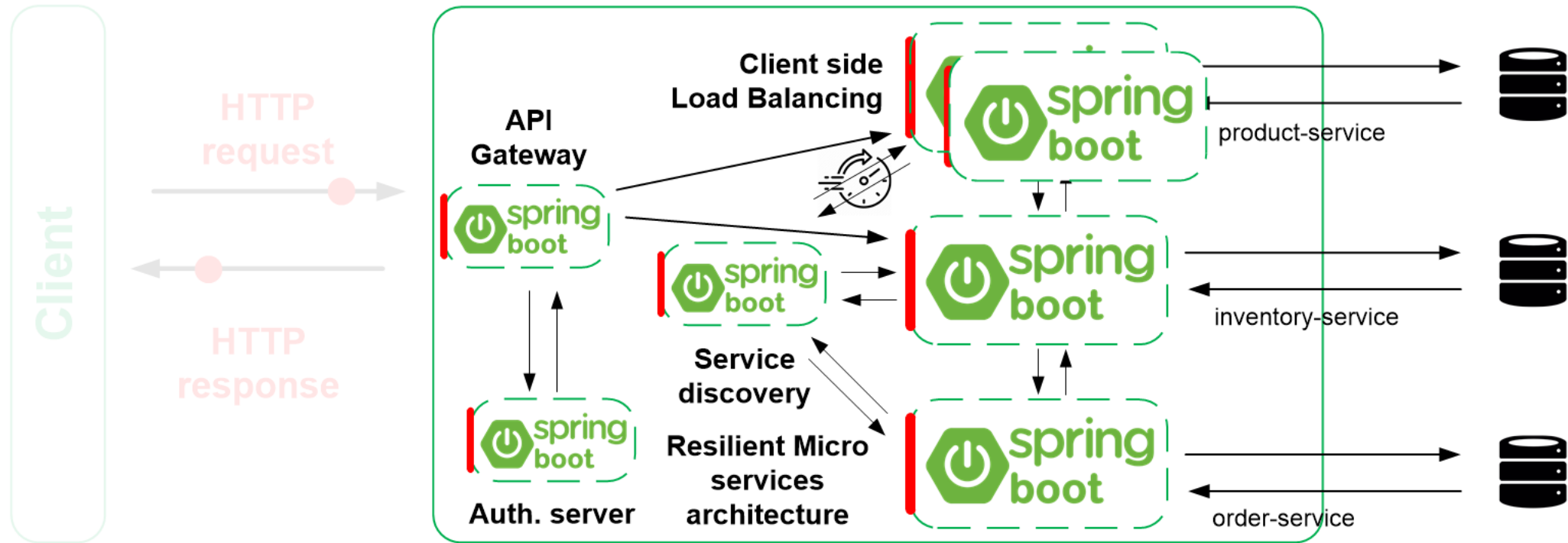


The performance of the all over system is degrading
For complex system it is not easy to identify the cause(s) of the problem



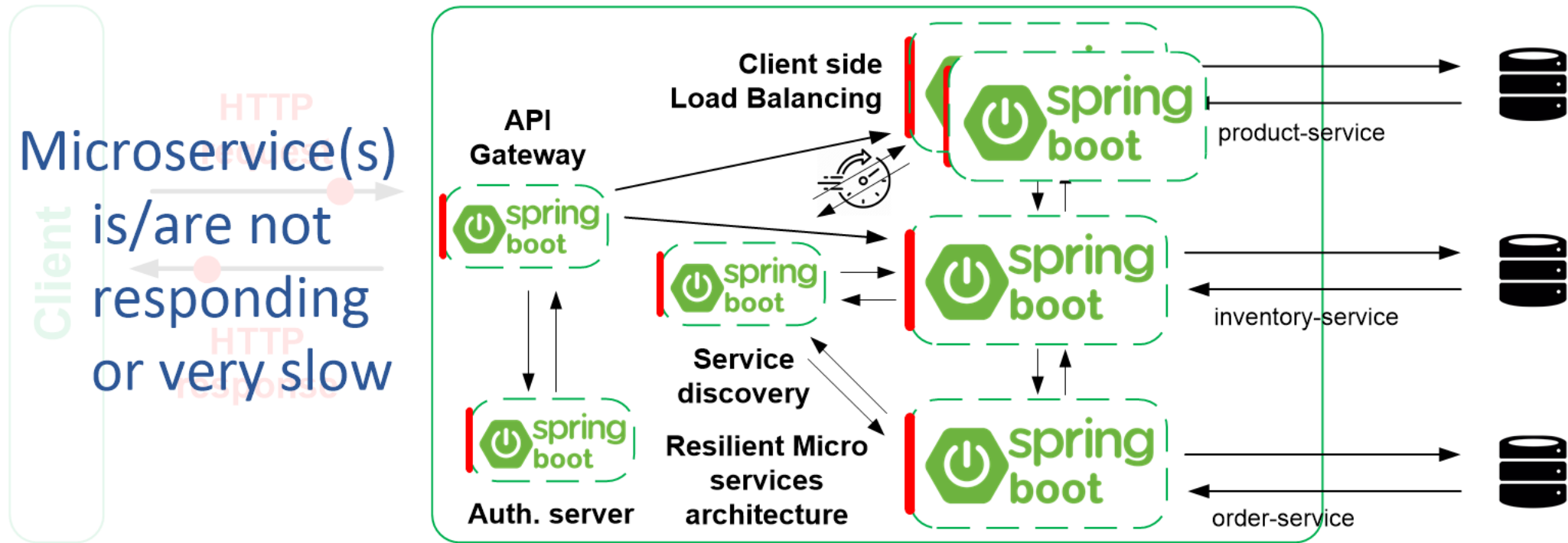
Resilient microservices patterns

What may lead to performance degradation?



Resilient microservices patterns

What may lead to performance degradation?





Resilient microservices patterns

How we can solve that?

Resilient microservices patterns

How we can solve that?

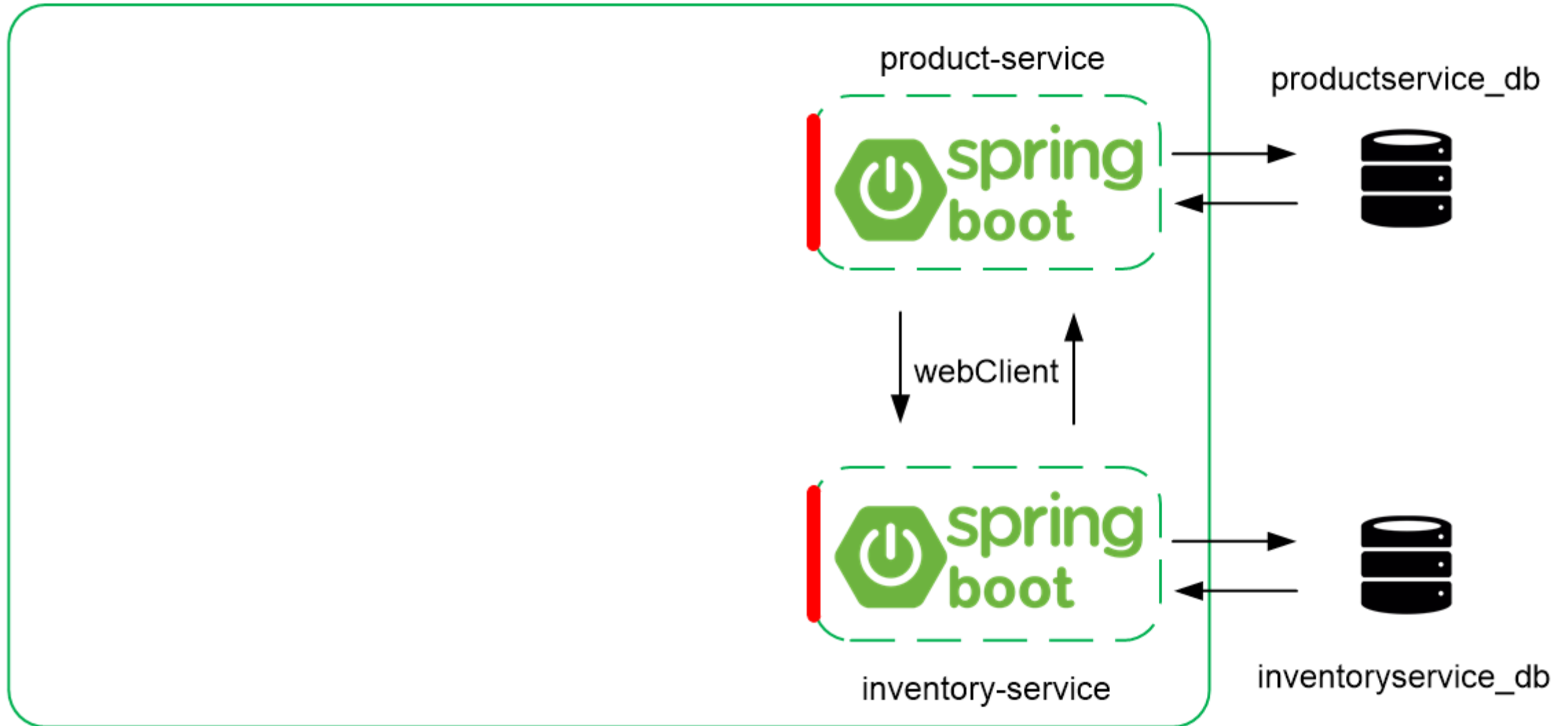
The Circuit Breaker pattern can be used to **prevent calls to a remote service** if we know that the call is **likely to fail**.

Time Limiter pattern can be used set a **threshold timeout value** for calls made to external systems. Unlike **CB**, **time limiter** will not wait the request to fail, but it **will terminates** the request if the wait time has been **reached** with **no response**.

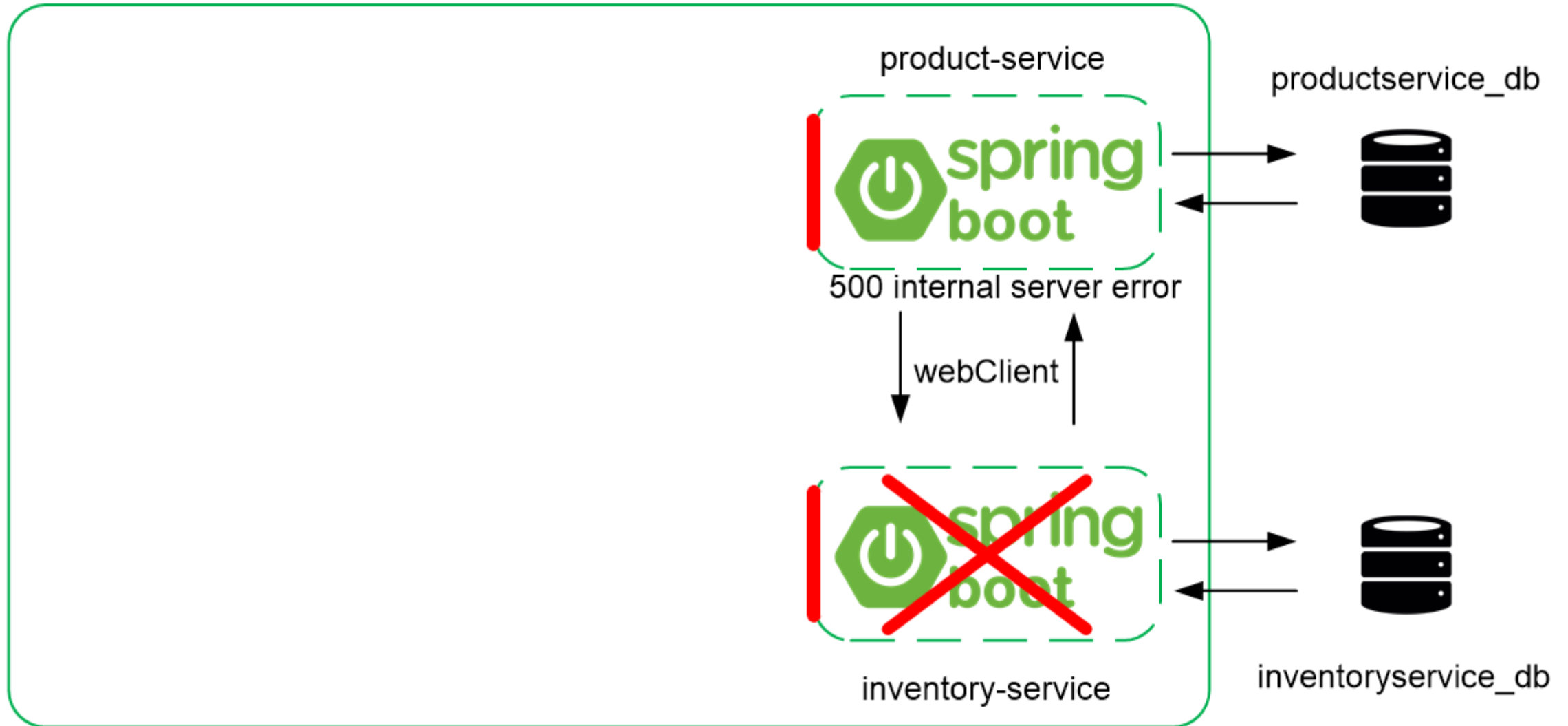
Retry pattern can be used to **retry** what has **failed** (the request).

Rate Limiter pattern can be used to make **services highly available** by **limiting the request** that can be “**send**” in a specific **window of time**.

Circuit Breaker



Circuit Breaker



Circuit Breaker

The **Circuit Breaker** pattern is derived from the **electrical switches**, which are designed to protect an electrical circuit from damage by **breaking the circuit**.

The idea of **circuit breakers**, in our work, is to **prevent calls to a remote service** if we know that the call is **likely to fail** since we do not waste critical resources both in our service and in the remote service.

Circuit Breaker

The **Circuit Breaker** pattern is derived from the **electrical switches**, which are designed to protect an electrical circuit from damage by **breaking the circuit**.

The idea of **circuit breakers**, in our work, is to **prevent calls to a remote service** if we know that the call is **likely to fail** since we do not waste critical resources both in our service and in the remote service.

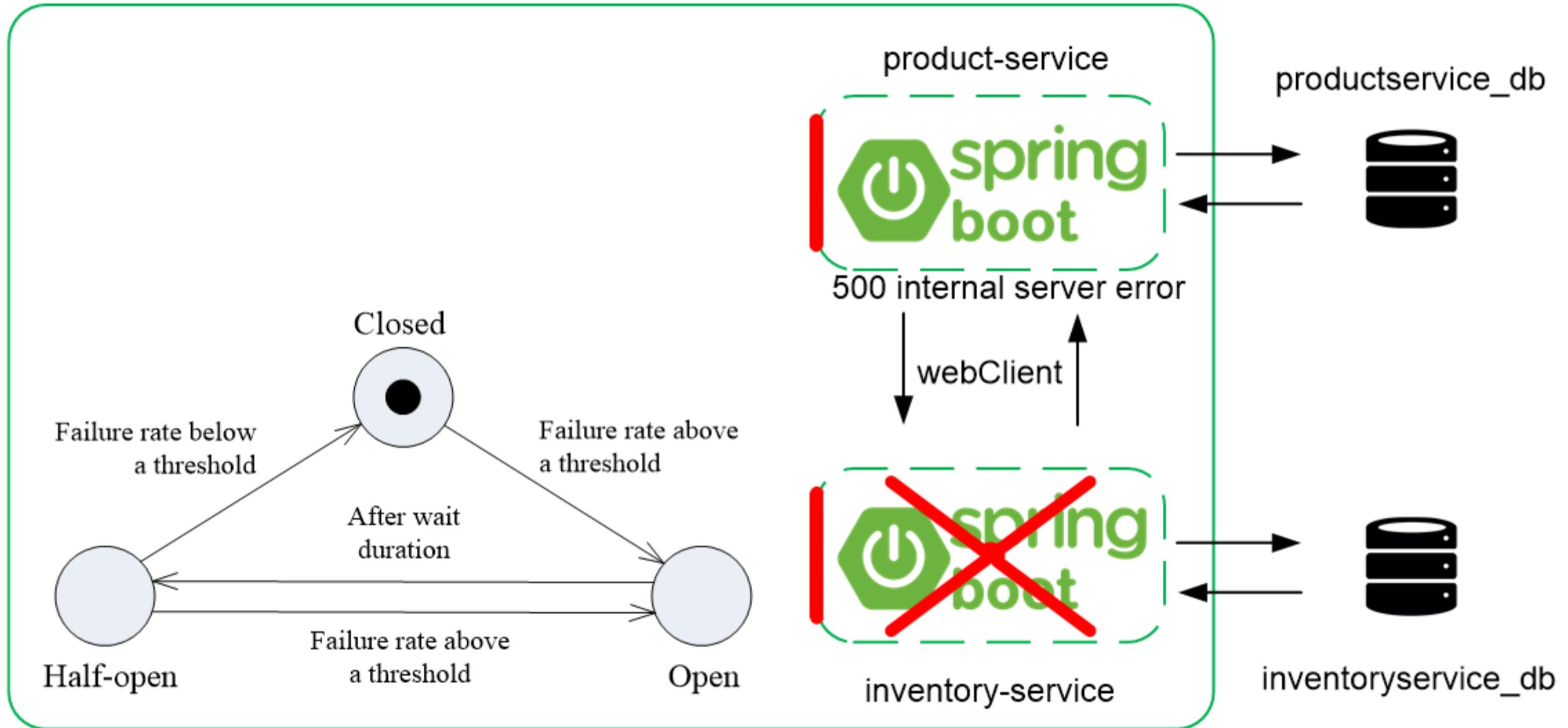
Circuit Breaker has 3 States:

CLOSED (default): this is the normal state in which all requests flow through the circuit without any restriction.

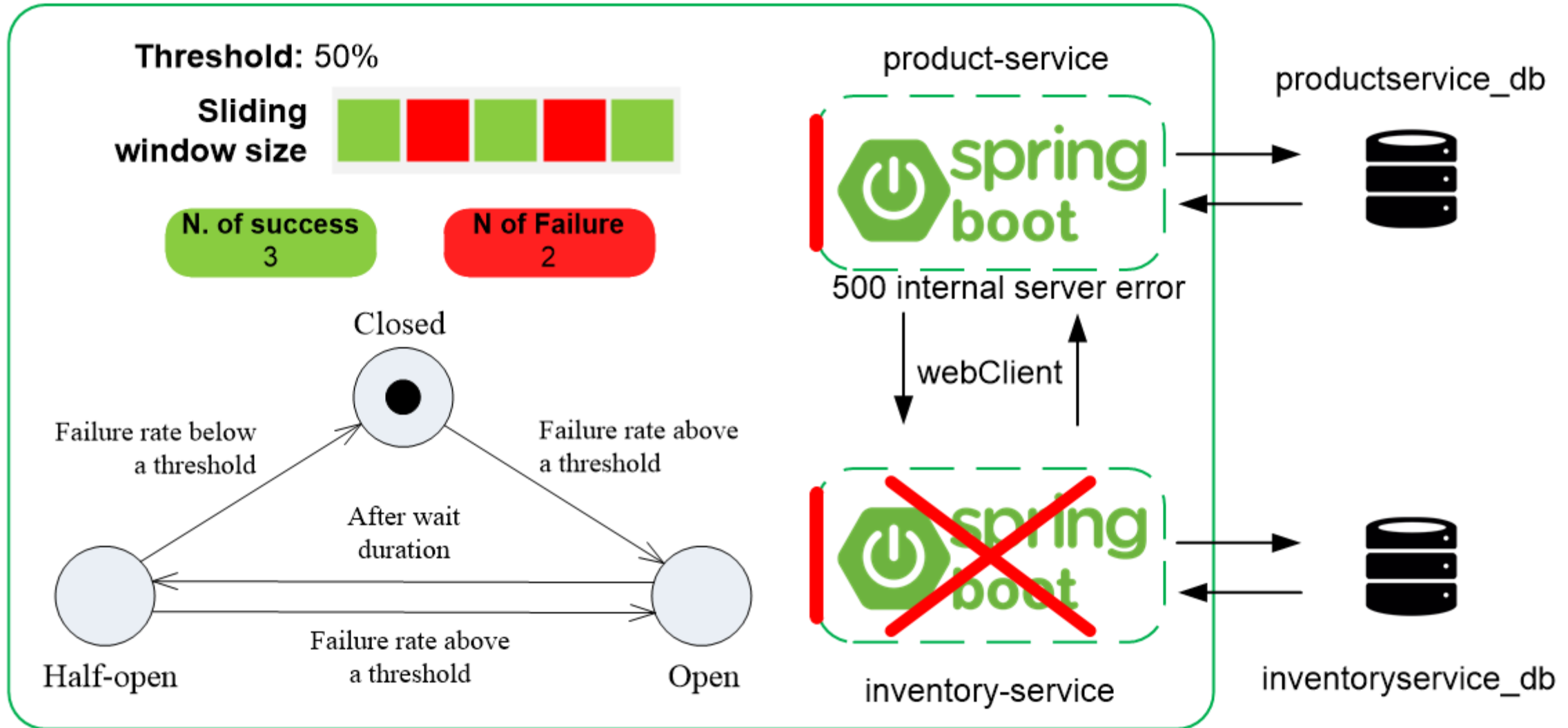
OPEN: in this state all calls will be restricted with `CallNotPermittedException`.

HALF_OPEN: After a configurable wait time in OPEN State, the circuit breaker allows a small number of requests to pass through.

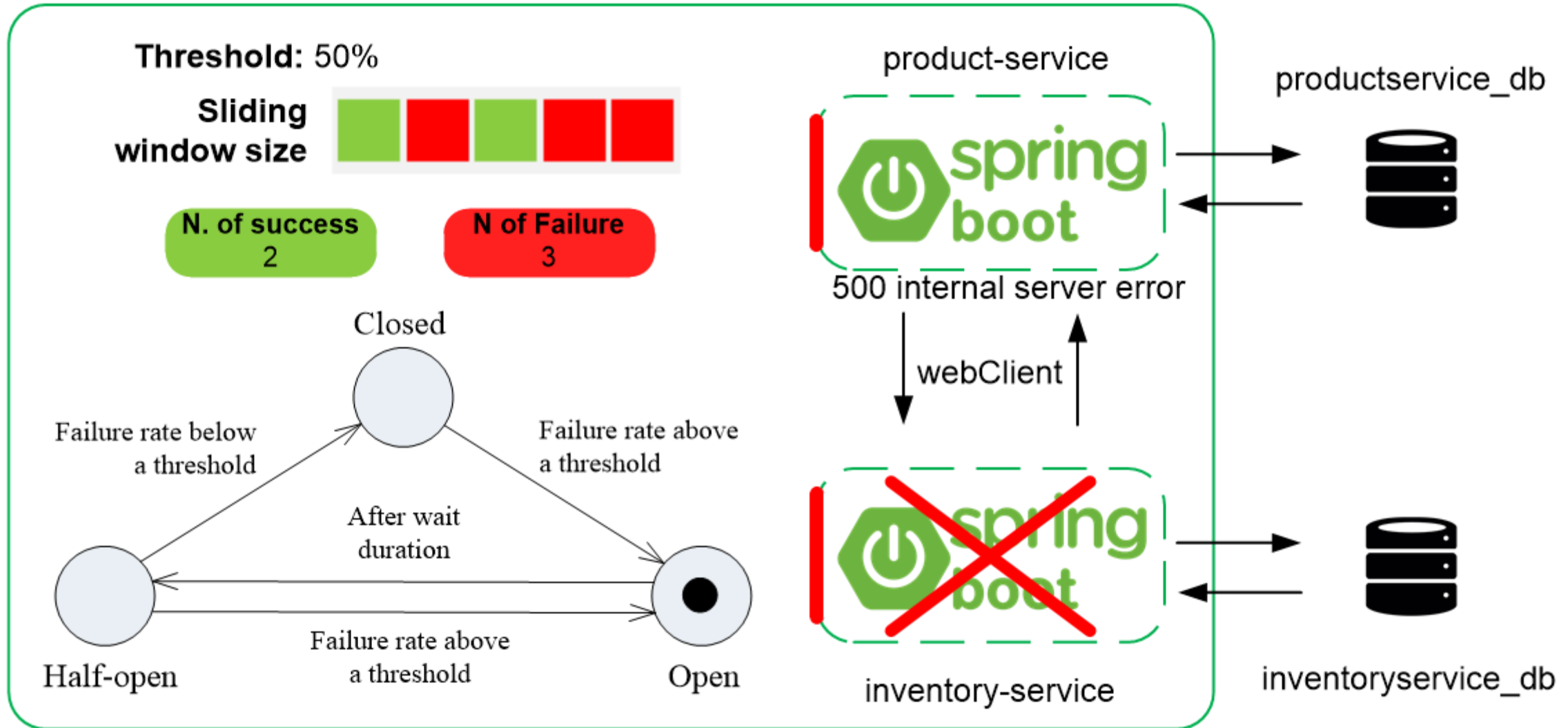
Circuit Breaker



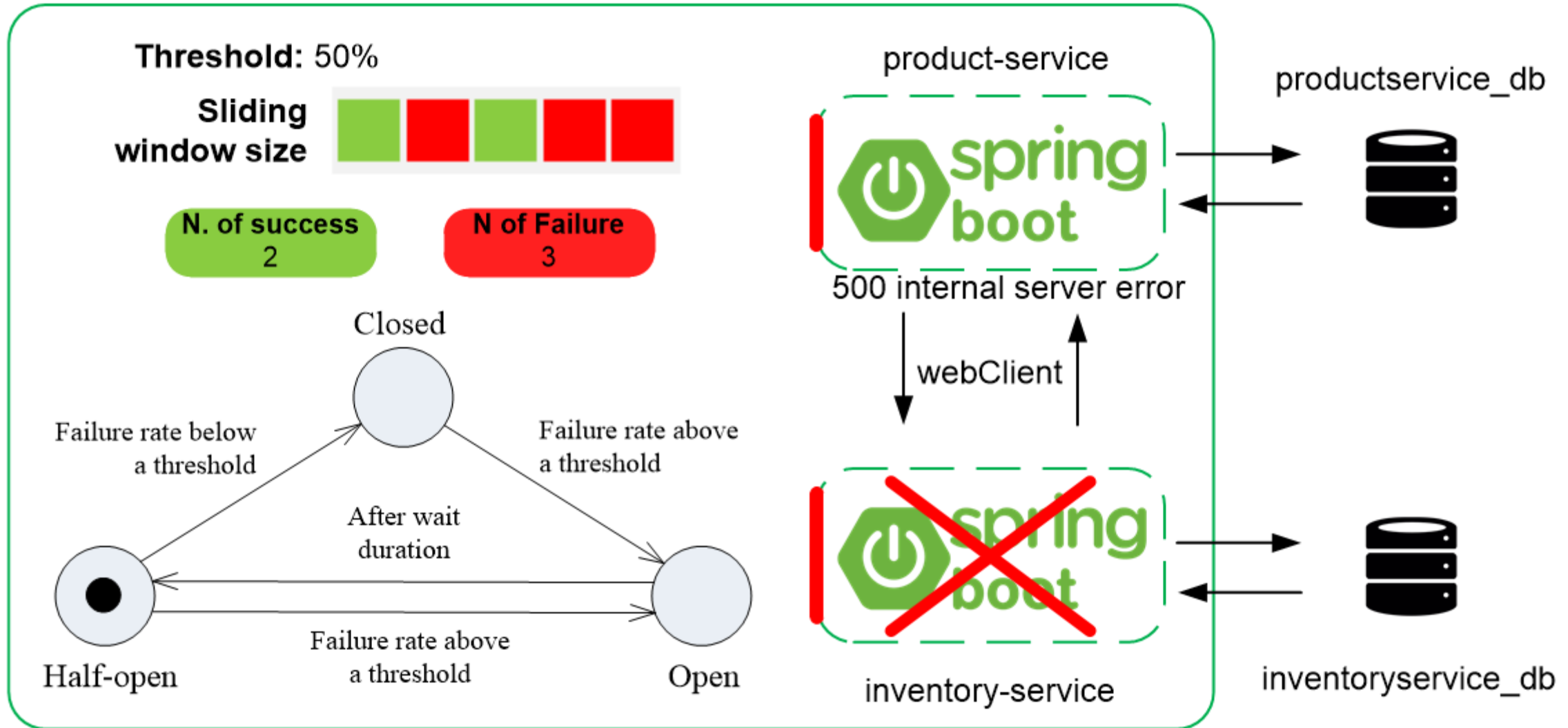
Circuit Breaker



Circuit Breaker



Circuit Breaker



Circuit Breaker

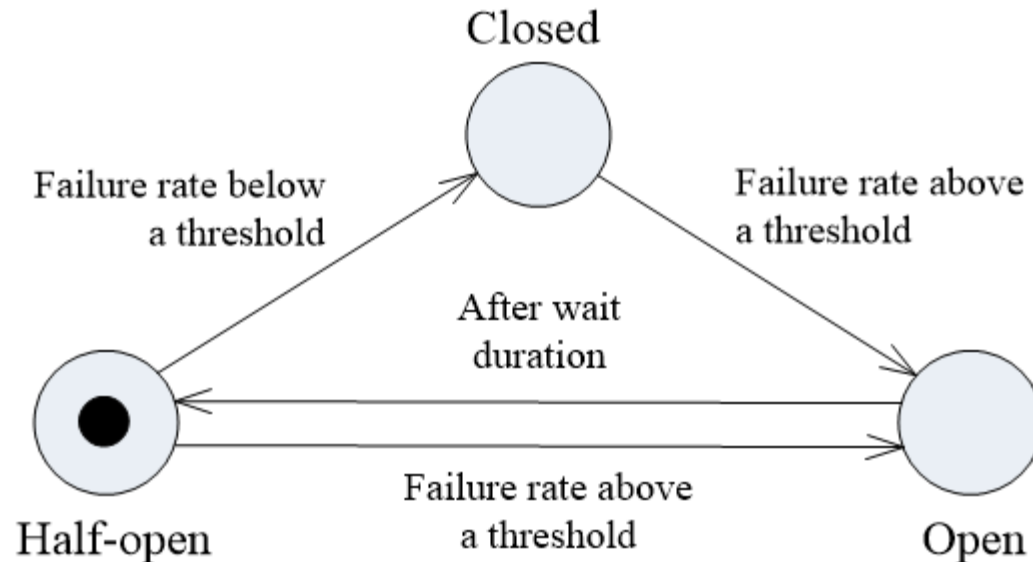
Threshold: 50%

Sliding
window size



N. of success
2

N of Failure
3



```
{
  "status": "UP",
  "components": {
    "circuitBreakers": {
      "status": "UP",
      "details": {
        "inventory": {
          "status": "UP",
          "details": {
            "failureRate": "-1.0%",
            "failureRateThreshold": "50.0%",
            "slowCallRate": "-1.0%",
            "slowCallRateThreshold": "100.0%",
            "bufferedCalls": 0,
            "slowCalls": 0,
            "slowFailedCalls": 0,
            "failedCalls": 0,
            "notPermittedCalls": 0,
            "state": "CLOSED"
          }
        }
      }
    }
  }
}
```

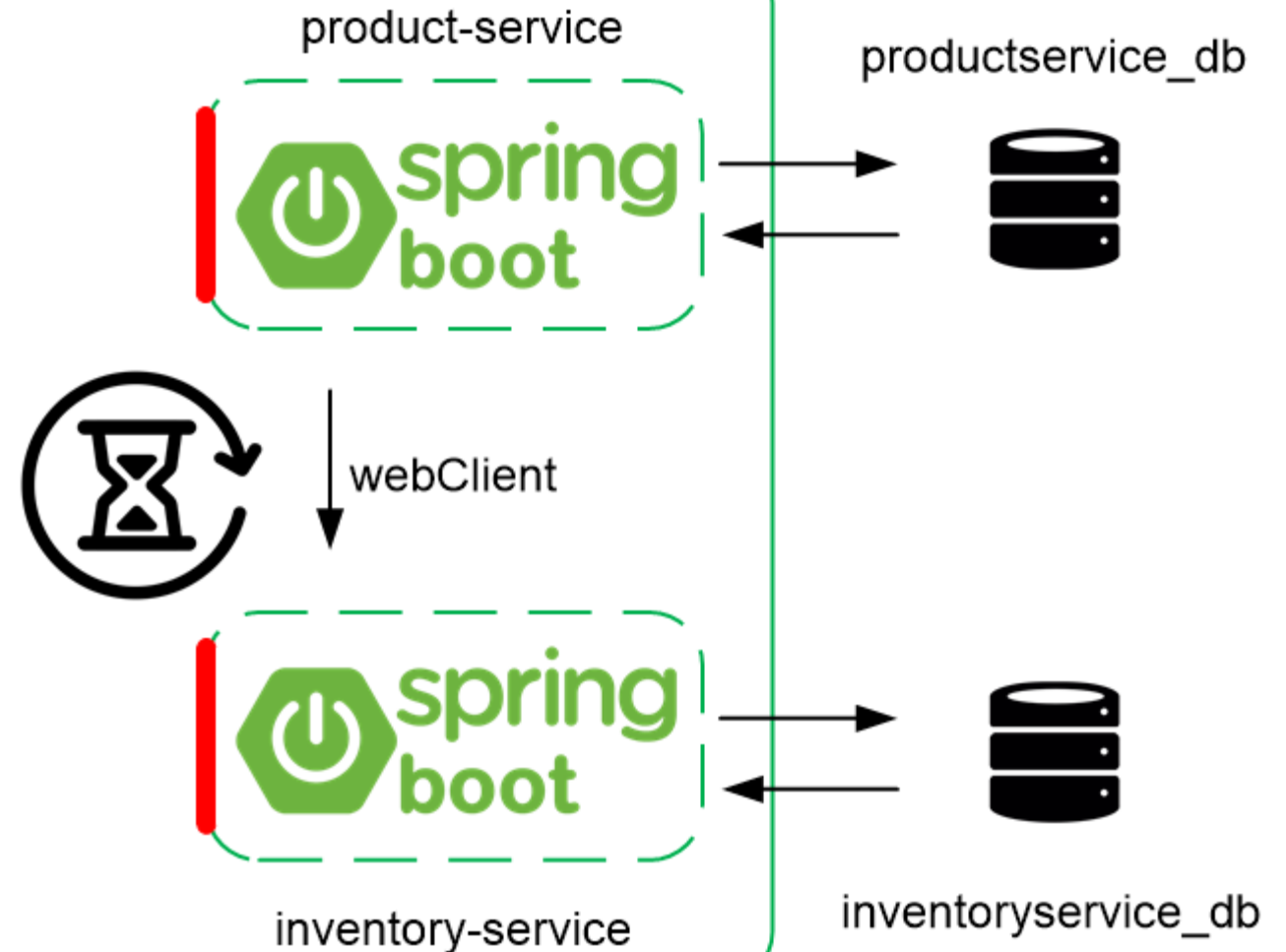
To view json in format not raw, you may install json-formatter plugin in your browser

You can visit (<http://localhost:8082/actuator/health>) to check the status of the CB

Port 8082 should be replaced with the port where the service of concern is running

Time Limiter

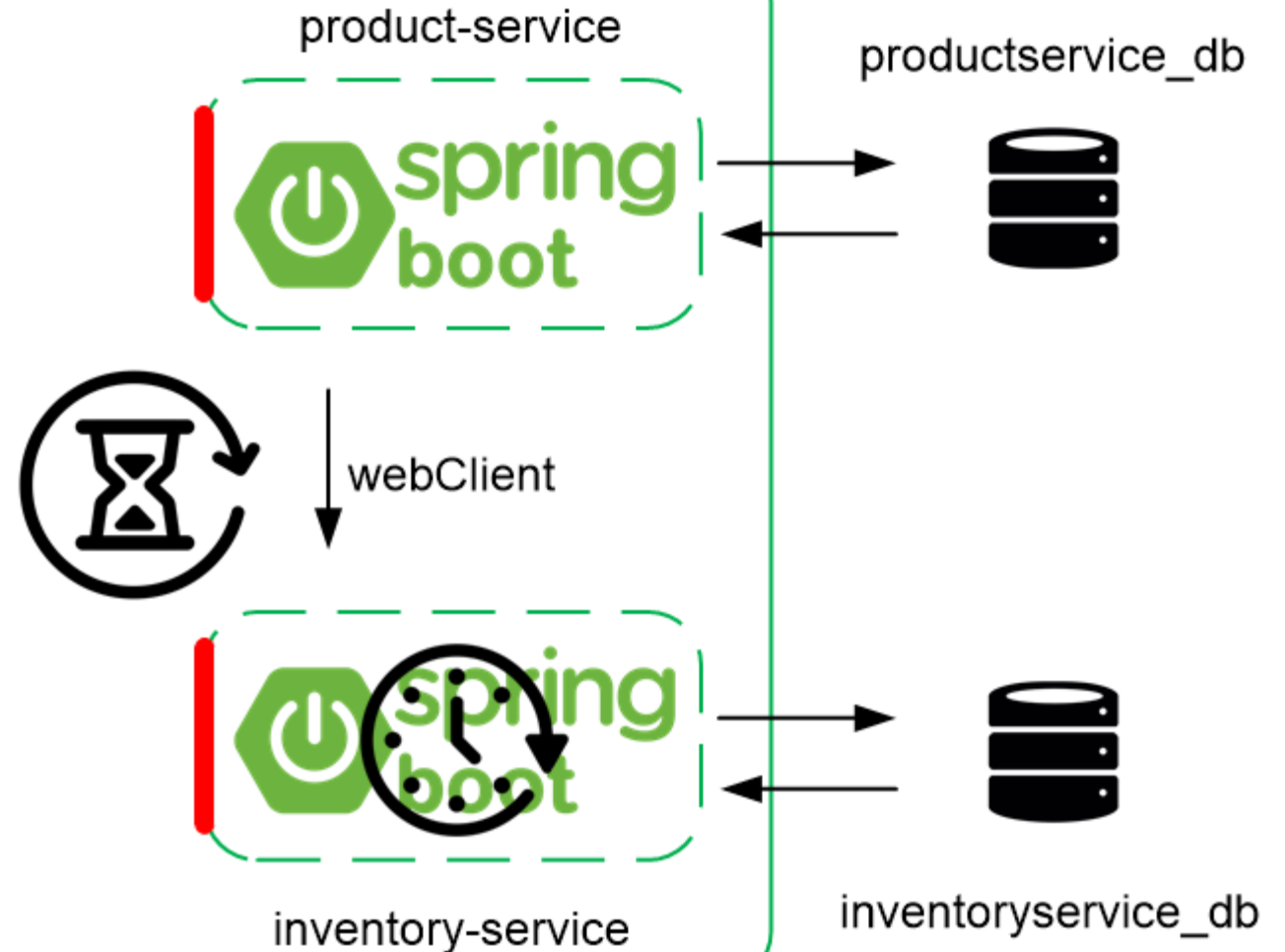
Specifies a
“timeout-duration”
for the request



Time Limiter

Specifies a
“timeout-duration”
for the request

If the time is **reached**
without **receiving**
the response.

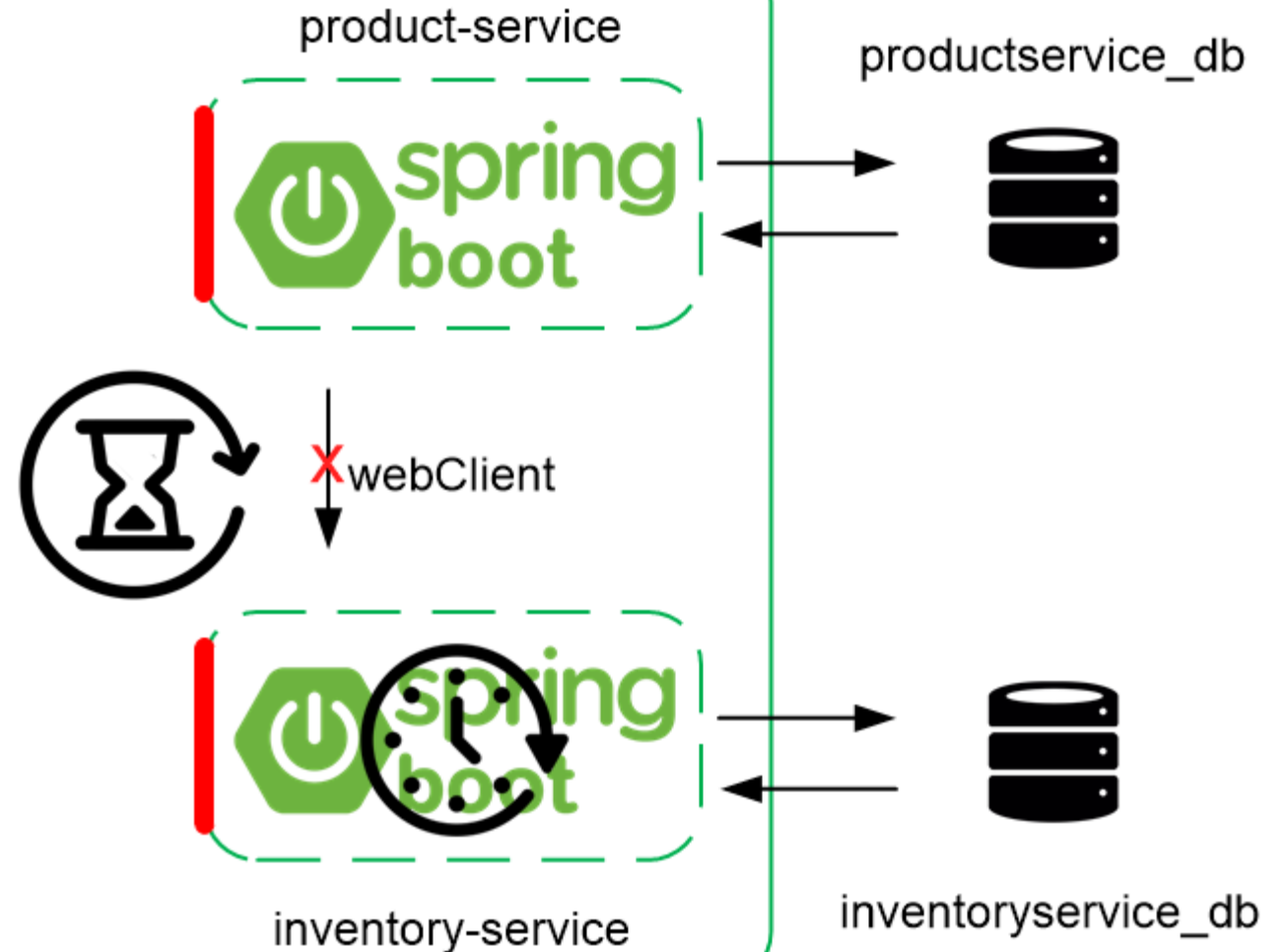


Time Limiter

Specifies a
“timeout-duration”
for the request

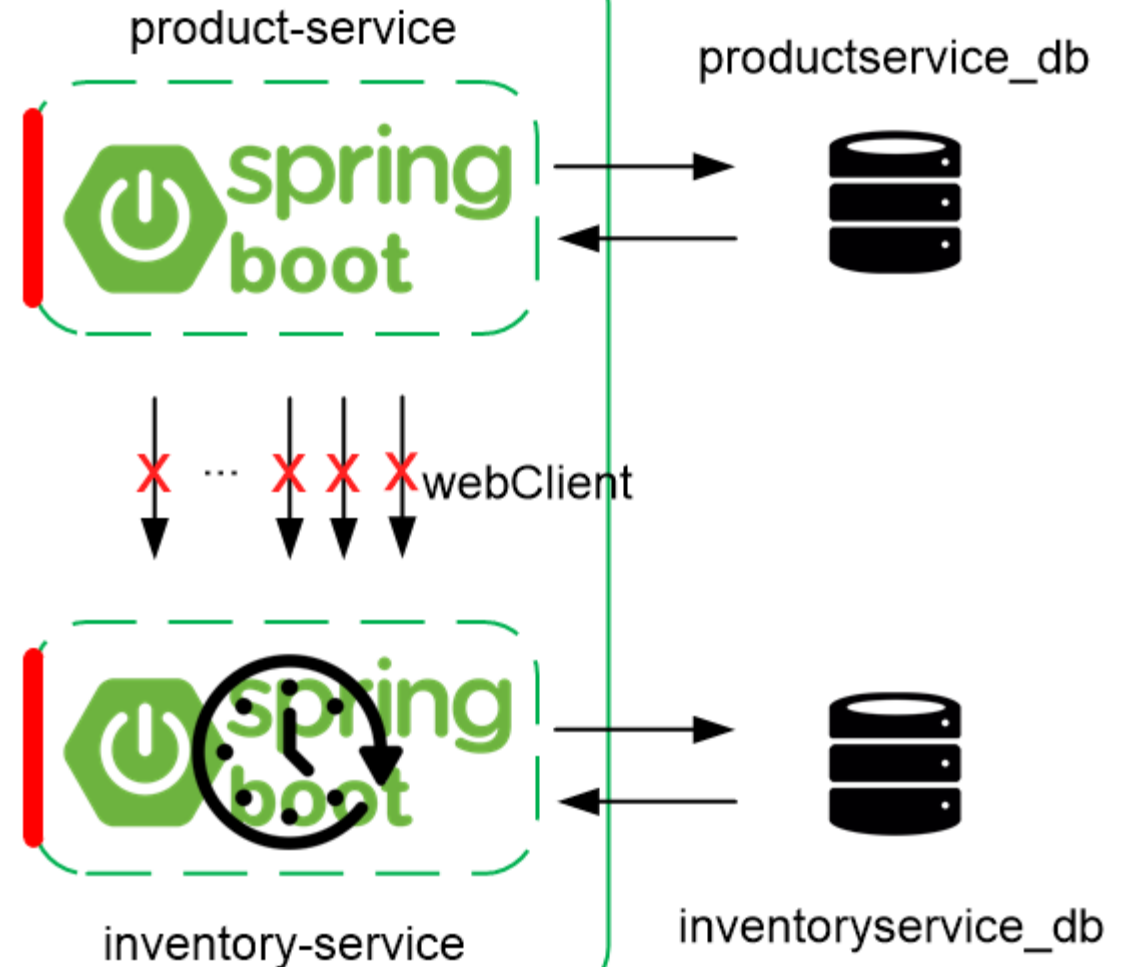
If the time is **reached**
without **receiving**
the response.

The request is
terminated



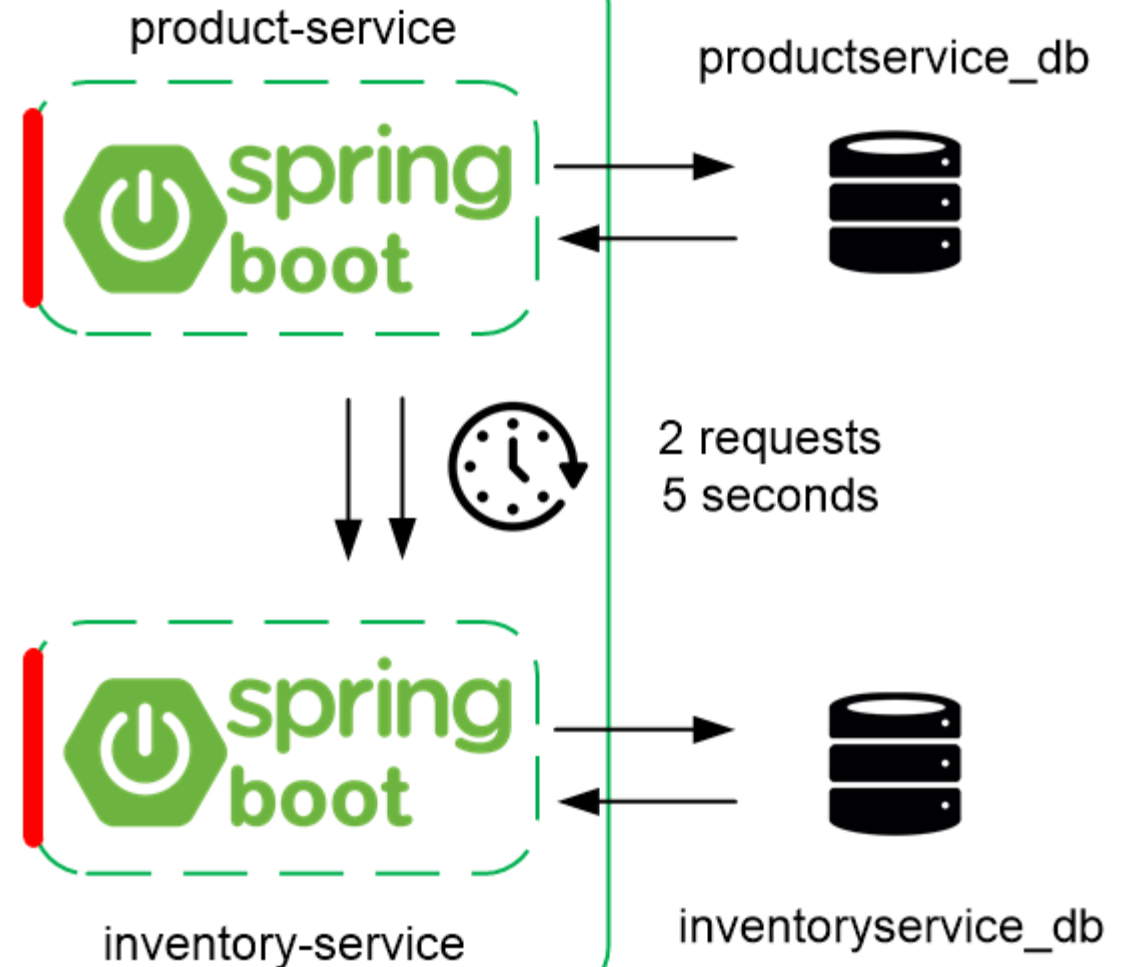
Retry

Repeated after a
"wait-duration"
until the
"max-attempts"
is reached



Rate Limiter

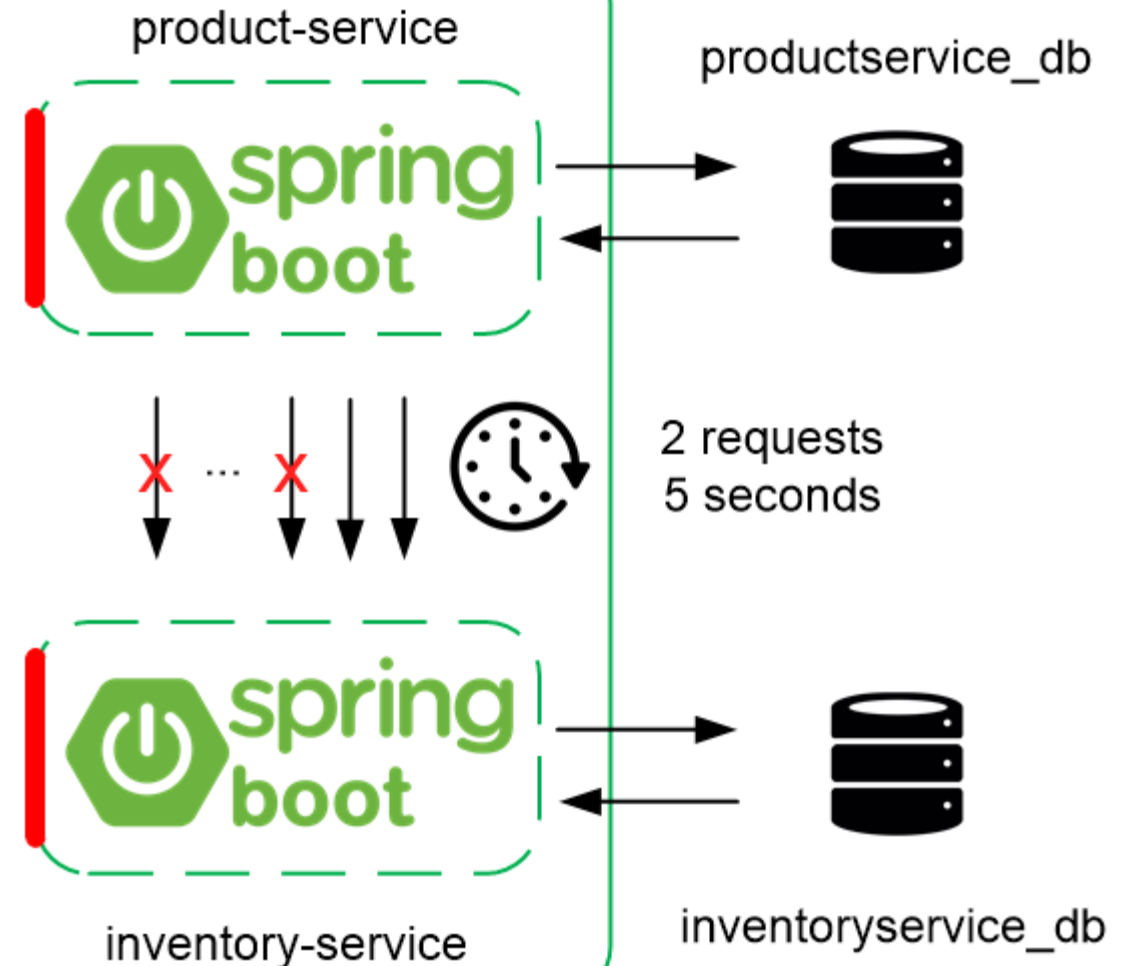
Rate Limiter specify the **max number** of “**requests**” that be send within a **period of time**.



Rate Limiter

Rate Limiter specify the **max number** of “**requests**” that be send within a **period of time**.

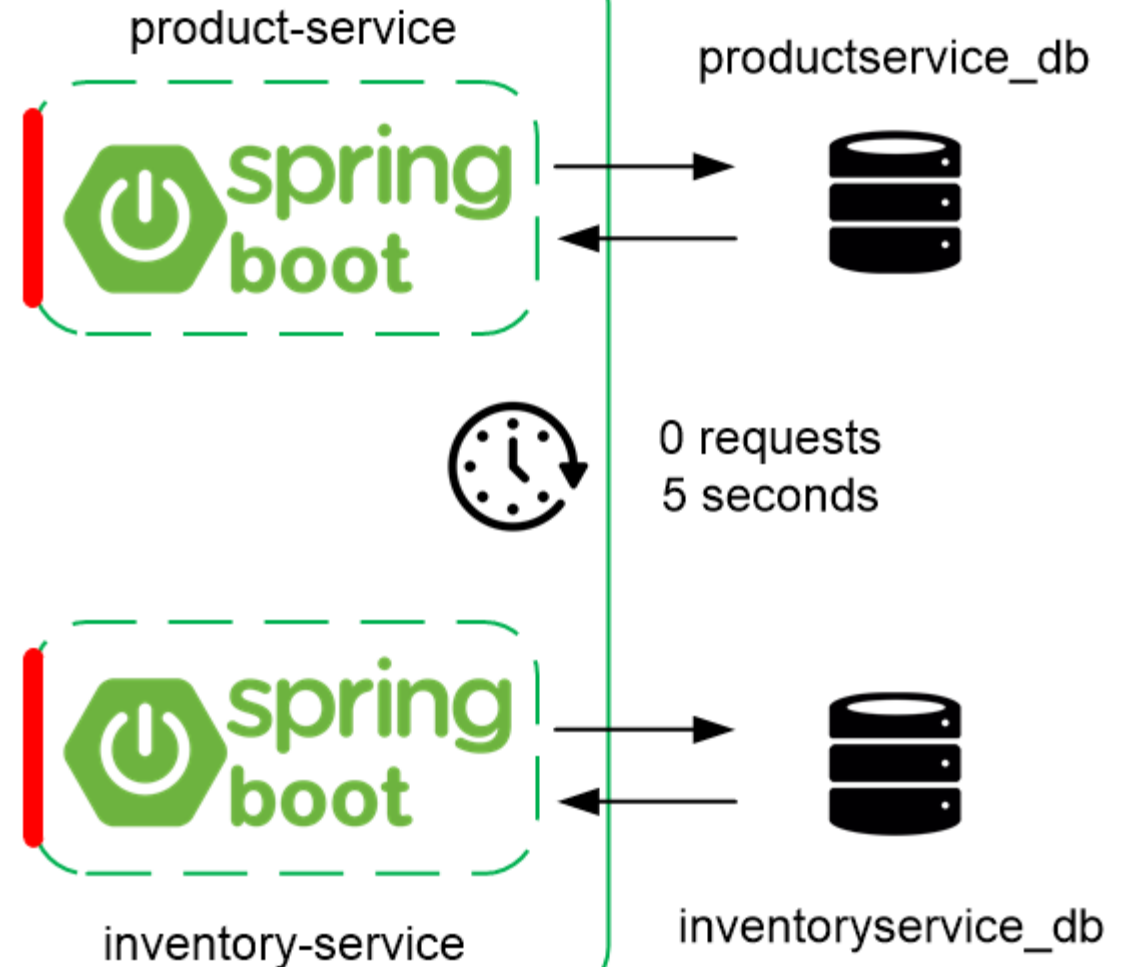
If the max request number has been reached. No further requests can be sent until the next **period of time starts**.



Rate Limiter

Rate Limiter specify the **max number** of “**requests**” that be send within a **period of time**.

If the max request number has been reached. No further requests can be sent until the next **period of time starts**.



Practical session 6

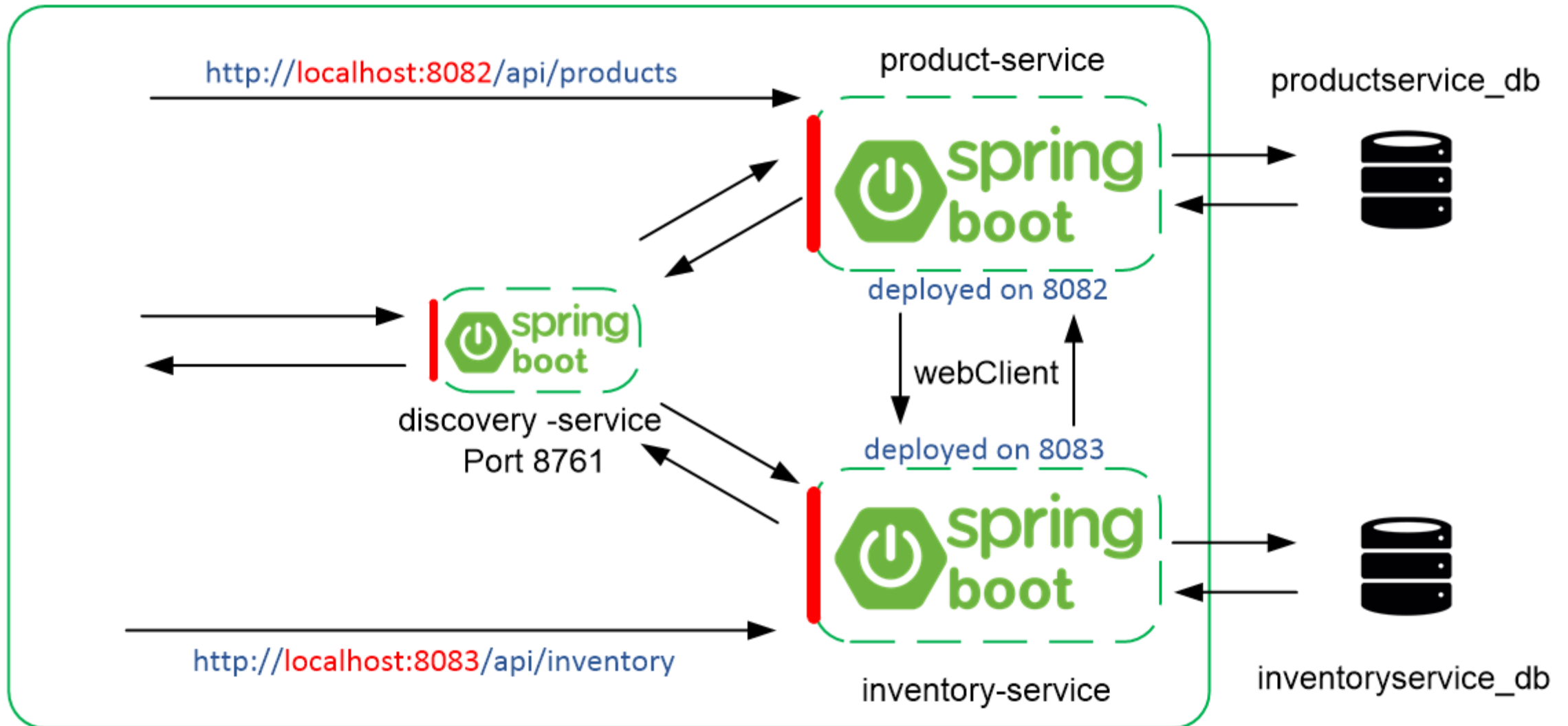


API Gateway + Resilient microservices patterns

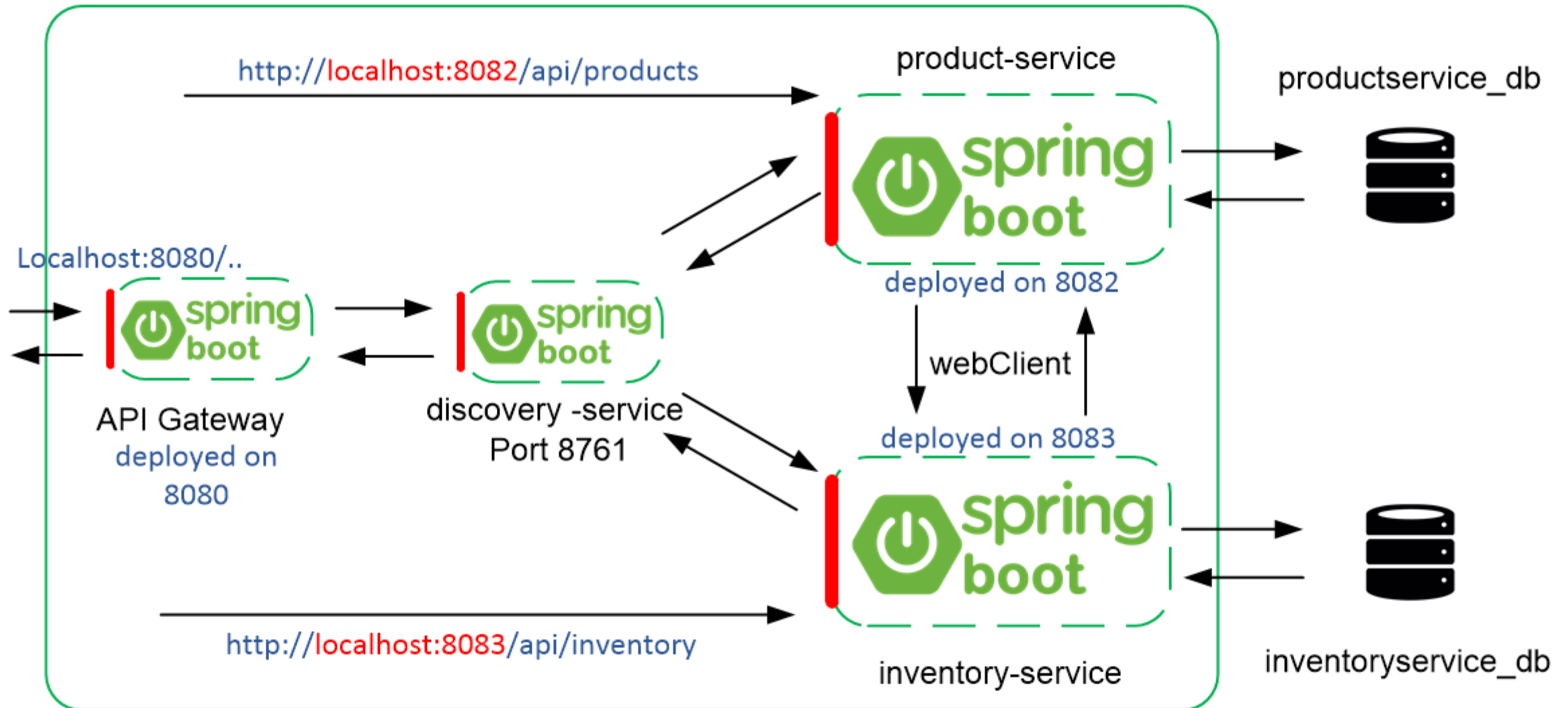
The project you will clone is composed of four different Spring Boot projects that represent two different microservices, a discovery server (the same as last week), and a service to work as our API Gateway. We will create the **API Gateway** and **register** the **services** with it.

You will use the same cloned project to implement 4 different patterns that can be used to design a resilient microservice architecture.

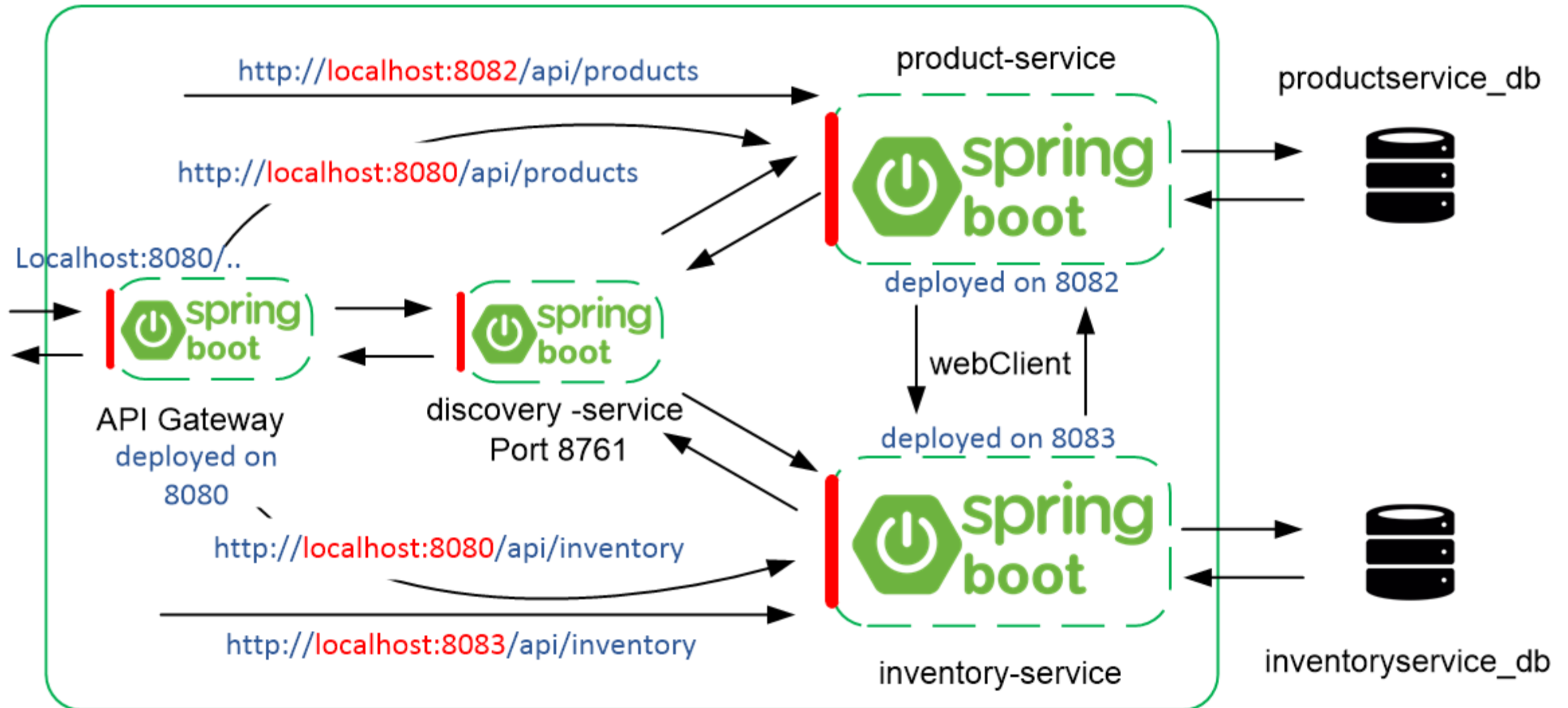
API Gateway



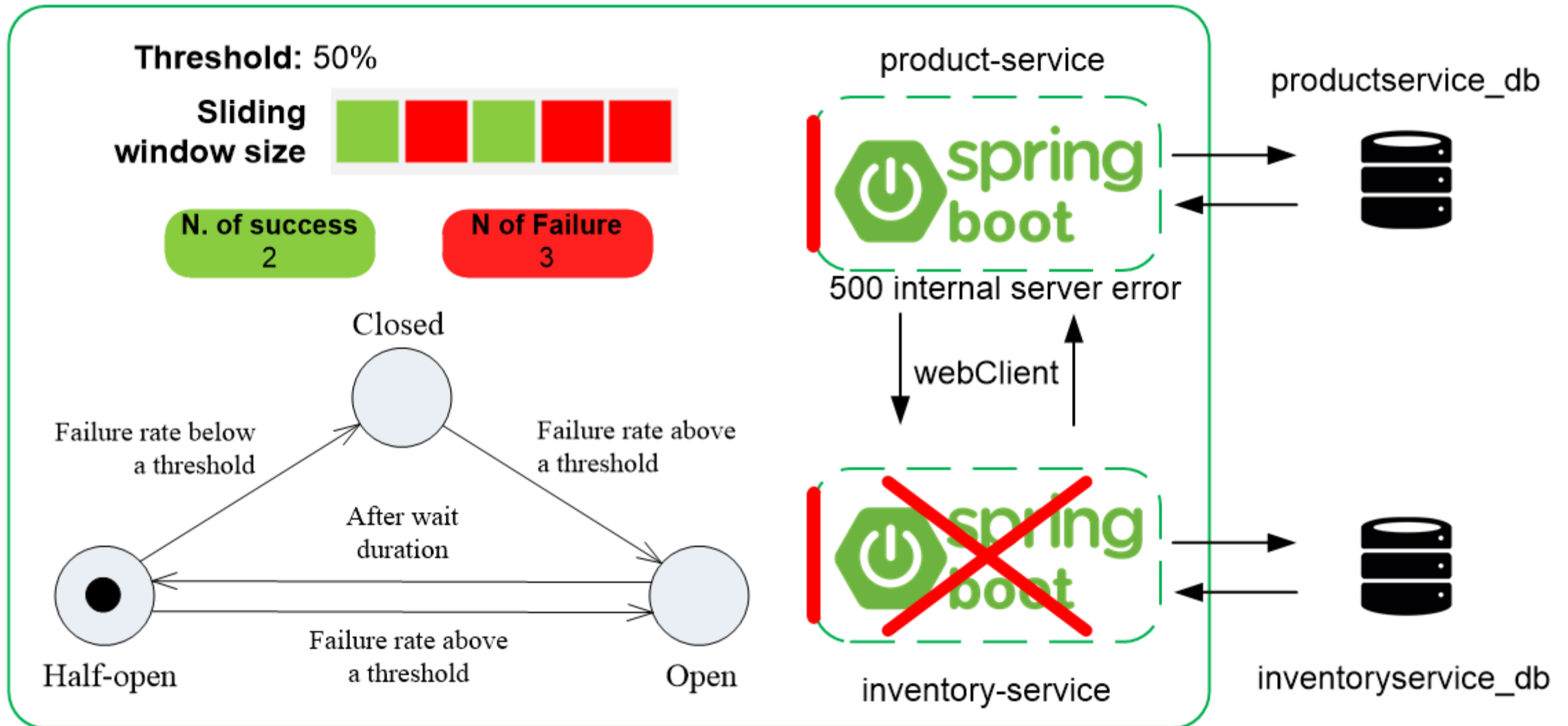
API Gateway



API Gateway



Resilient microservices patterns



Resilient microservices patterns

Threshold: 50%

Sliding
window size



N. of success
2

N of Failure
3

Fault tolerance library designed for functional programming

Build passing nexus v2.0.2 nexus v2.1.0-SNAPSHOT license ASF2

coverage 85.8% maintainability A reliability A security A vulnerabilities 0 bugs 0

Hystrix: Latency and Fault Tolerance for Distributed Systems

oss lifecycle maintenance build error maven central 1.5.18 License Apache 2

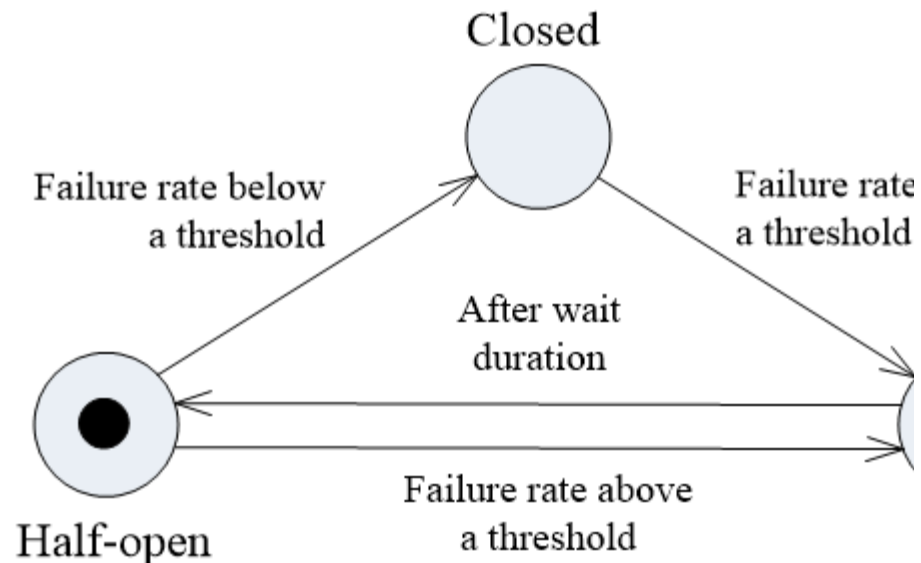
Hystrix Status

Hystrix is no longer in active development, and is currently in maintenance mode.

Open

inventory-service

inventoryservice_up





UNIVERSITY OF TARTU

**Thank You
for your attention**

Mohamad Gharib
mohamad.gharib@ut.ee



unitartu



tartuuniversity

